

PROJECT NEARFALL

CAKE for Midnight: Scalable Proving,
Threshold Signing, Intent Settlement

*Outsourced proofs for retail scale, MPC custody for cross-chain reach,
folding acceleration for throughput. Midnight's co-processor for cross-chain settlement.*

Document Version	4.2 (Final)
Status	Ready for Technical Review
Scope	Merkle Commitment, Strict Sequential Nonces

READY FOR TECHNICAL REVIEW

Architecture complete. Security model defined.
Awaiting gate validation and audit preparation.

Table of Contents

Part I: Executive Summary & Introduction

- 1.0 Introduction
- 1.1 What This Document Is
- 1.2 The Core Requirements
- 1.3 CAKE Framework and Intent Standards
- 1.4 What Must Be True
- 1.5 Unknowns That Decide the Project
- 1.6 The NEAR Experiment
- 1.7 Timing
- 1.8 The Counterargument
- 1.9 Economics
- 1.10 Governance and Operator Model
- 1.11 What Success and Failure Look Like
- 1.12 Document Navigation

Part II: Technical Architecture

- 2.1 Recursive SNARK Architecture
- 2.2 System Architecture
- 2.3 End-to-End Data Flow
- 2.4 Midnight Integration Technical Requirements

Part III: Implementation Plan

- 3.1 Workstreams, Readiness Gates, and the Feasibility Phase
- 3.2 Workstream 1: TEE + Attestation Bootstrap
- 3.3 Workstream 2: External Chain Coordination
- 3.4 Workstream 3: Midnight Integration
- 3.5 Workstream 4: Production Hardening
- 3.6 Sirius: Folding Infrastructure

Part IV: MPC and Signing Infrastructure

Part V: Security Analysis

Part VI: Risk Management

- 6.1 Risk Overview and Framework
- 6.2 Cross-Chain Atomicity Protocol

6.3 Risk Register

6.4 Anti-Rollback Mechanism

6.5 Residual Risk Acknowledgment

*Outsourced proofs for retail scale, MPC custody for cross-chain reach, folding acceleration for throughput.
Midnight's co-processor for cross-chain settlement.*

Part I: Executive Summary & Introduction

1.0 Introduction

Alice wants to swap ETH for NIGHT without showing the order until it's already done. She posts an intent on Ethereum. A solver matches her with Bob. The match settles on Midnight. The solver gets paid only after finality.

That story breaks today for concrete reasons.

The signatures don't line up. Midnight authorizes with BIP-340 Schnorr. Ethereum expects ECDSA. They are not interchangeable. NEAR Chain Signatures gives you threshold ECDSA, but it does not give you BIP-340. Without a separate threshold signing path, Midnight can't safely authorize actions on external chains. This is not an optimization. It's a hard incompatibility.

Proving is not retail-grade. Generating a Midnight proof requires 8GB+ RAM and tens of seconds to minutes. That sounds like an implementation detail. It's not. It's the difference between "privacy for everyone" and "privacy for people who own gaming laptops." On a budget Android phone—roughly 40% of mobile users—you don't get slower proofs. You get no proofs. That's most of the world excluded from what's supposed to be permissionless infrastructure.

The obvious workaround destroys the point. You can outsource proving to a server, but if that server sees plaintext intents or witness material, you've built a privacy system that leaks the thing users came for. A proving server with plaintext access is a surveillance point.

NearFall exists to close exactly these gaps:

- **BIP-340 threshold signing** for Midnight (FROST MPC path)
- **Outsourced proving** without exposing plaintext (confidential compute boundary via TEE)
- **Batch economics** so verification cost doesn't scale linearly with users (folding/aggregation path)

That's the whole claim: make cross-chain intent settlement *possible* for Midnight, and make it *usable* for retail.

1.1 What This Document Is

This is an engineering and deployment-readiness document: architecture, integration requirements, security model, gates, and risk management. It is written for people deciding whether NearFall can serve as Midnight's cross-chain settlement co-processor.

What it does not cover: Business model, go-to-market, regulation, or client SDK design.

Deliberate constraints on v1:

Constraint	Rationale
Midnight is the truth anchor	NearFall targets cross-chain intent settlement, not "privacy for everything everywhere"
TEEs provide confidentiality, not correctness	ZK proofs and chain verification handle correctness; a compromised TEE leaks privacy but does not produce invalid settlements
ERC-7683 compatibility required	We prove compatibility through roundtrip execution, not by inventing a new intent standard

Terms, used narrowly:

- **Intent:** A user's desired cross-chain action, submitted in encrypted form until settlement.
- **Confidential:** Pre-trade privacy (hide order details before execution) and post-settlement privacy (Midnight state remains private).
- **Settlement:** The protocol that ensures matched intents execute atomically (or unwind) across chains.

1.2 The Core Requirements

Three requirements shape NearFall's architecture. Each is non-negotiable; failure on any one breaks the system.

1.2.1 Cross-Chain Atomicity

Cross-chain settlement has no shared transaction coordinator. Midnight and Ethereum finalize differently and can't run a native two-phase commit together. So the system needs a protocol that neither chain can violate unilaterally.

Think of two safes. Safe E (Ethereum) holds Alice's ETH. Safe M (Midnight) holds Bob's NIGHT. Neither safe should open permanently unless there is evidence the other side has committed.

Failure modes that must not exist:

State	Why It's Unacceptable
Alice has neither ETH nor NIGHT	She lost funds; the system stole from her
Bob has both ETH and NIGHT	He got free money; the system printed value
Alice sent, Bob received, but settlement reverted	Double-spend; atomicity violated

NearFall's intended shape:

- Escrow on both sides before settlement
- Release gated on finality, not inclusion
- Timeout-bounded abort so users can reclaim without NearFall's cooperation if the system stalls

The invariant: Every lock either commits to a completed swap or can be refunded by the owner after bounded timeouts. No state exists where a user has lost access to both sides. (Detailed protocol in Part VI, §6.2.)

1.2.2 Retail-Grade Proving

Midnight's ZK system produces strong privacy guarantees, but the proving cost excludes most users:

[Assumption — device capability distribution]

Device Class	RAM	Proving Capability
Gaming laptop / workstation	16GB+	Full local proving (minutes)
Standard laptop	8GB	Marginal; may fail on complex proofs
Budget Android phone	2-4GB	Cannot prove
Browser environment	Variable	Cannot prove reliably

The "40% of mobile users" figure is an estimate based on market share of sub-\$200 Android devices. These users cannot generate Midnight proofs locally. Without proof outsourcing, they cannot use Midnight for private settlement.

Validation: Device capability distribution will be validated via Midnight network telemetry and user surveys before production capacity claims. These figures are motivating assumptions, not validated measurements.

The outsourcing dilemma: A proving server must see witnesses to generate proofs. Witnesses contain the private data users want to hide. Naive outsourcing leaks everything.

NearFall's solution: Prove inside a TEE. The operator runs the hardware but cannot extract memory contents. The proof server sees witnesses only inside the enclave; the operator sees ciphertext and attestation tokens. This is not perfect security—TEE side-channels exist—but it's categorically better than plaintext outsourcing.

1.2.3 BIP-340 Threshold Signing

Midnight requires BIP-340 Schnorr signatures. Ethereum requires ECDSA. NearFall must sign transactions on both chains.

Chain	Signature Scheme	MPC Protocol Required
Midnight	BIP-340 Schnorr	FROST (threshold Schnorr)
Ethereum	ECDSA secp256k1	Threshold ECDSA (NEAR codebase)

The incompatibility: NEAR Chain Signatures provides production-tested threshold ECDSA. It does not provide BIP-340 Schnorr. The NEAR codebase cannot be "configured" to produce Schnorr signatures; the cryptographic protocols are fundamentally different.

NearFall's solution: Operate two signing paths:

- Fork NEAR Chain Signatures for Ethereum (threshold ECDSA)
- Implement FROST MPC for Midnight (threshold Schnorr)

This is the critical path risk. FROST implementations exist but lack large-scale production deployments comparable to threshold ECDSA. Gate G3 validates this path with 100 consecutive BIP-340 signatures accepted on Midnight testnet.

1.3 CAKE Framework and Intent Standards

1.3.1 CAKE Layer Mapping

Chain abstraction requires four layers to mature together. The weakest layer caps the entire system. NearFall addresses all four:

Layer	What Must Be True	NearFall Mechanism
Application	Users can authorize an action without revealing the order to operators/solvers before settlement	Encrypted intents; decryption and policy enforcement inside attested TEE
Permission	No single operator can unilaterally move custody funds	Threshold MPC signing with typed requests + monotonic nonce policy
Solver	Matching/execution cannot depend on privileged visibility into user order flow	Matching inside TEE; batch manifests committed and auditable
Settlement	Settlement validity is enforced by the chain, not by an off-chain claim	Midnight verifies proofs and anchors <code>state_root</code> ; external-chain release gated on finality + membership proofs

1.3.2 Intent Standards Compatibility

NearFall must interoperate with the emerging Ethereum intent ecosystem. Ignoring standards has a cost: 70+ protocols have adopted ERC-7683. A NearFall-specific format means solvers must integrate us separately—and they won't.

Primary standard: ERC-7683 (Cross-Chain Intents Standard), co-authored by Across and Uniswap Labs. Defines `CrossChainOrder` structs and settlement interfaces. Settlement-agnostic: any verification mechanism can fulfill orders.

Project validation requirement: NearFall must demonstrate roundtrip intent execution using ERC-7683:

- User creates `CrossChainOrder` on Ethereum (origin chain)
- NearFall TEE receives and decrypts intent
- Settlement engine matches intent, produces ZK witness
- MPC signs fill transaction (ECDSA for Ethereum, BIP-340 for Midnight)
- Settlement proof anchored on Midnight
- Filler repayment released on origin chain after Midnight finality

This roundtrip validates all four CAKE layers in production configuration. (Detailed specification in [§3.1.11.](#))

1.4 What Must Be True

Six findings determine feasibility. Each maps to a gate that produces evidence, not opinion:

Finding	Status	Gate	Implication if Unresolved
BIP-340 incompatibility with NEAR codebase	Confirmed	G3	FROST MPC required; no workaround
FROST production readiness	Unvalidated	G3	Signing path blocked if implementations aren't ready
ProtoGalaxy verifier feasibility	Unvalidated	G0.5	Folding path may require fallback
Curve mismatch (BN254 vs BLS12-381)	Requires resolution	G0.5	Sirius port or native aggregation required
ERC-7683 roundtrip validation	Required	G2	Integration failure if roundtrip doesn't complete
Cross-chain atomicity protocol	Specified	G2	Design in Part VI; implementation validates

The falsifiable check is a single end-to-end trace:

- User creates an ERC-7683 `CrossChainOrder` on Ethereum
- NearFall receives it inside the confidential boundary and matches it
- MPC signs the destination-side actions (BIP-340 for Midnight; ECDSA for Ethereum)
- Midnight verifies the settlement proof and anchors the state transition
- Ethereum release happens only after Midnight finality is observed
- Under adversarial testing—reorgs, restarts, congestion, replay attempts—the system either completes the swap or cleanly unwinds

If this roundtrip completes with the right invariants under hostile conditions, the system works. If any step fails, the corresponding security claim is false.

Threat model snapshot:

Dimension	Assumptions & Goals
Attacker capabilities	Front-running / MEV extraction; compromised single operator; partial MPC collusion (< threshold); TEE side-channels (Spectre/Foreshadow class); L1 chain reorganizations
Confidentiality goals	Pre-settlement intent privacy; post-settlement amounts remain private on Midnight
Integrity goals	Settlement executes exactly as specified or aborts atomically; no cross-chain double-spend; MPC cannot sign unauthorized transactions
Liveness goals	Graceful degradation under partial failures; no single operator can censor intents; multi-cloud TEE failover
Explicit non-goals	No protection against global TEE platform compromise; MPC supermajority collusion ($\geq$ threshold); Midnight L1 consensus failure

Security checks (falsifiable predictions):

Claim	Check	If This Fails...
"TEE hides intents from operator"	Operator with root access cannot extract intent plaintext from memory dumps	Confidentiality model is broken; TEE is security theater
"MPC prevents unilateral signing"	Single compromised MPC node cannot produce valid signature	Threshold assumption is wrong or implementation is buggy
"Folding preserves correctness"	Invalid settlement in batch → decider proof rejected	Folding implementation is unsound; audit required
"Finality-gated release prevents double-spend"	Release on Ethereum before Midnight finality → funds recoverable or release blocked	Atomicity protocol is broken; cross-chain race exists

Deployment modes (cross-chain verification):

Ethereum cannot natively verify Midnight state. Three verification modes exist, with different trust/complexity tradeoffs:

Mode	Mechanism	Trust Assumption	When Used
Mode A	Light-client proof (BEEFY/GRANDPA)	Cryptographic (Midnight consensus)	Mainnet target
Mode B	Optimistic oracle + dispute window	Economic (challenger incentives)	Alternative if Mode A infeasible
Mode C	Trusted attestation from NearFall	Operational (NearFall honesty)	Bootstrap / testnet only

Gate policy:

- **G2 (testnet):** Mode C is acceptable with TVL caps ($\leq \$100K$) and circuit breakers (pause on anomaly)
- **Mainnet launch:** Mode A or B required; Mode C is not acceptable for production TVL
- **TVL gates:** TVL expansion requires verification mode upgrade or additional security controls

Mainnet mode selection criteria:

Criterion	Mode A (Light Client)	Mode B (Optimistic Oracle)
Implementation complexity	High — requires BEEFY/GRANDPA verifier on Ethereum	Medium — requires oracle infrastructure + dispute mechanism
Gas cost per verification	~200-400K gas (proof verification)	~50K gas (attestation) + dispute bond economics
Latency	Finality time + proof generation	Finality time + dispute window (hours)
Trust assumption	Midnight consensus honest	Oracle honest OR challenger exists
Failure mode	Invalid proof rejected; no false positives	False attestation succeeds if unchallenged

Decision rule: Mode A is preferred if gas cost is acceptable for target batch size and proof generation latency fits UX requirements. Mode B is fallback if Mode A proves infeasible due to gas cost or implementation complexity. The decision is made at **G3.5** based on G0.5 benchmarks and G2 testnet experience. G3 validates Midnight integration works on testnet; G3.5 finalizes the verification mode choice.

What "infeasible" means: Mode A is infeasible if (a) BEEFY verifier gas cost exceeds \$50/batch at target gas prices, OR (b) proof generation latency exceeds user-acceptable wait time (target: <5 minutes), OR (c) implementation audit reveals critical issues not remediable before mainnet timeline.

This is not a "pick later" decision. The verification mode determines whether cross-chain release is cryptographically secured or operationally trusted. §6.2.5 specifies the protocol for each mode.

1.5 Unknowns That Decide the Project

Four uncertainties are genuinely decision-grade. They are not "we'll optimize later."

BIP-340 Threshold Signing Maturity

Threshold Schnorr at production scale is under-proven in the environment we need. There are no large-scale Bitcoin FROST deployments to point to. Gate G3 requires 100 consecutive BIP-340 signatures accepted on Midnight testnet under real deployment conditions. If FROST implementations aren't production-ready on our timeline, the signing path is blocked.

Folding Economics on Real Curves

Whether batching reduces cost depends on real performance on BLS12-381, not on optimism. If folding is materially more expensive than projected—say, 2×—break-even volume doubles and the economic model may not survive. Gate G0.5 must produce benchmarks, not estimates.

Curve Compatibility and Verification Placement

Sirius uses BN254; Midnight uses BLS12-381. Different fields, different group orders, different trusted setups. If proofs are generated in one stack and verified in another, the port/aggregation path is core scope, not optional work. You can't hand-wave this; it determines feasibility and timeline.

Demand

Even a perfectly engineered system can be irrelevant. The differentiator here is "the solver cannot see the order," not "the solver promises not to exploit it." Whether users will pay for that stronger guarantee is a market fact you do not get for free from correctness.

The gate sequence validates capability: Can we build threshold signing? Can we make folding economics work? Can Midnight accept the proofs? It does not validate demand. There is no G0 that asks whether anyone wants this badly enough to pay for it. That gap is real and unaddressed.

1.6 The NEAR Experiment

Key framing: fork, not dependency. NearFall does not use NEAR's live network. It forks the NEAR Chain Signatures codebase and operates its own MPC nodes. This gives us production-tested code without external dependencies—we maintain the fork, we run the infrastructure, and NEAR's network changes do not affect us.

What "Fork" Means Operationally

Aspect	NEAR Chain Signatures (Live)	NearFall Fork
Network	NEAR mainnet validators	NearFall-operated MPC nodes
Governance	NEAR protocol governance	NearFall operational control
Codebase	Upstream development	Pinned fork with selective backports
Key custody	NEAR network derived keys	NearFall-provisioned keys
Attestation	NEAR consensus	TEE attestation + MPC threshold

Why Fork Instead of Use

1. **Signing scheme limitation:** NEAR Chain Signatures produces ECDSA. Midnight requires BIP-340 Schnorr. No amount of configuration changes this; the cryptographic protocols are different.

- **Operational independence:** Depending on NEAR mainnet for signing means NEAR network issues become NearFall outages. A fork isolates us from upstream instability.
- **Policy control:** NearFall needs attestation-gated signing policies that don't exist in the NEAR codebase. Forking lets us add them without upstream coordination.

What We Take from NEAR

The NEAR Chain Signatures codebase provides:

- Production-tested threshold ECDSA implementation
- MPC node coordination protocols

- Key derivation and rotation mechanisms

What we add:

- FROST MPC for BIP-340 Schnorr (parallel implementation, not modification)
- TEE attestation integration for signing policy
- NearFall-specific key provisioning

Evaluation criteria for the fork:

Criterion	Measurement	Gate
Signing latency	p50/p95 under load	G0.5
Throughput	Sustained signatures/second	G0.5
Availability	Behavior under partial node loss	G1
Policy binding	Attestation-gated sessions work	G1

1.7 Timing

Intent standards and solver plumbing harden quickly. Once a workflow becomes the default—public order flow, reputation-based privacy, existing settlement assumptions—adding a privacy-preserving settlement anchor later becomes an integration problem against an installed base that has no intrinsic incentive to accommodate you.

The numbers suggest the window is now:

- ERC-7683 went from proposal to 50-70+ protocol commitments in under a year
- CoW Swap recorded ~\$9B in July 2025
- Across runs ~\$46M daily
- The solver-mediated execution layer is forming

The timing claim is not "now or never." It's narrower:

> It's easier to become part of the standard workflow while the workflow is still being chosen than after it has become invisible infrastructure.

That's the only "why now" you need.

1.8 The Counterargument

Private routing already exists. High-value flow avoids the public mempool. Solvers clear large volumes today without ZK settlement proofs on a private chain. Why build a complicated system?

Answer: Private routing hides transactions from the public mempool. It does not hide them from the solver. The solver sees amounts, counterparties, trade direction. For many users, that's fine; they accept reputational trust. Solver reputation matters, and for most flows it's probably enough.

NearFall is only justified if there is a meaningful segment that wants privacy *against the solver itself*, not just against front-runners. That segment may be large enough—or it may not.

The architecture answers "can we build it?" The deployment decision still needs evidence that the stronger privacy property is worth paying for.

1.9 Economics

At ~10bps take rate—consistent with comparable intent venues—break-even occurs around \$30M daily volume. CoW Swap's July 2025 record month averaged roughly \$290M/day. NearFall's break-even is about 10% of a leading venue's daily volume.

That's non-trivial. It requires meaningful adoption, not a few enthusiasts. But it's not absurd for infrastructure with differentiated security properties—**assuming the differentiation matters to enough users.**

That assumption is unvalidated and should be tied to a measurement plan before significant resource commitment.

Economic Model Components (Summary)

Component	Status	Notes
Take rate	Target: 5-15bps	Competitive with existing intent venues
Solver incentives	Deferred	Requires solver economics analysis
Fee distribution	Deferred	Protocol fee vs. solver fee vs. operator fee
Filler repayment	Specified in §6.2	Repayment after Midnight finality + membership proof
MEV policy	Partial	See below

MEV and Ordering Policy

The TEE executes matching; the operator cannot see plaintext intents. This eliminates *operator* MEV extraction. However, ordering within a batch still creates MEV opportunities:

Ordering policy (v1): First-in-first-out (FIFO) by TEE receipt timestamp. The TEE timestamps intents at decryption; batch ordering follows timestamps.

Why FIFO: Simplicity and auditability. The batch manifest includes timestamps; ordering can be verified against the manifest. More sophisticated ordering (e.g., uniform random, commitment-reveal) adds complexity without clear benefit at v1 volumes.

Residual MEV risk (EC-4): A malicious TEE operator who compromises the enclave could reorder intents for MEV extraction. This is addressed by the same controls that address TEE compromise generally: MPC policy enforcement, on-chain verifiers, and the residual risk acknowledgment in §6.5.

Solver Integration Economics

Solvers integrate with intent venues when expected profit exceeds integration cost. NearFall's solver economics are not yet specified in detail. Key questions for solver integration:

Question	Current Answer
What's the solver's margin?	Solver captures spread between intent price and execution price, minus fees
How does filler repayment work?	Filler repaid on origin chain after finality proof (§6.2.5)
What's the capital lockup period?	Settlement time + finality observation (~15-30 minutes target)
Why integrate vs. other venues?	Privacy differentiation; some order flow prefers confidential execution

Economic design document: A separate economic design document will specify fee structures, solver incentive analysis, and capital efficiency optimization. This is deferred to post-G2 when testnet data informs parameter selection.

1.10 Governance and Operator Model

NearFall requires multiple independent operators for MPC threshold security. This section summarizes the governance model; operational details are specified in Part III ([§3.1.8](#)) and Part IV ([§4.4.1](#)).

Operator Requirements

Requirement	Specification
Minimum operators	5 independent entities for 3-of-5 threshold
Independence	No two operators under common legal control or shared infrastructure
Jurisdiction diversity	Operators distributed across ≥ 3 legal jurisdictions
Technical capability	Ability to operate MPC node, respond to incidents, participate in key ceremonies
Commitment	SLA for availability, incident response, and ceremony participation

Governance Actions

Action	Authority	Mechanism
Measurement allowlist update	Operator multisig (3-of-5)	On-chain governance or off-chain signature aggregation
Emergency measurement revocation	Any operator (break-glass)	Immediate; requires multisig to re-enable
TVL cap adjustment	Operator multisig (3-of-5)	Requires justification and risk assessment
Protocol parameter change	Operator multisig with timelock	48-hour delay for non-emergency changes
Key rotation	All operators (ceremony)	Requires n-of-n participation for new key generation
Break-glass activation	Committee (3-of-5)	On-chain proposal with defined trigger conditions

What's Not Specified

Gap	Status
Operator selection criteria	Deferred — requires legal and operational framework
Legal structure	Deferred — depends on jurisdiction and entity type
Slashing mechanism	Deferred — requires economic design
Operator compensation	Deferred — requires fee distribution model
Dispute resolution	Deferred — requires governance framework

Governance design document: A separate governance design document will specify operator selection, legal structure, and dispute resolution. This is deferred to pre-mainnet preparation when operator candidates are identified.

1.11 What Success and Failure Look Like

If NearFall works: Midnight becomes a settlement anchor for confidential intent flow: encrypted intents in, verifiable settlements out, and no single operator who can both observe and steal.

If it fails—technically, economically, or because adoption never materializes—Midnight remains correct but isolated: able to settle private state, but not positioned to capture cross-chain intent flow at retail scale.

The components are not proprietary. FROST implementations exist. TEE infrastructure is available. ERC-7683 is an open standard. A well-funded competitor could assemble the same pieces. What they would have to build is the integration—binding BIP-340 signing, TEE confidentiality, and Midnight-verifiable proofs into a coherent system. That integration is hard, but it's a head start, not a moat.

The advantage is execution speed in a closing window.

This project should be judged on gates and traces, not enthusiasm.

1.12 Document Navigation

Part	Content	Key Decisions
Part I	Executive Summary & Introduction	Problem framing, requirements, threat model
Part II	Technical Architecture	Proof boundary (§2.1), system components (§2.2), data flow (§2.3), Midnight integration (§2.4)
Part III	Implementation Plan	Workstreams, gates, deliverables, audit scope
Part VI	Cross-Chain Protocols & Risk	Atomicity protocol (§6.2), risk register (§6.3), anti-rollback (§6.4), residual risks (§6.5)

Reading paths:

Reader	Recommended Path
Executive / decision-maker	§1.0–§1.10 , then §1.4 (What Must Be True)
Security reviewer	§1.4 (threat model), Part VI (§6.1–§6.5)
Protocol engineer	Part II (§2.1–§2.4), then Part III workstreams
Integration engineer	§2.3 (data flow), §2.4 (Midnight integration), §3.1.11 (ERC-7683 roundtrip)

Part II: Technical Architecture

How to read Part II: This part is an "explained spec." Read it linearly once to understand the architecture, then use the tables as lookup references. Each unit opens with the question it answers and closes with a bridge to the next unit.

Status labels used throughout:

Marker	Meaning
■	Established / Known — validated or directly observable from codebase
■■	Requires work — feasible but needs implementation or validation
■	Blocking — must be resolved before proceeding
■	Open / Unvalidated — decision pending or requires Gate validation
[Target]	Placeholder number pending benchmarks
[Gate G0.5]	Resolved during feasibility validation

Forward references:

Parts I–III reference sections in Parts IV–VI. Part VI (Cross-Chain Protocols and Risk Analysis) is complete and included in this document. Parts IV–V are included as stub sections with key subsections completed (FROST selection criteria §4.2.1, key ceremony requirements §4.4.1); full specification is deferred to implementation phase.

ZK terminology for non-specialists:

Part II uses zero-knowledge terminology that may be unfamiliar. Here are the key terms:

Term	What it means	Analogy
Circuit	A computation expressed as a series of addition and multiplication gates over a finite field. ZK proofs prove you executed the circuit correctly without revealing the inputs.	Like a spreadsheet formula: cells reference other cells, and the final answer depends on hidden input cells.
Constraints	The "size" of a circuit—how many equations the prover must satisfy. 50K constraints is a small circuit; 500K is large. More constraints = slower proving, bigger proofs.	Like the number of rows in a spreadsheet. More rows = more computation.
PLONK	A specific proof system (like RSA is a specific encryption system). Midnight uses PLONK.	—
PLONKish	PLONK plus custom gates (optimized sub-circuits for common operations). Halo2 is a PLONKish implementation.	PLONK with plugins.
KZG	A commitment scheme that lets you prove properties of data without revealing it. Used inside PLONK proofs.	Like a sealed envelope that can be verified without opening.
Decider SNARK	The final proof that "closes" a folding accumulator. You fold cheaply many times, then run one expensive decider proof at the end.	The final exam after many quizzes.
Verifier gadget	A circuit that verifies another proof. Expensive because proof verification involves elliptic curve operations, which translate to many constraints when expressed as a circuit.	A Rube Goldberg machine that checks if another Rube Goldberg machine ran correctly.
Curve (e.g., BN254, BLS12-381)	An elliptic curve used for cryptographic operations. Different curves have different security properties and are not interchangeable—a proof generated on one curve cannot be verified on another.	Like different languages: French and Spanish are both Romance languages, but a French sentence isn't valid Spanish.

Canonical terminology mapping:

This spec uses canonical terms that may appear with different names in contract code. The mapping:

Canonical Term (Spec)	Midnight Contract	Ethereum Contract	Meaning
<code>batch_nonce</code>	<code>lastBatchNonce</code>	<code>settlementNonce</code>	Monotonic counter preventing replay
<code>state_root_prev / state_root_next</code>	<code>currentStateRoot / newRoot</code>	<code>stateRoot / newRoot</code>	Settlement state commitment
<code>proof_artifact</code>	<code>batchProof</code>	N/A	Midnight-verifiable proof object
<code>finality_proof</code>	N/A	<code>finalityProof</code> parameter	Evidence of Midnight finalization
<code>expires_at</code>	N/A	N/A	Intent expiry timestamp (envelope field)

Contract-level names follow language conventions (camelCase for Solidity, snake_case for Rust). Spec-level names are canonical for cross-referencing.

2.1 Recursive SNARK Architecture

This section explains why "one proof per settlement" does not scale, what Midnight's proof system actually supports, and what NearFall must decide before any recursion work is credible: **where the proof boundary lives**. That single decision determines the meaning of "batch proof," the circuit shape, the verification model, and whether folding frameworks like ProtoGalaxy/Sirius are even relevant.

2.1.1 The Scaling Problem

If each cross-chain settlement is proved and verified independently, NearFall pays linear cost in two places: **TEE proving time** and **on-chain verification count**. At moderate volume, this stops being an optimization problem and becomes a hard throughput ceiling.

Concrete load (1,000 settlements/day):

Cost	Calculation	Impact
TEE proving time	~3s per proof × 1,000 = 50 min/day	Consumes TEE capacity
On-chain verifications	1,000 separate proof checks	Gas cost scales linearly
Batch binding	None — proofs are isolated	No cryptographic link between settlements

> **Source for 3s estimate:** PLONK proof generation for ~50K constraint circuits, measured on 8-core Xeon, 32GB RAM. Actual NearFall circuits may differ.

The third row is the structural problem. The first two rows are cost problems — expensive but solvable by throwing resources. The third row is a correctness problem: independent proofs mean the prover could selectively withhold, reorder, or substitute settlements without detection. Each proof attests to one transition; nothing attests to "exactly this batch, in exactly this order, with exactly these outcomes."

The requirement is therefore: one verifiable artifact that commits to the entire batch, with a verification cost that does not grow with batch size.

Batch binding is what recursion, folding, or aggregation provides — but the technique depends on what Midnight can actually verify.

2.1.2 Midnight's Proof System

Before choosing a recursion strategy, we must know what artifact Midnight can accept. NearFall does not get to dictate Midnight's verifier; NearFall must produce what Midnight expects.

Midnight's proof stack (as assumed in this document):

Component	Specification
Proof system	Halo2-based PLONKish (KZG commitments; must match Midnight verifier)
Arithmetization	PLONKish (custom gates, lookup arguments)
Curves	BLS12-381 (pairing) + JubJub (embedded Edwards)
Commitment scheme	KZG
Compilation path	Compact → ZKIR → Halo2 circuits

How Midnight verifies contracts:

Midnight does not have a "call `verifyProof()` inside a contract" pattern. The model is different:

- A Compact contract compiles to a circuit
- Executing the contract produces a proof
- `pallet-midnight` verifies the proof as part of block validation

The verification model constrains integration. NearFall cannot simply "submit a proof" to an arbitrary endpoint. NearFall must either:

- Make settlement logic a Compact contract (and let Midnight verify it natively), or
- Embed a verifier gadget inside a Compact contract (and pass an external proof as witness)

These are the two coherent integration paths. They are not interchangeable — they imply different artifacts, different constraint budgets, and different definitions of "batch proof."

2.1.3 The Proof Boundary Decision

The proof boundary is the fork in the road. Until this decision is made, "recursive SNARK architecture" is a label without content.

Disputable claim: Path A (native contract) should be the default.

Argument chain:

Premise	Status	Evidence
P1: Path A has ~10x fewer constraints (~50K vs ~500K+)	■ Established	Circuit analysis
P2: Fewer constraints → faster proving, simpler audit, smaller attack surface	■ General principle	—
P3: Path B's benefit ($O(1)$ IVC) matters only if Path A is insufficient	Conditional	Depends on throughput test

Conclusion: Choose Path A unless Path A fails the throughput test at Gate G0.5.

Strongest objection: "Path B is more powerful; why not build it from the start?"

Response: Power unused is complexity incurred. Path B requires implementing a PLONK verifier in Compact (~500K constraints), auditing that implementation, and maintaining it. The burden of proof is on Path B to justify its complexity—not on Path A to prove it can scale to arbitrary load.

Stakes: If we choose Path B unnecessarily, we add significant implementation complexity and a permanent audit surface. If we choose Path A and it fails, we pivot to Path B with known scope impact.

Path A: Settlement Logic as Native Contract (recommended starting point)

The settlement transition logic lives directly inside a Compact contract circuit. Midnight verifies the resulting proof exactly like any other contract execution.

Property	Implication
Proof artifact	Standard Compact contract proof
Constraint budget	Settlement logic only (~10K–50K constraints) [Target]
Batching mechanism	Relies on Midnight-native aggregation (if available)
Verifier complexity	None beyond standard pallet-midnight
Curve constraints	Must use BLS12-381/JubJub (Midnight-native)

Why start here: Path A avoids embedding a PLONK verifier inside a circuit. If Midnight's native aggregation can batch enough settlements to meet throughput targets, Path A is simpler, faster to implement, and has a smaller attack surface.

Path B: Verifier Gadget Inside the Contract

The contract circuit contains a verifier gadget that checks an externally generated proof (a folded accumulator or compressed batch proof). The contract proof attests to "I verified another proof."

Property	Implication
Proof artifact	External IVC/folding artifact + contract proof wrapping it
Constraint budget	Settlement logic + verifier gadget (~500K+ constraints) [Target]
Batching mechanism	True IVC folding — arbitrary batch sizes, $O(1)$ verification
Verifier complexity	PLONK verifier implemented in Compact/Halo2
Curve constraints	Same as Path A, plus verifier circuit must handle curve arithmetic

When Path B is necessary: If Path A cannot meet throughput/latency targets — for example, if Midnight's aggregator adds too much latency or has batch size limits — Path B provides true IVC with $O(1)$ final verification regardless of batch size.

The tradeoff: Path B adds ~10x constraint overhead (verifier gadget) and requires implementing/auditing a PLONK verifier in Compact. Do not choose Path B unless Path A demonstrably fails.

Path B audit burden: A PLONK verifier circuit at ~500K constraints is a massive attack surface. If we take Path B, the verifier circuit must be either (a) derived from a formally verified reference implementation, or (b) subjected to a dedicated cryptographic audit before mainnet deployment. "Build and ship" is not acceptable for this component—a bug in the verifier circuit breaks the entire security model. Path B requires a substantial audit investment if necessary.

> **Gate G0.5 (Feasibility):** Choose one path. Measure Midnight aggregator performance. If it meets SLOs (>10 settlements/second sustained, p95 verification latency <5s), commit to Path A. Otherwise, escalate to Path B with explicit scope change.

2.1.4 Aggregation vs Folding (Do Not Conflate)

Before discussing tooling, a clarification. "Aggregation" and "folding" are sometimes used interchangeably. They are not the same technique.

Property	SNARK Aggregation	IVC Folding
Core idea	Verify N independent proofs inside one proof	Update a running accumulator step-by-step
Per-step cost	Aggregator circuit grows with N	Ideally $O(1)$ per fold step
Final artifact	"I verified N proofs"	"I executed N steps correctly"
State binding	Possible but not automatic	Accumulator inherently commits to history
Typical use case	Batch verification for gas savings	Long-running computations, rollups

Consequence for NearFall:

- If NearFall uses **aggregation**, batch size may be constrained by aggregator circuit size
- If NearFall uses **folding**, batch size is limited only by time/memory, not circuit complexity

Protostar vs ProtoGalaxy (why k-folding matters):

Both are folding schemes for PLONKish circuits. The difference is in how many instances you can fold at once:

- **Protostar**: Folds 2 instances per step (binary folding). To fold 100 settlements, you do 99 fold operations in sequence.
- **ProtoGalaxy**: Folds k instances per step (k-folding). To fold 100 settlements with $k=10$, you do 10 fold operations.

Why does this matter beyond raw throughput? **Parallelism**.

With binary folding, each fold depends on the previous fold's output—the operations are inherently sequential. With k-folding, you can prepare k instances in parallel, then fold them together. In a TEE environment where we're already doing parallel witness generation, k-folding lets us overlap work: while the prover folds the current batch, witness generators prepare the next k instances.

The practical constraint k-folding relaxes is **latency under load**. If settlements arrive faster than sequential folding can process them, the queue grows without bound. K-folding absorbs bursts by processing multiple

instances per fold operation.

Gate G0.5 must determine what Midnight's native tooling actually provides. If it's aggregation, understand the batch size ceiling. If it's folding, understand the accumulator format and whether k-folding is supported.

2.1.5 The Curve Reality (Why Sirius Doesn't Work)

Sirius is the only available open-source PLONKish folding implementation with Halo2 integration. It supports custom gates and lookups, which production settlement circuits require. On paper, it matches Midnight's arithmetization. Note: The upstream repository is archived; NearFall must own a maintained fork with pinned dependencies and security patch process.

But curve compatibility is not optional.

Property	Sirius	Midnight	Compatible?
Arithmetization	PLONKish	PLONKish	■ Yes
Commitment scheme	KZG	KZG	■ Same scheme
Pairing curve	BN254	BLS12-381	■ No
Embedded curve	Grumpkin	JubJub	■ No

A proof generated with BN254's KZG cannot be verified by a BLS12-381 verifier. These are different fields, different group orders, different trusted setups. "KZG on different curves" is not interoperable. This is a **blocking incompatibility**.

Why you can't just "swap the curves" (the short version):

Elliptic curve cryptography relies on the specific mathematical properties of each curve—the prime number defining the field, the curve equation parameters, the group structure. Changing curves means:

- Different prime = different arithmetic (like switching from base-10 to base-7 arithmetic)
- Different group order = different valid signatures and commitments
- Different trusted setup = you can't reuse the "public parameters" that make KZG work

It's not like changing from pounds to kilograms where you just multiply by a constant. It's like changing from English to Mandarin—the entire grammar changes, and you have to re-translate everything.

Resolution options (in order of sanity):

Option	Complexity	Risk	Recommendation
1. Use Midnight-native aggregation	Low	Medium — may lack IVC semantics	Try first
2. Port Sirius to BLS12-381/JubJub	High	High — curve-cycle assumptions may not transfer	Fallback if (1) fails
3. Adapter/wrapper layer	Medium-High	High — new trust assumptions	Avoid if possible
4. Build from scratch	Very High	Very high	Last resort

Why porting is hard (not "swap constants"):

Sirius relies on **CycleFold** — a technique that uses a "support curve" to make expensive non-native field operations tractable inside the circuit. CycleFold works because BN254 and Grumpkin form a curve cycle: each curve's base field is the other's scalar field.

BLS12-381 and JubJub do not form a true cycle in the same way. A port likely requires:

- Redesign of non-native arithmetic strategy
- Replacement or restructuring of CycleFold assumptions
- Fresh security analysis
- Fresh performance benchmarking

Porting Sirius is cryptographic engineering, not software engineering. Budget accordingly.

Operational implication: If NearFall adopts Sirius, it must own this maintained fork — security updates, dependency pinning, audit scope expansion. Treat "Sirius adoption" as an explicit multi-year maintenance commitment.

2.1.6 Accumulator Semantics and Pipeline

If NearFall uses folding (Path B or a folding-capable version of Path A), the **accumulator** is the central data structure. It must bind the entire batch history so that the final proof implies correctness of all included settlements.

What folding actually does (the mechanism):

Think of folding like this: instead of verifying 100 proofs one at a time, you maintain a single "running summary" that gets updated each time a new settlement is processed.

Here's the key insight. A normal SNARK proof says: "I did computation X correctly." Verifying it is expensive—elliptic curve pairings, field operations. If you have 100 proofs, you do 100 expensive verifications.

Folding changes the game. Instead of producing a proof for each step, you produce a *claim* and fold it into the accumulator. The fold operation is cheap—a few group operations, no pairings. At the end of 100 steps, you have:

- An accumulator (the running summary)
- A single "decider" proof that says: "This accumulator correctly summarizes 100 valid computations"

You verify the decider once. That single verification tells you all 100 steps were correct.

The cost tradeoff:

- Without folding: 100 proofs $\times$ 100 verifications = $O(n^2)$ work
- With folding: 100 cheap folds + 1 expensive decider = $O(n)$ work

The accumulator is what makes this possible. It carries forward the cryptographic commitment to "everything so far was valid" without requiring you to re-verify everything so far.

Accumulator contents (normative):

```
accumulator = {
  state_root:      bytes32, // Merkle root of settlement state after step i
  batch_commitment: bytes32, // H(batch_id || ordered_intent_hashes)
  execution_digest: bytes32, // H(output_1 || output_2 || ... || output_i)
  step_counter:    uint64,   // Monotonic, prevents replay
  acc_internal:    [opaque]  // ProtoGalaxy-specific: error terms, challenges, etc.
}
```

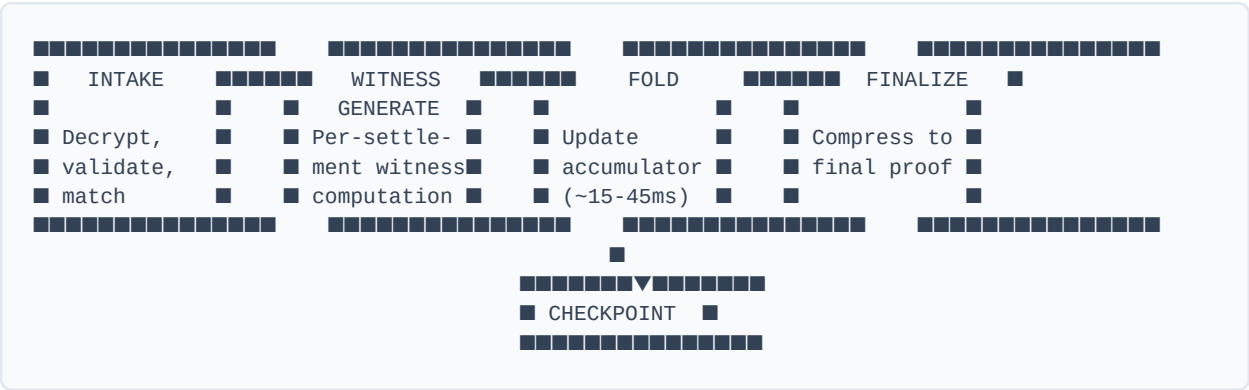
The first four fields are NearFall-specific—they bind the accumulator to our settlement semantics. The last field (`acc_internal`) is the cryptographic machinery that makes folding work: error polynomials, random challenges, commitment openings. Its exact shape depends on which folding scheme we use.

Binding requirements:

What	Why	Verified How
State root	Prevents attaching proof to wrong state	Must match on-chain state
Batch commitment	Ensures exact batch contents	Included in proof's public inputs
Execution digest	Binds outcomes, not just inputs	Committed before signing
Step counter	Prevents replay/reordering	Monotonic, checked on-chain

Critical cross-layer binding: The MPC signing envelope must include `batch_commitment`. This ensures "what gets authorized" equals "what gets proven." See §4.3 for signing payload format.

Pipeline shape:



Batch trigger policy (configurable): Emit a proof artifact every **100 intents or 30 seconds**, whichever comes first. This bounds worst-case latency while allowing batching efficiency.

Checkpoint requirements (folding makes the accumulator crash-sensitive):

Check point	When	Contents	Why
Post-fold	After each successful fold	Accumulator snapshot, processed intent hashes	Recover from mid-batch crash
Pre-finalize	Before finalization	Full accumulator + witness material	Finalization is expensive; don't repeat
Post-sign	After MPC signing	Signed transaction, ready to broadcast	Prevent double-spend on retry

Without post-sign checkpointing, a crash after signing but before broadcast could strand a valid signature. On retry, the system might sign again — creating a double-spend vector.

> **Honest uncertainty:** The "15-45ms per fold" target is based on ProtoGalaxy paper estimates for PLONKish circuits in the 50-100K constraint range. It has not been measured on NearFall circuits or on BLS12-381. Do not use for capacity planning until Gate G0.5 benchmarks are complete.

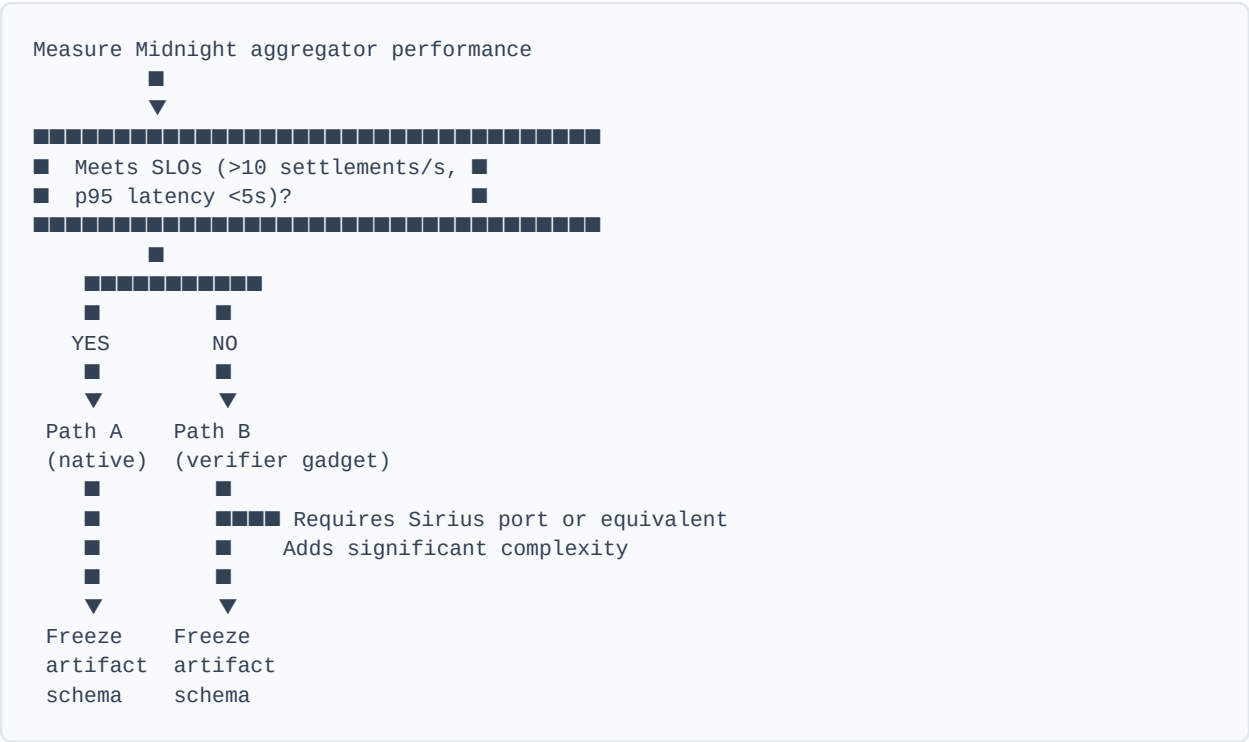
2.1.7 Gate G0.5: What Must Be Resolved Before Implementation

Until the following are fixed, "recursive SNARK architecture" is a placeholder label, not an implementable design.

G0.5 Deliverables:

Deliverable	Question	Acceptable Outcomes
D1: Proof boundary	Does Path A meet SLOs?	Path A selected if SLOs met; Path B selected only if Path A demonstrably fails (explicit scope change)
D2: Curve strategy	Midnight-native or Sirius port?	If native: confirm aggregation meets SLOs. If port: significant additional complexity
D3: Artifact definition	What exact proof object is emitted?	Schema frozen: public inputs, commitment format, verification interface
D4: Benchmark evidence	What are actual proving/verification times?	Measured on target curve stack, in TEE environment, with representative circuits
D5: Proof size constraints	What are the size limits for proofs and transactions?	Measured proof sizes fit within Midnight block/transaction limits; no truncation required

Decision tree:



Requirement: Gate G0.5 must close before downstream work begins. Implementation cannot proceed until this gate passes.

Contingency: What if BLS12-381 folding is 10x slower than BN254?

This is a plausible outcome. BLS12-381 has larger field elements (381 vs 254 bits), which means more expensive field arithmetic. If benchmarks show folding on BLS12-381 is dramatically slower:

- **If Path A still meets SLOs:** Proceed with Path A. Slower folding doesn't matter if we're using native aggregation.
- **If Path A fails and Path B is too slow:** The fallback is to reduce batch sizes and accept higher per-settlement costs. This is a business decision (cost vs. throughput), not an architecture change.
- **If both paths fail SLOs:** Escalate to IOG. The options are (a) accept lower throughput as the launch ceiling, (b) invest in curve-optimized implementations (high complexity, extended scope), or (c) reconsider the Midnight integration entirely.

The "10x slower" scenario does not change the architecture—it changes the operating parameters. The system still works; it just processes fewer settlements per second.

2.1.8 Summary: What This Section Establishes

This section established the scaling problem, the proof boundary decision, and the curve compatibility blocker. The table below captures the key claims and their status.

Claim	Status	Dependency
Per-settlement proofs don't scale (linear TEE + verification cost)	■ Established	—
Batch binding requires cryptographic commitment to ordered history	■ Established	—
Midnight verifies PLONK/KZG on BLS12-381	■ Established	—
Two coherent integration paths exist (Path A, Path B)	■ Established	—
Path A is simpler; Path B is more powerful	■ Established	—
Sirius (BN254) cannot directly produce Midnight-verifiable proofs	■ Blocking	Gate G0.5
Aggregation $\neq$ Folding; different scaling properties	■ Established	—
Path A vs Path B choice	■ Open	Gate G0.5
Curve resolution (native aggregation vs Sirius port)	■ Open	Gate G0.5
Accumulator must bind batch, state, execution, and MPC envelope	■ Required	—
Checkpointing is mandatory if folding is used	■ Required	—

Cross-references:

- Gate G0.5 validation criteria: [§3.1.3](#), [§3.1.7](#)
- MPC signing envelope format: [§4.3](#) (*Part IV — not in scope for Parts I–III*)
- Crash recovery and anti-rollback: [§6.4](#) (*Part VI — complete*)
- Cross-chain atomicity (batch commitment binding): [§6.2](#) (*Part VI — complete*)

What comes next: We now know what recursive proving must produce and why curve compatibility and proof-boundary choices dominate the design space. The next section names the concrete components that

implement this architecture—and the trust boundaries between them.

2.2 System Architecture

Section contents: Layered architecture (§2.2.1) → Core components (§2.2.2) → Trust boundaries (§2.2.3) → Midnight integration (§2.2.4) → MPC layer (§2.2.5) → Chain adapters (§2.2.6) → Checkpoint/recovery (§2.2.7) → Summary (§2.2.8)

NearFall is a privacy-preserving co-processor for Midnight. Users submit signed intents; NearFall executes matching and settlement inside a TEE, produces a proof artifact that Midnight can verify, and requests threshold signatures for the chain transactions required to settle.

The system is built around one rule: **no cleartext intent or witness data leaves the TEE**, and **no single machine holds signing keys**.

Break the first half and you leak user intent data. Break the second and you create a single point of compromise for fund custody. The architecture enforces both by construction.

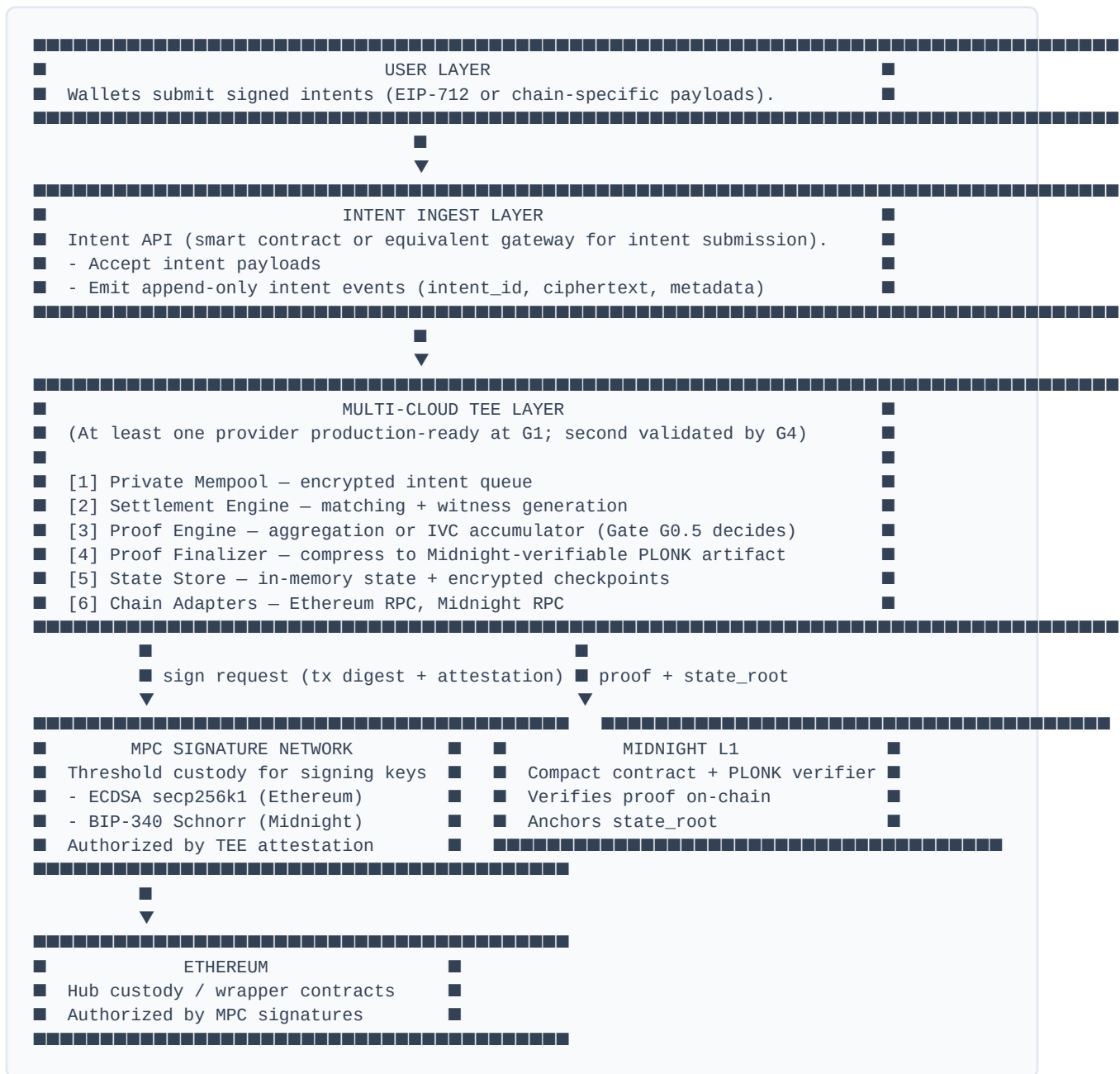
2.2.1 Layered Architecture

The system is organized into five layers, each with distinct trust assumptions. Data flows downward; authorization flows upward through attestation.

How to read the architecture: Two loops meet at two handoff points.

- **Inner loop (TEE):** decrypt → match → prove → checkpoint. Processes intents, produces proofs. Never touches signing keys.
- **Outer loop (MPC + chains):** sign → commit → confirm → checkpoint. Authorizes fund movements, anchors state. Never sees plaintext intents.

The security model requires that neither loop can forge the other's artifacts.



Handoff points in detail:

- **TEE → MPC:** The TEE sends (tx_digest, attestation) to request signatures. The MPC verifies the attestation matches an allowed measurement before signing.
- **Chains → TEE:** Confirmation events flow back to the TEE, which advances its checkpoint only after on-chain confirmation.

Trust model by layer:

Layer	Trust Basis	Failure Mode
User	User's private key	Phishing, key compromise
Ingest	Append-only publication	DoS, censorship
TEE	Hardware attestation	Platform exploit, side-channel
MPC	Quorum honesty (< threshold corrupt)	Collusion, key compromise
Settlement	Chain consensus	Reorg, liveness failure

Each layer can fail independently. Non-cascading failure is enforced by separation of concerns:

- TEE compromise alone should not be sufficient for theft, assuming MPC policy enforcement and on-chain verifiers remain intact
- MPC compromise enables signing but cannot target specific intents (TEE holds plaintext)
- Neither can move funds without the other's cooperation

The MPC layer catches a compromised TEE; the TEE keeps intent data private from MPC nodes.

Cross-cloud attestation interoperability:

GCP Confidential Space, AWS Nitro, and Azure Confidential Computing produce different attestation token formats. They are not interchangeable. This creates a design question: does the MPC layer need to understand three different attestation vocabularies?

The answer is **yes, but with a normalization layer**. The MPC signing policy must verify attestations from any supported cloud provider. In practice:

- Each cloud produces a provider-specific attestation document (GCP: OIDC token with TEE claims; AWS: signed attestation document from Nitro Security Module; Azure: SGX quote or SEV-SNP report).
- A **normalization service** (which can run inside the TEE itself) extracts the relevant claims: code measurement, platform identity, debug status.
- The MPC policy evaluates against normalized claims, not raw tokens.

This is not a trivial integration. Each provider's attestation has different trust roots, different revocation mechanisms, and different claim semantics.

Stakes: If normalization mishandles provider differences, the system fails in one of two ways: (1) MPC accepts invalid attestations → signs unauthorized requests, or (2) MPC rejects valid attestations → system unavailable. Both break the security model. The normalization layer is as critical as the signing policy itself.

Normalization layer requirements (falsifiable):

Requirement	Validation
Unit tests for each provider	Test suite covers GCP, AWS, Azure claim extraction
Negative tests for malformed tokens	Malformed/expired/invalid tokens are rejected with deterministic error
Canonical claims schema	Versioned schema with hash included in gate evidence
Claims extraction correctness	Test vectors for each provider that verify correct field mapping

The normalization layer audit scope is equal to the signing policy audit scope.

> **Open question for Part V:** Should NearFall define a canonical attestation format and require all TEE instances to produce it? Or should the MPC layer remain attestation-format-aware? The tradeoff is complexity in the TEE (canonical format) vs. complexity in the MPC policy (multi-format).

2.2.2 Core Components and Contracts

Five components implement the architecture. Each has a stated purpose and a contract—the invariant it must maintain.

(A) Intent API (Ingest Boundary)

Purpose: Accept intents without exposing them to operators or competing solvers.

Contract: Append-only publication of `(intent_id, ciphertext, metadata)`.

Field	Type	Description
<code>intent_id</code>	bytes32	Unique identifier (monotonic or collision-resistant)
<code>ciphertext</code>	bytes	Encrypted intent body
<code>expires_at</code>	uint64	TTL; used for purge and fail-safe
<code>origin_chain</code>	enum	Ethereum / Midnight / other
<code>policy</code>	bytes	Batch preferences (optional)

(B) Private Mempool (Inside TEE)

Purpose: Queue intents and control disclosure.

Contract: Plaintext exists only inside the enclave. Any outside persistence is encrypted and sealed to the TEE measurement.

(C) Settlement Engine (Inside TEE)

Purpose: Match intents and compute witnesses deterministically.

Contract: For a given `(intent_set, current_state)`, output:

- `transition_witnesses` — per-settlement ZK witnesses
- `state_root_next` — new state commitment
- `batch_manifest` — intent_ids, ordering, fees, expiry outcomes

Batch trigger policy (configurable): 100 intents OR 30 seconds, whichever comes first. An operator "flush" command can also trigger.

(D) Proof Engine + Finalizer (Inside TEE)

Purpose: Produce a Midnight-verifiable proof artifact binding the batch to a state transition.

Gate G0.5 determines the proof boundary:

- **Path A (Native Contract):** Settlement logic proven as contract execution
- **Path B (Verifier Gadget):** Contract proof verifies an external accumulator

Contract: Output `(state_root_next, proof_artifact)` where `proof_artifact` is what Midnight verifies on-chain.

(E) MPC Signature Network (Outside TEE)

Purpose: Sign chain transactions without placing keys in any single runtime.

Contract: A signing request is honored only if:

- Accompanied by a valid TEE attestation token
- Attestation matches expected code measurement
- `batch_nonce == last_seen_nonce + 1` (strict sequential — no skips, no repeats)
- Transaction digest matches expected format

Nonce policy note: MPC enforces strict sequential nonces (`== last + 1`). Resubmission of a failed-but-signed transaction is achieved by re-broadcasting the same signed tx; MPC never re-signs the same nonce.

Protocols and keys (evaluated/selected in Part IV):

- Ethereum: threshold ECDSA on secp256k1 (NEAR codebase fork)

- Midnight: BIP-340 Schnorr on secp256k1 (requires FROST MPC; NEAR codebase cannot do Schnorr)

2.2.3 Trust Boundaries

Security rests on two anchors: TEE attestation for privacy, and threshold MPC for authorization. Neither can move funds alone.

- **TEE attestation** — privacy and correct execution of the batching/proving pipeline
- **Threshold MPC** — no single machine can authorize chain actions

Defense layers:

Layer	Protects Against	Does NOT Protect Against
TEE hardware	Malicious host, memory inspection	Platform exploits (Spectre, Foreshadow)
MPC threshold	TEE signing unauthorized transactions	TEE + MPC threshold collusion
ZK verification	Invalid proofs reaching L1	Valid proofs for malicious transitions
Multi-cloud	Single provider failure	Correlated platform vulnerability

Failure implications stated plainly:

- A compromised TEE can leak intent data and emit malicious transactions — unless MPC attestation checks stop it
- MPC prevents unilateral signing but does not protect against threshold collusion
- Multi-cloud reduces single-provider failure but does not eliminate correlated vulnerabilities

Residual risk floor: An attacker controlling both TEE and MPC threshold can sign arbitrary transactions. No cryptographic defense exists; mitigation is operational (distribute MPC nodes across jurisdictions/operators).

2.2.4 Midnight Integration

NearFall extends Midnight without replacing its core infrastructure. The table shows what NearFall adds versus what stays native.

Capability	Midnight Native	NearFall Extension
Proof system	PLONK/KZG on BLS12-381	IVC proofs verified as contract execution
Signing	User signs ledger transactions	MPC signs on user's behalf
Submission	<code>send_mn_transaction(midnight_tx)</code>	Same API
State model	UTXO-based confidential state	Mirrors state roots via Compact contracts
Fees	DUST	DUST (NearFall pays settlement fees)
Proof generation	Local (8GB RAM, 60-120s)	TEE-delegated (5-15s target)

What NearFall does NOT replace:

- Midnight consensus (GRANDPA finality, ~6s blocks)
- Midnight proof verification (`pallet-midnight`)
- Midnight state model (UTXOs, confidential values)
- Midnight transaction format (`midnight_tx`)

NearFall is a **submission path**, not a fork. Transactions produced by NearFall must be indistinguishable from user-submitted transactions once they reach `pallet-midnight`.

Indistinguishability requirements (falsifiable):

Requirement	Validation
Same transaction format	NearFall transactions use identical encoding as wallet-submitted transactions
Same validation rules	<code>pallet-midnight</code> applies identical validation; no special-case code paths
No NearFall-specific metadata	Transaction contains no fields identifying NearFall as submitter
Mempool acceptance	Reference node mempool accepts NearFall transaction identically to user transaction
Block production	Block producer cannot distinguish NearFall submission from user submission

2.2.5 MPC Layer (NEAR Chain Signatures Fork — Experiment Boundary)

The NEAR Chain Signatures codebase is evaluated for forking. NearFall will operate its own MPC nodes—no runtime dependency on NEAR's network. The evaluation determines whether the codebase is suitable for this purpose.

Property	NEAR Codebase Capability	NearFall Requirement	Status
ECDSA (secp256k1)	■ Native	External chain signing (Ethereum)	Compatible
Ed25519	■ Native (Additive KD)	Substrate chains	Compatible
BIP-340 Schnorr	■ Not supported	Midnight ledger signing	■ Blocking
TEE attestation	OIDC token verification	Bind signing to attested TEE	■■ Requires integration
Code quality	Production-tested codebase	Patches apply cleanly; test suite passes	■ Review needed
Key derivation	Additive (not BIP-32)	Deterministic child keys	■■ Partial

Hybrid architecture (likely outcome):

Chain	Signature Scheme	MPC Backend
Ethereum	ECDSA (secp256k1)	NEAR codebase fork (NearFall-operated)
Midnight	BIP-340 Schnorr	FROST MPC (separate network)
Other Substrate	Ed25519 or ECDSA	Fork or chain-specific

Fallback: If the NEAR codebase proves unsuitable for forking (code quality, maintenance burden, attestation limitations), the architecture uses FROST for both chains, eliminating the NEAR dependency entirely.

Attestation Binding Protocol (Required Upgrade)

NEAR's current attestation model (OIDC pass/fail) is insufficient. NearFall requires **measurement-bound authorization**: signing is gated on the TEE running expected code, not just "a TEE exists."

Session binding flow:

1. TEE boots, obtains attestation evidence (GCP OIDC / AWS Nitro doc)

2. TEE generates ephemeral session keypair (Ks_pub, Ks_priv)

3. TEE binds Ks_pub to attestation evidence (signed bundle)

4. Client verifies attestation before sending encrypted intent

5. MPC nodes verify attestation + session binding before signing

SigningRequest includes:

- attestation_bundle (measurement + session key binding)
- session_signature (request signed by Ks_priv)
- request_payload (typed request format)

MPC rejects if:

- measurement not in allowlist
- session key not bound to valid attestation
- attestation expired or revoked

Measurement allowlist governance:

Action	Authority	Process
Add measurement	Operator multisig (e.g., 3-of-5)	Code review → deploy → hash → add to allowlist
Remove measurement	Operator multisig	Emergency: 1-of-5 with time-delay; routine: 3-of-5
Emergency revocation	Any operator	Immediate disable, requires multisig to re-enable

The allowlist is stored on-chain or in a quorum-signed config that MPC nodes fetch. A measurement not in the allowlist cannot authorize signing—even if the attestation is cryptographically valid.

Why this matters: Without measurement binding, a rogue operator could deploy modified TEE code that leaks secrets or signs unauthorized transactions. The MPC layer would see "valid attestation" and sign anyway. Measurement binding closes this gap.

2.2.6 Chain Adapters (Settlement on Midnight, Coordination on External Chains)

NearFall operates across two chains with different roles. **Midnight is the settlement layer**—where proofs are verified and state is anchored. **Ethereum is an external coordination chain**—where NearFall can lock, release, and move assets on Midnight's behalf.

Midnight (Settlement Layer)

- On-chain verifier checks Compact contract proof (PLONK/Halo2 on BLS12-381 + JubJub with KZG)
 - Settlement state anchored as contract state on Midnight
 - Transactions submitted via `send_mn_transaction(midnight_tx)`
 - Fees paid in DUST for verification
 - Finality inherits Midnight's block characteristics (~6s)
 - **This is where correctness is proved and recorded**

Ethereum (External Coordination)

- Standard ECDSA authorization on secp256k1
 - Hub custody / wrapper contracts perform lock/unlock actions
 - Ethereum does not verify batch proof; **correctness is enforced by settlement on Midnight**
 - MPC signature provides authorization to move assets
 - **This is where NearFall extends Midnight's reach**

Midnight-First Finality (Cross-Chain Release Protocol)

External chain actions are **subordinate** to Midnight settlement. The release sequence:

1. User locks assets on Ethereum (escrow)
 2. NearFall processes intent, generates batch proof
 3. Midnight accepts proof, updates state root ← "included"
 4. Midnight block finalizes (~6s) ← "finalized"
 5. NearFall obtains finality proof
 6. External chain release authorized ← only NOW

The invariant: External chain cannot advance settlement state without proof of Midnight-finalized batch. This is not optimistic—it is cryptographic.

Proof paths for external chain release:

Path	Mechanism	Trust Assumption
BEEFY commitment	Midnight light-client proof of finalized block containing batch	Cryptographic (preferred when available)
Attestation fallback	TEE attestation that enclave observed Midnight finality	TEE integrity (fallback if BEEFY unavailable)

The external chain custody contract accepts either proof type. BEEFY is preferred; attestation is the bootstrap path until Midnight ships BEEFY.

Why this ordering matters: If external chain released before Midnight finality, a Midnight reorg could settle a different batch while ETH funds are already released. Midnight-first finality eliminates this race.

Cross-chain atomicity: Settlements spanning both chains require coordination (ETH locks → Midnight settles → ETH releases; failure on either side must revert). See §6.2 for the atomicity protocol.

2.2.7 Checkpoint and Recovery

Crash recovery is a correctness feature, not an operational convenience. Checkpoints must support resumability without replaying settled batches or double-broadcasting transactions.

Inclusion vs. Finality (critical distinction):

State	Meaning	Safe to release external funds?
Included	Midnight tx in a block, state root updated	■ No — block could reorg
Finalized	Block containing tx is finalized (~6s after inclusion)	■ Yes — cryptographic finality

Concrete failure scenario (why this matters):

[illegible]

Timeline (correct: release on finality)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

t=0: User locks 1 ETH on Ethereum

```
t=1:  NearFall settles batch #42 on Midnight (included in block 1000)
```

```
t=2: NearFall waits...
```

```
t=3: Midnight reorgs; block 1000 replaced by block 1000'
```

```
t=4: NearFall detects reorg; batch #42 is no longer included
```

```
t=5: NearFall re-submits batch #42 to new chain tip
```

```
t=6: Block 1001' includes batch #42, finalizes
```

```
t=7: NearFall releases 1 ETH to user on Ethereum ← SAFE
```

Checkpoints track both states separately. A batch marked "included" is not "settled" until finality is confirmed. External chain release waits for finality, not inclusion.

Checkpoint granularity:

- After each successful fold: `accumulator_state + batch_nonce`
- After Midnight inclusion: `midnight_block_hash + inclusion_height`
- After Midnight finality: `finality_proof` (BEEFY or attestation)
- After MPC signing: signed transaction ready for broadcast

Checkpoint invariants (required properties):

Invariant	Description
Inclusion vs. finality distinction	Checkpoint must distinguish between "included in block" and "finalized" states
Measurement binding	Checkpoint must be bound to TEE measurement; host cannot unseal
Exactly-once semantics	Must support replay prevention via monotonic <code>batch_nonce</code>
Phase tracking	Must record current phase (FOLDING → SUBMITTED → INCLUDED → FINALIZED → RELEASED)
Proof binding	Must bind <code>proof_artifact</code> to <code>state_root</code> transition

Minimum checkpoint fields:

- Batch identifier and monotonic nonce
- Current phase in settlement lifecycle
- Accumulator/proof state (if folding)

- State roots (previous and next)
- Chain inclusion evidence (when available)
- Finality evidence (when available)
- Measurement binding for sealed storage

> **Note:** Exact field names, serialization format, and optional fields are implementation choices. The invariants above are normative; the schema is not.

Recovery procedure:

- New TEE instance boots and attests to expected measurement
- Retrieves latest sealed checkpoint from storage
- Validates `checkpoint_hash` and measurement binding
- Resumes from recorded `phase`, redoing only uncheckpointed work
- MPC layer enforces monotonic `batch_nonce`, rejecting duplicates

Failure modes:

Failure	Recovery Action	Data Loss
TEE crash mid-fold	Resume from checkpoint	Uncommitted folds
TEE crash after sign	Retrieve signed tx, broadcast	None
Cloud storage failure	Restart batch from scratch	Current batch
MPC unavailable	Retry with exponential backoff	None (batch queued)

Exactly-once guarantee: `batch_nonce` is monotonic and checked by MPC wrapper. Duplicate (`batch_id`, `batch_nonce`) requests are rejected. Uncommitted intents return to mempool after timeout.

2.2.8 Summary

The architecture has five components with interlocking trust assumptions. No single component can move funds alone.

Comp onent	Role	Trust Basis	Key Constraint
Intent API	Accept encrypted intents	Append-only log	No plaintext exposure
TEE layer	Private processing, proving	Hardware attestation	Cannot sign without MPC
MPC layer	Threshold signing	Quorum honesty	Cannot sign without attestation
Midnig ht	Settlement layer	Chain consensus	BIP-340 Schnorr required
Ethere um	External coordination	Chain consensus	ECDSA sufficient

Cross-references:

- Proof boundary decision (Path A/B): §2.1.3
- MPC wrapper contract: §4.3
- BIP-340 analysis: §4.2
- Cross-chain atomicity: §6.2
- Crash recovery details: §6.4
- TEE security analysis: §5.1

What comes next: With the components fixed, we can trace a single batch through them. The next section follows the artifacts—`intent_ciphertext`, `batch_manifest`, `proof_artifact`, signed digests—from user submission to on-chain finality.

2.3 End-to-End Data Flow

This section specifies the end-to-end flow from **user intent submission** to **cross-chain settlement**, including the confidentiality boundary (TEE), the authorization boundary (MPC signing), and the on-chain commitment (Midnight proof + state update). Artifacts are named, phases are explicit, and failure handling is bounded.

The crux is binding: The MPC must only sign digests that commit to the same `batch_manifest_hash` and `state_root_next` that the proof attests to. If signing and proving can diverge, the system can authorize transactions that don't match what was proven—breaking the entire security model.

> **Running Example: The ETH→NIGHT Swap** > Throughout this section, we trace one concrete case: Alice wants to swap 1 ETH for 100 NIGHT without revealing her trade until settlement. > > 1. Alice encrypts her intent and submits it to the Intent API > 2. NearFall TEE decrypts, finds a matching counterparty (Bob: 100 NIGHT → 1 ETH), computes witnesses > 3. TEE builds `batch_manifest` (Alice's intent + Bob's intent + outcomes), proves the state transition > 4. TEE constructs **three** transactions (pending/commit model): > - **Midnight** `recordPending`: Proves batch, reserves Bob's NIGHT (no final transfer yet) > - **Ethereum** `release`: Releases Alice's ETH to Bob (requires Midnight finality proof + membership proof) > - **Midnight** `commit`: Finalizes Bob's NIGHT to Alice (requires Ethereum release proof) > 5. MPC signs transaction digests as needed (ECDSA for Ethereum, BIP-340 for Midnight) > 6. Transactions execute in dependency order: `recordPending` → wait for Midnight finality → `release` → `commit` > 7. Alice and Bob receive their funds; settlement is atomic (both succeed or both unwind via timeout) > > The rest of this section specifies each step. When we say "intent" or "batch," think Alice and Bob.

2.3.1 Actors and Artifacts

Six actors participate in the flow. Thirteen artifacts move between them.

Actors:

Actor	Role
User Wallet	Signs intent payloads
Intent API	Accepts intents, publishes append-only events
NearFall TEE	Decrypts, matches, proves, requests signatures
MPC Network	Holds key shares; signs under policy + attestation
Ethereum	Wrapper/custody contracts
Midnight	Compact contract + on-chain proof verification

Canonical Artifact Names:

These names are used consistently throughout this document. Code snippets should map to these names.

Artifact	Description
<code>intent_ciphertext</code>	Encrypted intent body (plaintext never leaves TEE)
<code>intent_id</code>	Stable identifier for ordering, replay control, audit
<code>batch_id</code>	Batch identifier
<code>batch_nonce</code>	Monotonic nonce enforced by MPC (replay prevention)
<code>batch_manifest</code>	Ordered <code>intent_ids</code> + outcomes + fees + expiry decisions
<code>batch_manifest_hash</code>	$H(\text{batch_manifest})$ — commitment to batch contents
<code>witness_digest</code>	$H(\text{witness_set})$ — binds proving to specific witness
<code>state_root_prev</code>	State commitment before batch
<code>state_root_next</code>	State commitment after batch
<code>proof_artifact</code>	Midnight-verifiable proof (PLONK on BLS12-381, KZG)
<code>tx_eth</code> , <code>tx_midnight</code>	Chain transaction payloads
<code>tx_digest_eth</code> , <code>tx_digest_midnight</code>	Digests that MPC signs
<code>attestation_token</code>	TEE attestation binding code identity + measurement

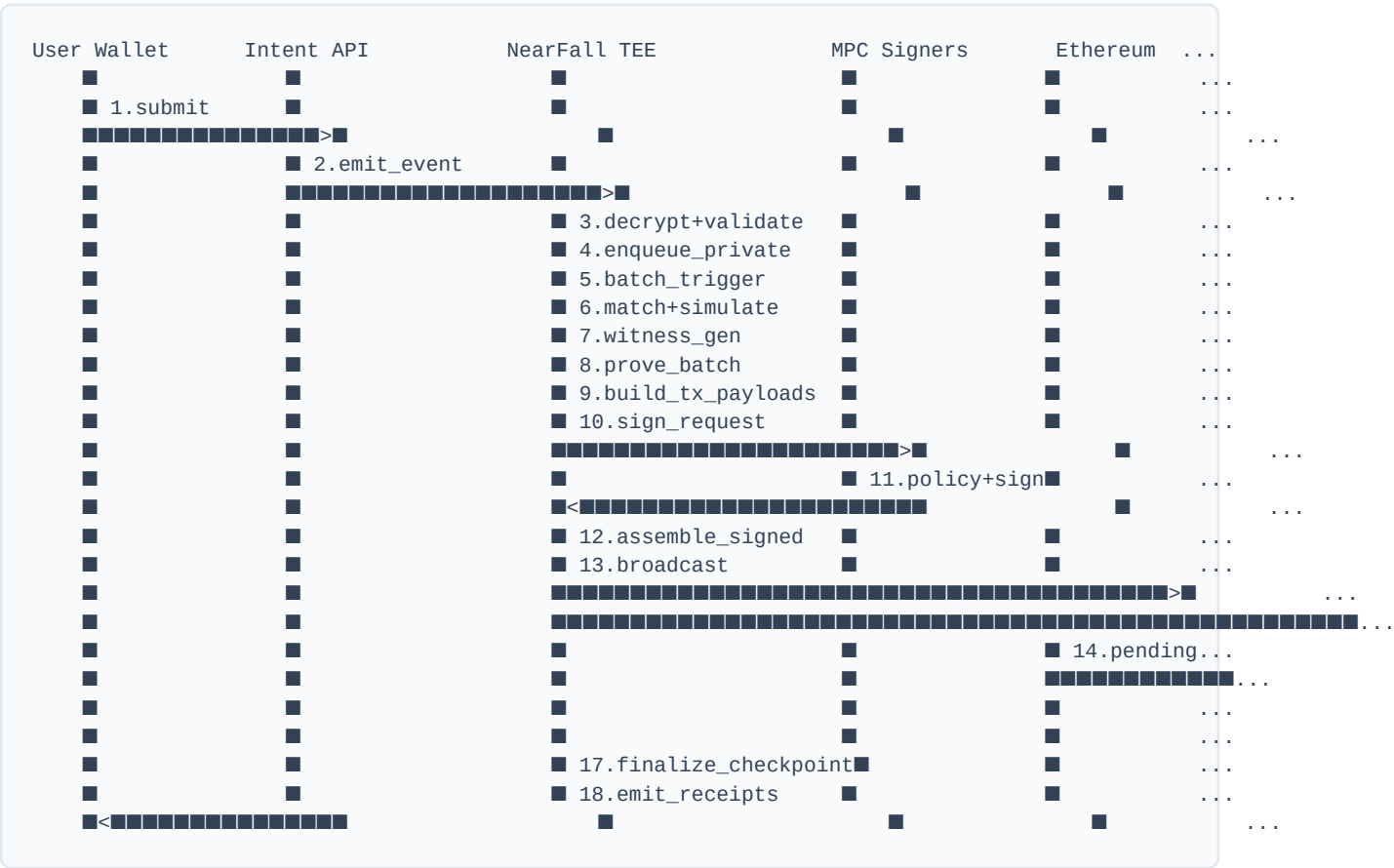
2.3.2 Phase Overview

The flow divides into five phases. Each phase has a clear boundary and a defined output.

Phase	Steps	Location	Purpose
1. Intent Submission	1-2	User → API → TEE	User commits to trade
2. TEE Processing	3-8	TEE	Decrypt, match, prove
3. MPC Signing	9-11	TEE ↔ MPC	Authorize chain transactions
4. Chain Settlement	12-16	TEE → Chains	Broadcast, verify, commit
5. Finalization	17-18	TEE → User	Checkpoint, emit receipts

2.3.3 Sequence Diagram

The diagram traces one batch from submission to finality. Steps 1-8 are the inner loop (TEE); steps 9-16 are the outer loop (MPC + chains).



2.3.4 Step Narrative (Normative)

Each step below is normative: implementations must preserve these semantics. The running example (Alice's ETH→NIGHT swap) is referenced where it clarifies.

Step 1 — Intent Submission (User → Intent API)

The wallet submits:

- `intent_id`
- `intent_ciphertext`
- `expires_at`
- Routing metadata (origin chain, asset pair, slippage bounds)

Intent Plaintext Schema (inside wallet before encryption):

Field	Type	Confidentiality	Description
<code>source_chain</code>	enum	Public (in routing metadata)	Origin chain identifier

Field	Type	Confidentiality	Description
target_chain	enum	Public (in routing metadata)	Destination chain identifier
source_asset	bytes	Confidential	Asset being sold
target_asset	bytes	Confidential	Asset being bought
source_amount	uint256	Confidential	Amount being sold
min_target_amount	uint256	Confidential	Minimum acceptable amount
source_address	bytes	Confidential	Sender address
target_address	bytes	Confidential	Recipient address
expires_at	uint64	Public	Intent expiry timestamp
nonce	uint64	Confidential	Replay prevention

> **Note:** The following TypeScript interface is illustrative pseudocode, non-normative. The canonical encoding is defined at implementation time.

TYPESCRIPT

```
// Illustrative only – not a committed API
interface SettlementIntentPlaintext {
  sourceChain: "ethereum" | "midnight";
  targetChain: "ethereum" | "midnight";
  sourceAsset: AssetId;
  targetAsset: AssetId;
  sourceAmount: bigint;
  minTargetAmount: bigint;
  sourceAddress: string;
  targetAddress: string;
  expires_at: number;
  nonce: bigint;
}
```

Wire format: Intents are transmitted as `IntentEnvelope { intent_id, ciphertext, expires_at, routing_metadata }`. Plaintext fields above are never published outside the TEE.

Requirement: Plaintext intent data must never be published outside the TEE.

Step 2 — Append-Only Publication (Intent API → TEE)

The Intent API emits an append-only event:

- `(intent_id, intent_ciphertext, metadata, expires_at)`

Requirement: Ordering must be reproducible (event log order or stable ordering rule).

Step 3 — Decrypt + Validate (Inside TEE)

The enclave:

- Decrypts `intent_ciphertext` using sealed private key
- Verifies wallet signature on intent payload
- Checks validity: expiry, replay flags, format constraints

Failure handling: Invalid or expired intents are rejected with stable error code; not reprocessed.

Step 4 — Private Mempool Enqueue (Inside TEE)

Validated intents enter the private mempool with:

- `intent_id`, decrypted record, admission timestamp, expiry timestamp

Requirement: Any persistence is sealed/encrypted; no plaintext storage outside TEE.

Step 5 — Batch Trigger

A batch starts when **either**:

- 100 intents are queued, **or**
- 30 seconds elapse since first queued intent

The batch is assigned `batch_id` and `batch_nonce` (monotonic).

Step 6 — Match + Simulate (Inside TEE)

The engine computes an execution plan. The matching algorithm must be deterministic given the same inputs.

> **Note:** The following Rust pseudocode is illustrative, non-normative.

RUST

```
// Illustrative only – not a committed implementation
fn match_intents(mempool: &Mempool, state: &State) -> BatchManifest {
    let mut manifest = BatchManifest::new();
    for buy in mempool.buys() {
        for sell in mempool.sells() {
            if compatible(buy, sell) {
                let price = clearing_price(buy, sell);
                if satisfies_slippage(buy, sell, price) {
                    manifest.add_match(buy, sell, price);
                }
            }
        }
    }
    manifest.compute_fees();
    manifest.mark_expired();
    manifest
}
```

Output: `batch_manifest` (ordered intent_ids with per-intent outcomes)

Requirement: Ordering and matching are deterministic given `(state_root_prev, intent_set)`.

Step 7 — Witness Generation (Inside TEE)

For each matched intent, generate ZK witness. The witness must contain sufficient information to prove state transition validity.

Witness Schema:

Field	Visibility	Description
<code>old_state_root</code>	Public	State root before batch
<code>new_state_root</code>	Public	State root after batch
<code>settlement_hash</code>	Public	Commitment to settlement details
<code>intent_a,</code> <code>intent_b</code>	Private	Matched intent payloads
<code>signature_a,</code> <code>signature_b</code>	Private	Intent authorization signatures
<code>balance_proofs</code>	Private	Merkle proofs for balance updates

> **Note:** The following Rust struct is illustrative, non-normative.

RUST

```
// Illustrative only – not a committed implementation
struct SettlementWitness {
    // Public inputs
    old_state_root: [u8; 32],
    new_state_root: [u8; 32],
    settlement_hash: [u8; 32],

    // Private inputs (hidden by ZK)
    intent_a: Intent,
    intent_b: Intent,
    signature_a: Signature,
    signature_b: Signature,
    balance_proofs: Vec<MerkleProof>,
}
```

Output: `witness_set, witness_digest = H(witness_set)`

Step 8 — Prove Batch (Inside TEE)

Produce:

- `state_root_next`
- `proof_artifact` binding `(state_root_prev, batch_manifest, witness_digest)` to `state_root_next`

Proof boundary (Gate G0.5):

- **Path A (Native Contract):** `proof_artifact` is contract execution proof
- **Path B (Verifier Gadget):** `proof_artifact` verifies external accumulator

Requirement: Proof must be verifiable by Midnight (PLONK on BLS12-381 + JubJub, KZG).

Step 9 — Build Chain Transactions (Inside TEE)

Construct unsigned payloads:

- `tx_eth`: Ethereum wrapper/custody actions
- `tx_midnight`: Midnight transaction with `proof_artifact` and `state_root_next`

Compute digests:

- `tx_digest_eth = H(tx_eth)`
- `tx_digest_midnight = H(tx_midnight)`

Step 10 — Attested Signing Request (TEE → MPC)

Request contains:

- `attestation_token` (measurement + policy claims)
- `batch_nonce`
- `tx_digest_eth`
- `tx_digest_midnight`

Step 11 — MPC Policy Checks + Signing

The MPC network signs only if:

- Attestation measurement matches approved enclave identity
- `batch_nonce == last_signed_nonce + 1` (strict sequential — skips rejected)
- Digests match expected format (domain separation, chain ID)

Output: `sig_eth`, `sig_midnight`

Failure handling: If MPC refuses, batch remains unbroadcast; retry or abort per policy. Resubmission of a signed-but-not-executed tx is done by re-broadcasting; MPC never re-signs the same nonce.

Step 12 — Assemble Signed Transactions (Inside TEE)

Attach signatures to payloads, producing broadcast-ready transactions.

Step 13 — Broadcast (TEE → Chains)

Broadcast:

- `tx_eth_signed` to Ethereum
- `tx_midnight_signed` to Midnight

Requirement: Broadcast is idempotent with respect to `(batch_id, batch_nonce)`.

Step 14 — Ethereum Wrapper Pending/Commit

Ethereum wrapper contracts process asset coordination (lock/release) under pending/commit model:

- Prevent premature release before Midnight settlement
- Tolerate delayed finality between chains

Steps 15-16 — Midnight Verification + Pending Record

Midnight settlement follows a pending/commit model (detailed in §6.2):

COMPACT

```
// ILLUSTRATIVE – field semantics are normative; syntax is not

contract SettlementVerifier {
  ledger {
    currentStateRoot: Bytes32;
    pending_settlements: Map<BatchNonce, PendingSettlement>;
  }

  // Step 15: Record pending (proof-verified)
  circuit record_pending(
    witness batchProof: Proof,
    oldRoot: Bytes32,
    newRoot: Bytes32,
    batch_commitment: Bytes32,
    batch_nonce: BatchNonce,
    expires_at: BlockHeight
  ) {
    assert(oldRoot == ledger.currentStateRoot);
    assert(batchProof.validates(oldRoot, newRoot, batch_commitment));
    assert(pending_settlements[batch_nonce] == null);

    ledger.currentStateRoot = newRoot;
    ledger.pending_settlements[batch_nonce] = {
      status: PENDING,
      batch_commitment: batch_commitment,
      expires_at: expires_at
    };
  }

  // Step 16: Commit (after Ethereum release proof)
  circuit commit(
    batch_nonce: BatchNonce,
    eth_release_proof: Proof
  ) {
    assert(pending_settlements[batch_nonce].status == PENDING);
    assert(verify_eth_release_proof(eth_release_proof, batch_nonce));
    assert(current_block < pending_settlements[batch_nonce].expires_at);

    // Finalize transfers to recipients
    ledger.pending_settlements[batch_nonce].status = COMMITTED;
  }
}
```

Pending/commit separation: `record_pending` reserves assets based on ZK proof. `commit` finalizes transfers only after Ethereum release is confirmed. This prevents the "Midnight commits before Ethereum releases" failure mode (§6.2.8).

Step 17 — Finalize Checkpoint (Inside TEE)

Update sealed checkpoint:

- Phase → `SETTLED`
- Record chain tx hashes, block heights, finality markers

Step 18 — Emit Receipts

Emit to user-facing layer:

- Per-intent outcome
- Inclusion in `batch_id`
- Chain tx hashes for Ethereum and Midnight

2.3.5 Timing Breakdown [Target]

All latencies in this table are **targets pending Gate G0.5 benchmarks**. Do not treat as commitments.

Phase	Steps	Target Latency	Bottleneck
Intent Submission	1-2	< 500ms	Network RTT
Decrypt + Validate	3-4	< 50ms	Signature verification
Batch Accumulation	5	≤ 30s	Trigger policy
Match + Simulate	6	< 100ms	$O(n^2)$ matching
Witness Generation	7	< 50ms/match	Merkle proofs
Prove Batch	8	5-15s [Target]	Decider SNARK
MPC Signing	10-11	< 2s	Threshold rounds
Broadcast	13	< 500ms	Network RTT
Chain Verification	14-16	< 100ms	PLONK verification
Finalization	17-18	~6s	Midnight block time

End-to-end for 100-intent batch: ~25-40 seconds total **[Target]**.

> **Honest uncertainty:** Proving time (5-15s) is based on BN254 estimates scaled for BLS12-381. Actual TEE performance may differ by 1.5-2×. Gate G0.5 must validate these numbers with actual Midnight circuit benchmarks before committing to SLOs.

2.3.6 Failure Handling (Bounded)

Eight failure points have defined recovery paths. None require manual intervention except "partial broadcast" (which has explicit remediation procedures).

Failure Point	Detection	Recovery	Constraint
Invalid intent	Step 3	Reject with error code	Not reprocessed
Intent expired	Step 5	Exclude from batch	Resubmit with new intent_id
No match found	Step 6	Wait in mempool	Until deadline or match
Prove failure	Step 8	Abort batch	Intents return to mempool
MPC timeout	Step 11	Retry with backoff	Same nonce if permitted
MPC refusal	Step 11	Abort or re-attest	Batch unbroadcast
Partial broadcast	Step 13	Pending/commit discipline	Explicit remediation
Chain reorg	Step 16	Re-submit if reverted	Wait for finality

Replay prevention: MPC enforces monotonic `batch_nonce`. Repeated request for same nonce is rejected, preventing duplicate authorizations even under enclave restart.

Reorg/finality: Checkpoint records finality markers. Settlement not marked final until defined finality condition met on both chains.

2.3.7 Confidentiality and Accountability

The flow achieves three properties simultaneously. No single failure mode breaks all three.

Property	Enforcement
Confidentiality	Intent plaintext and witnesses exist only inside TEE. Outside artifacts are ciphertexts, digests, and proofs.
Authorization	All chain actions require threshold signatures gated by attestation + nonce policy.
Accountability	Midnight verifies <code>proof_artifact</code> and anchors <code>state_root_next</code> . Each batch is externally auditable without revealing intent plaintext.

2.3.8 Summary

Each phase has a defined input, output, and trust assumption. The table serves as a reference for tracing failures.

Phase	Input	Output	Trust Assumption
Intent Submission	User-signed intent	Encrypted intent in mempool	User key security
TEE Processing	Encrypted intents	<code>proof_artifact</code> + <code>state_root_next</code>	TEE attestation
MPC Signing	<code>tx_digests</code> + attestation	<code>sig_eth</code> , <code>sig_midnight</code>	MPC quorum honesty
Chain Settlement	Signed transactions	State root anchored on-chain	Chain consensus
Finalization	Chain confirmations	User receipts	Indexer liveness

Cross-references:

- Batch trigger policy: [§2.2.2\(C\)](#)
- Checkpoint schema: [§2.2.7](#)
- MPC wrapper contract: [§4.3](#)
- Proof boundary decision: [§2.1.3](#)
- Error recovery details: [§6.4](#)

- Cross-chain atomicity: §6.2

What comes next: This flow describes what NearFall *does*; the remaining question is what Midnight must *accept* for it to count as settlement. The next section pins down the Compact contract semantics and proof-binding requirements that make this flow real.

2.4 Midnight Integration Technical Requirements

NearFall treats Midnight as the **settlement layer**—where proofs are verified and state is anchored with cryptographic finality. External chains like Ethereum are **coordination targets**—where NearFall locks, releases, and moves assets on Midnight's behalf. That split only works if the integration is explicit about three things:

- **What Midnight is verifying** — the proof boundary,
- **What Midnight is updating** — the state commitment, and
- **What NearFall is allowed to do after verification** — authorization and finality.

This section states those commitments in operational terms: what the proof must mean, what the Compact contract must enforce, what the signing and submission pipeline must guarantee, and what evidence must exist before calling the integration complete.

2.4.1 What Midnight Means in This Architecture

Midnight is not a passive ledger in NearFall. It plays two active roles:

Verifier of batch correctness. Midnight verifies a proof artifact that implies: given the prior state commitment and the declared batch outcomes, the new state commitment is correct under the settlement rules. In the ETH→NIGHT example from §2.3, this is the proof that Alice's 100 NIGHT and Bob's 1 ETH were correctly matched and transferred.

Canonical state anchor. Midnight stores (or can derive) the current `state_root` so every subsequent batch has a unique, unambiguous starting point.

The contract is the system's external memory. If it accepts the wrong thing once, everything downstream is contaminated.

2.4.2 Proof Compatibility Is Not a Detail

Midnight verifies proofs in its native environment. NearFall must treat that environment as a constraint, not a suggestion.

Midnight's ZK stack:

Component	Value
Circuit model	Halo2/PLONKish (custom gates + lookups)
Pairing curve	BLS12-381
Embedded curve	JubJub
Commitment scheme	KZG
Compilation pipeline	Compact → ZKIR → circuit

The "batch proof" is not an abstract object. It must be a proof artifact Midnight can verify *without* auxiliary trust assumptions and *without* a curve-adapter story that nobody can audit.

Two proof boundaries are possible (Gate G0.5 decides):

Native Contract boundary. The settlement transition is proven as a Compact contract execution proof. Midnight verifies it the same way it verifies any contract proof. No separate external `batchProof` to interpret; the batch is "just contract execution."

Verifier Gadget boundary. The Compact contract proof verifies a second proof (an accumulator or compressed proof) inside the circuit. The "batch proof" becomes an explicit artifact whose verification logic lives inside the contract circuit, with heavy constraint cost.

These are not interchangeable. They change what you build, what you benchmark, what you audit, and what can fail.

2.4.3 What the Proof Must Bind

The integration only works if the proof has the right meaning. The contract should not accept "a proof happened." It should accept a proof that binds the batch to a state transition.

Canonical commitment vocabulary:

`batchCommitment` must support **membership proofs** for Ethereum release verification (§6.2.4). A flat hash cannot support membership proofs. Therefore, `batchCommitment` is defined as a **Merkle root**:

```
headerLeaf  = H("hdr" ■ batchNonce ■ oldRoot ■ newRoot ■ count ■ witnessDigest)
fillLeaf_i  = H("fill" ■ lock_id ■ intent_id ■ asset ■ amount ■ recipient_eth ■ recipient_m...)

batchCommitment = MerkleRoot([headerLeaf, fillLeaf_1, fillLeaf_2, ..., fillLeaf_n])
```

Term	Definition	Where Used
<code>batchCommitment</code>	Merkle root over header leaf + fill leaves	Contract parameters, events, proof public inputs, Ethereum membership verification
<code>headerLeaf</code>	$H(\text{"hdr"} \parallel \text{batchNonce} \parallel \text{oldRoot} \parallel \text{newRoot} \parallel \text{count} \parallel \text{witnessDigest})$	First leaf; binds batch metadata
<code>fillLeaf_i</code>	$H(\text{"fill"} \parallel \text{lock_id} \parallel \dots)$	Per-fill leaves; enables membership proofs
<code>witnessDigest</code>	Commitment to witness assignment	Bound inside <code>headerLeaf</code>

Why Merkle root: Ethereum release (§6.2.4) requires `verify_merkle_proof(proof, batchCommitment, lock_id)` — proving a specific lock is included in the batch. A flat hash cannot support this. The Merkle structure enables:

- **Membership proofs:** Ethereum verifies `lock_id` is in the batch without seeing all fills
- **Single canonical commitment:** Midnight stores only `batchCommitment`; sub-data is reconstructible
- **Header binding:** `headerLeaf` binds `oldRoot`, `newRoot`, `witnessDigest`, `count`, `batchNonce` into the same root

`batchCommitment` is the single canonical commitment stored on-chain and emitted in events. The proof validates `batchCommitment`; the contract stores only `batchCommitment`; Ethereum verifies membership under `batchCommitment`.

At minimum, acceptance must imply correctness of:

- `oldRoot` — the pre-state commitment
- `batchCommitment` — the Merkle root binding batch contents and proof
- `newRoot` — the post-state commitment

Witnesses do not appear on-chain, but the proof must still bind to the witness assignment that produced the transition. The `witnessDigest` inside `headerLeaf` ensures the proof cannot be "reused" for a different witness assignment.

The rule:

> A batch is acceptable only if the proof makes `newRoot` inevitable from `oldRoot` and the declared `batchCommitment` under the settlement rules.

If the proof does not carry that meaning, Midnight is not anchoring correctness—only activity.

2.4.4 The Compact Contract Is the Integration Surface

NearFall can be sophisticated off-chain, but the integration succeeds or fails at the Compact contract boundary. The contract must make the on-chain acceptance rule sharp enough that operational shortcuts cannot creep in later.

The contract follows the **pending/commit model** specified in §6.2. Settlement is not atomic on Midnight alone—it requires confirmation that Ethereum has released funds before transfers finalize.

The contract maintains:

- the current `state_root`
- a replay-prevention handle (`last_batch_nonce`)
- a map of pending settlements awaiting Ethereum release confirmation

Three endpoints implement the atomicity protocol:

COMPACT

```
// ILLUSTRATIVE – field semantics are normative; syntax is not

contract SettlementVerifier {
    ledger {
        currentStateRoot: Bytes32;
        lastBatchNonce: Uint64;
        pendingSettlements: Map<Uint64, PendingSettlement>;
    }

    // Phase 2: Record pending settlement (proof-verified)
    // Reserves assets; does NOT finalize transfers
    circuit recordPending(
        witness batchProof: Proof,
        batchNonce: Uint64,
        oldRoot: Bytes32,
        newRoot: Bytes32,
        batchCommitment: Bytes32,
        expiresAt: BlockHeight
    ) {
        assert(oldRoot == ledger.currentStateRoot);
        assert(batchNonce > ledger.lastBatchNonce); // Contract uses > for retry tolerance
        assert(ledger.pendingSettlements[batchNonce] == null);
        assert(batchProof.validates(oldRoot, newRoot, batchCommitment));

        ledger.currentStateRoot = newRoot;
        ledger.lastBatchNonce = batchNonce;
        ledger.pendingSettlements[batchNonce] = {
            status: PENDING,
            batchCommitment: batchCommitment,
            expiresAt: expiresAt
        };

        emit BatchPending(batchNonce, oldRoot, newRoot, batchCommitment, expiresAt);
    }

    // Phase 4: Commit settlement (after Ethereum release confirmed)
    // Finalizes transfers to recipients
    circuit commit(
        batchNonce: Uint64,
        ethReleaseProof: Proof // Verification mode A/B/C per §6.2.5
    ) {
        assert(ledger.pendingSettlements[batchNonce].status == PENDING);
        assert(currentBlock < ledger.pendingSettlements[batchNonce].expiresAt);
        assert(verifyEthReleaseProof(ethReleaseProof, batchNonce));

        ... [truncated] ...

        emit BatchExpired(batchNonce);
    }
}
```

Why pending/commit instead of single-step settlement:

The single-step `submitBatch` model (proof → immediate transfer) creates the atomicity vulnerability described in §6.2: if Midnight commits before Ethereum releases, a failure on Ethereum leaves Bob with NIGHT while Alice's ETH remains locked. The pending/commit separation ensures NIGHT transfers finalize only after

Ethereum release is confirmed.

Contract invariants:

Invariant	Enforcement
Monotonic replay prevention	<code>batch_nonce</code> strictly increases; <code>recordPending</code> rejects duplicate nonces
Root continuity	<code>oldRoot</code> must match <code>currentStateRoot</code> ; no gaps in state chain
Proof validity	<code>batchProof</code> must verify for the claimed state transition
Timeout safety	<code>expiresAt</code> must be far enough in the future to allow Ethereum release + verification
No double-commit	<code>commit</code> only succeeds on PENDING settlements; COMMITTED/EXPIRED are terminal

Event semantics:

Event	Meaning	Who Watches
<code>BatchPending</code>	Assets reserved; awaiting Ethereum confirmation	Ethereum release contract (Mode A/B)
<code>BatchCommitted</code>	Transfers finalized; settlement complete	Users, auditors
<code>BatchExpired</code>	Timeout reached; assets returned to owners	Users (refund confirmation)

Idempotency requirement: Re-submission of the same `batch_nonce` fails deterministically. The contract does not create different states for the same batch identity.

2.4.5 State Commitment Requirements

`state_root` is the system's canonical commitment. If different components can compute different roots for "the same" state, you get non-reproducible proofs and impossible audits.

Two commitments must be version-stable:

State commitment (`state_root`):

Property	Requirement	Status
Hash function	Circuit-friendly (Poseidon preferred for in-circuit; Keccak256 acceptable for off-circuit)	■■ Decision at G0.5
Domain separation	Fixed prefix binding commitment type	■■ Pending
Encoding rules	Canonical serialization; field order, endianness, padding fixed	■■ Pending
Tree structure	Merkle tree with fixed depth and leaf encoding	■■ Pending

Changing encoding changes the meaning of state, even if the data "looks similar." The encoding rules must be frozen before G2 and documented in the gate evidence package.

Manifest commitment (`batch_manifest_hash`):

Property	Requirement	Status
Hash function	Collision-resistant; Keccak256 or SHA256	■■ Pending
Content binding	Commits to: intent identifiers, acceptance/rejection codes, fee allocations, expiry decisions	■ Specified
Ordering	Intent order in manifest must be deterministic and reproducible	■ Specified

The manifest hash prevents the system from quietly shifting from "we settled these intents in this order" to "we settled something like them."

> **Forward reference:** Concrete hash function and encoding choices are implementation decisions made before G2. The requirements above are normative; the specific algorithms are not.

2.4.6 Signing and Submission

From Midnight's perspective, NearFall is a transaction sender. From NearFall's perspective, transaction construction and signing are where safety is won or lost.

NearFall must sign and submit two distinct transaction types for the pending/commit model:

Transaction type 1: `recordPending`

- Parameters: `(batchNonce, oldRoot, newRoot, batchCommitment, expiresAt, proof_artifact)`
- Purpose: Record proof-verified batch as PENDING; reserve assets
- Signed by: FROST MPC (BIP-340 Schnorr)

Transaction type 2: `commit`

- Parameters: `(batchNonce, ethReleaseProof)`
- Purpose: Finalize transfers after Ethereum release is confirmed
- Signed by: FROST MPC (BIP-340 Schnorr)

Transaction type 3: `expirePending` (user-callable or operationally triggered)

- Parameters: `(batchNonce)`
- Purpose: Return reserved assets after timeout without Ethereum release
- Signed by: Any authorized caller (no MPC signature required for user-initiated expiry)

For each signed transaction, NearFall must:

- apply correct network domain separation (chain id / network id / genesis binding under Midnight conventions),
- obtain a chain-acceptable signature via the MPC custody layer,
- submit the transaction, and
- observe inclusion and finality.

The signing layer is policy enforcement, not a cryptographic vending machine. A sane signing policy requires:

Attestation-gated authorization. The signing network signs only when the request proves it came from an approved enclave measurement.

Strictly sequential nonce enforcement. A signing request must have `batchNonce == lastSignedNonce + 1`. Requests with gaps or repeats are rejected. This is stricter than monotonic ordering and eliminates skip-ahead ambiguity.

Digest-format validation. The signer policy does not sign arbitrary bytes; it verifies the digest corresponds to an allowed transaction shape (`recordPending` or `commit` with correct parameter encodings and network domain).

These three policies make enclave restart and retry safe: you can crash after building the transaction, recover, and retry submission without risking duplicate authorizations.

Custody constraints (Gap 2.6 resolution):

The signing policy must enforce custody limits in addition to attestation gating and replay prevention. These constraints bound exposure even if the TEE is compromised:

Constraint Type	Description	Enforcement Point
Per-transaction cap	Maximum value per signing request	MPC policy layer
Per-batch cap	Maximum aggregate value per batch	MPC policy layer
Rate limit	Maximum signing requests per time window	MPC policy layer
Per-asset exposure	Maximum outstanding custody per asset type	Custody contract + MPC policy

Escalation actions:

Trigger	Action
Single request exceeds per-transaction cap	Reject with <code>LIMIT_EXCEEDED</code> ; log for review
Batch aggregate exceeds per-batch cap	Reject entire batch; alert ops
Rate limit exceeded	Queue requests; alert if sustained
Per-asset exposure approaching limit	Warning alert; reject if exceeded

Configuration governance: Custody limits are configurable parameters, not hardcoded values. Limits must be defined before mainnet deployment and recorded in gate evidence. Changes require governance approval and re-attestation.

> **Note:** Specific limit values are operational policy decisions, not architectural constraints. The architecture requires that limits exist and are enforced; the values are defined in deployment configuration.

Key derivation paths:

Purpose	Path	Notes
Midnight mainnet	<code>midnight/mainnet/cish/settlement/{index}</code>	BIP-340 Schnorr
Midnight testnet	<code>midnight/testnet/cish/settlement/{index}</code>	BIP-340 Schnorr
Ethereum mainnet	<code>ethereum/mainnet/cish/settlement/{index}</code>	ECDSA secp256k1

Finality is a state machine input. NearFall must treat "included" and "finalized" as distinct and must not mark a batch settled until the configured finality condition is met. The checkpoint layer records tx hashes, block heights, and finality markers so the system can resume deterministically after outages.

2.4.7 The BIP-340 Signing Constraint

Midnight ledger transactions use **BIP-340 (Schnorr over secp256k1)** for signing. This is the compatibility issue introduced in §1.4, now stated as an integration requirement.

Source	Reference
Toolkit code	<code>midnight-node-main/util/toolkit/src/toolkit_js/mod.rs</code> expects "BIP-340 signing key"
Documentation	<code>midnight-docs-main/docs/develop/reference/./Signature.md</code> describes "BIP-340 signature"

> ■ **BLOCKING: NEAR Codebase Cannot Sign Midnight Transactions** > > The NEAR Chain Signatures codebase supports ECDSA (secp256k1) and Ed25519, but **not** BIP-340 Schnorr. BIP-340 and ECDSA are different signing algorithms on the same curve—an ECDSA signature cannot substitute for a Schnorr signature.

> > **Consequence:** The NEAR fork cannot be the MPC signer for Midnight ledger transactions. A separate FROST MPC network is required. See §4.2 for full analysis.

Required MPC architecture (resolves the design space stated in §1.4):

Chain	Signing Scheme	MPC Backend
Ethereum	ECDSA	NEAR codebase fork (NearFall-operated)
Midnight	BIP-340 Schnorr	FROST MPC (NearFall-operated, separate network)

Trade-off: Two MPC networks to operate, but both are NearFall-operated with no external dependencies.

FROST implementation status:

FROST (Flexible Round-Optimized Schnorr Threshold signatures) is the standard protocol for threshold BIP-340 signing. Several implementations exist:

Implementation	Status	Notes
ZCash Foundation FROST	Production-ready	Rust, audited, used in ZCash
tss-lib (Binance)	Mature	Go, includes Schnorr support
frost-secp256k1 (LIT Protocol)	Active development	Rust, secp256k1-specific

The plan is to **integrate an existing FROST implementation**, not build from scratch. Candidate selection criteria:

- Must be audited and deployed in production
- Must support Rust integration (matches NearFall stack)
- Must support BIP-340 compatibility mode

Library selection finalized at Gate G3, with fallback criteria if audit or benchmark requirements are not met.

The integration work includes adapter code, key ceremony tooling, and TEE attestation binding. This is parallel to the main NearFall implementation and does not block Gate G0.5.

2.4.8 Fees, Size Limits, and Practical Constraints

Midnight verification is not free. NearFall must budget for proof verification and contract execution, and must respect transaction size limits.

Two constraints matter immediately:

Proof size limits. If the proof artifact can exceed safe transaction sizes, the interface must define how it is carried (chunking, references, or a different proof boundary). Hand-waving here becomes a production outage later. Proof size constraints are validated as part of Gate G0.5 (D5).

Fee predictability. NearFall must preflight or estimate fees and refuse to submit batches that violate configured budgets. Otherwise the system fails only after doing the most expensive work (proving).

Falsifiable requirements:

- `max_fee_per_batch` is configurable
- When estimated fee exceeds budget, batch is rejected with deterministic error code before proving begins
- Test case: "attempt submit with estimated fee > max_fee → rejected before proving"

2.4.9 Error Model

Error taxonomy is a correctness feature because it determines recovery behavior. When the contract rejects a batch, the reason must be stable and enumerable so NearFall can respond deterministically.

The contract should distinguish:

Error	Retryable ?	Recovery
Nonce replay	No	Batch already processed; confirm or skip
State root mismatch	Maybe	Likely stale view; refresh and rebuild
Proof invalid	No	Bug or corruption; abort, investigate
Batch already accepted	No	Idempotent success; proceed
Malformed input	No	Bug in transaction construction

NearFall maps each error to an operational outcome: retryable vs non-retryable, whether intents return to the private mempool, and whether operator intervention is required. If you cannot make this mapping deterministic, you cannot run the system under fault.

2.4.10 Integration Tests: What Must Be Demonstrably True

We call NearFall "Midnight-integrated" only if the system can pass falsifiable checks that match the architecture's promises:

Pending/commit correctness: A submitted batch executes the full atomicity protocol:

- `recordPending` advances `state_root` and reserves assets with `BatchPending` event
- `commit` (after Ethereum release proof) finalizes transfers with `BatchCommitted` event
- `expirePending` (after timeout without Ethereum release) returns assets with `BatchExpired` event

Cross-chain atomicity: The following failure scenarios are tested and handled correctly:

- Ethereum release succeeds → Midnight `commit` succeeds → both parties receive assets
- Ethereum release fails/times out → Midnight `expirePending` succeeds → both parties refunded
- Midnight `recordPending` fails → Ethereum lock times out → Alice refunded

Replay-safe operation under restart: Crash/restart and retry do not produce double-sign or double-settle because MPC enforces nonce monotonicity and the contract is idempotent on batch identity.

Reproducibility: A pinned harness can regenerate the same proof/verifier outcome from fixed artifacts and golden vectors, and the node interface can reliably report inclusion and finality.

Gate G0.5 must be backed by measurements: proving time, proof size, verification cost, and memory footprint must be benchmarked on the Midnight curve stack in the target TEE environment before any capacity planning is treated as real.

2.4.11 Summary

Seven requirements define the Midnight integration. One is blocking; four depend on Gate G0.5.

Requirement	Status	Blocking?
Transaction submission via <code>pallet-midnight</code>	■ Interface known	No
BIP-340 Schnorr signing	■ NEAR codebase incompatible	Yes — requires FROST
Contract proof model (Path A or B)	■■ Gate G0.5 decides	Potentially
Curve compatibility (BLS12-381)	■■ Gate G0.5 decides	Potentially
State commitment format	■■ Needs specification	No for MVP

Requirement	Status	Blocking?
Idempotent contract behavior	■■ Design required	No
Falsifiable integration tests	■■ Not yet implemented	No for MVP

Cross-references:

- Proof boundary decision: [§2.1.3](#)
- BIP-340 analysis: [§4.2](#)
- MPC wrapper contract: [§4.3](#)
- Gate G0.5 validation criteria: [§3.1.3](#), [§3.1.7](#)
- Cross-chain atomicity: [§6.2](#)

What comes next: Part II has specified the architecture, components, data flow, and integration requirements. Part III defines the implementation plan: milestones, gates, deliverables, and the criteria that determine whether this design becomes real.

Part III: Implementation Plan

How to read Part III: This part is a commitment device. Part II described what NearFall *should do*. Part III describes *how we will know if it works*, and *in what order we will find out*. Read the gate definitions first—they are the decision points. Read the dependencies second—they show ordering. Read the deliverables last—they are what gets built.

3.1 Workstreams, Readiness Gates, and the Feasibility Phase

Section contents: Workstream overview (§3.1.1) → Gate model (§3.1.2) → Feasibility phase (§3.1.3) → FROST track (§3.1.4) → Workstream discipline (§3.1.5) → Sirius productization (§3.1.6) → Gate schedule (§3.1.7) → Production deliverables (§3.1.8) → Audit scope (§3.1.9) → Risk assessment (§3.1.10) → ERC-7683 roundtrip (§3.1.11) → Commitments (§3.1.12)

NearFall is not a "build everything, then integrate" project. It is a system whose hardest failures are *category errors*: choosing the wrong proof boundary, assuming curve compatibility, or treating an MPC network as a drop-in signer when the signing algorithm does not match the chain. The implementation plan is therefore organized around **readiness gates**: evidence-backed decisions that prevent downstream work from running on assumptions.

Two principles govern this plan:

- **Every workstream must end with a verifiable artifact.** "We believe it works" is not an output. A gate closes only when we can run an end-to-end test or reproduce a benchmark.
- **Gates control sequencing.** They are not management theater. A gate is a technical commitment: once it closes, downstream work is allowed to become irreversible.

3.1.1 Workstream Overview

The work divides into a short feasibility phase and four implementation workstreams. Each workstream ends with a readiness gate that produces an evidence package and a go/no-go decision.

Workstream	Objective	Output Artifacts	Readiness Gate	Complexity
Feasibility	Prove the architecture is coherent on <i>actual</i> signing and proof constraints	Signing test vectors, proof-boundary decision, curve strategy, latency benchmarks	G0.5	Medium
Workstream 1: TEE + Attestation	Establish the confidentiality boundary and execution identity	Attested TEE runtime, sealed storage, checkpoint skeleton, intent ingest path	G1	Medium
Workstream 2: External Chain + IVC Foundation	Enable cross-chain coordination (Ethereum custody) and build provable batch pipeline	Ethereum wrapper path, digest policy, proof pipeline skeleton, ERC-7683 roundtrip test , end-to-end test harness	G2	High
Workstream 3: Midnight Integration	Anchor correctness on Midnight (proof verification + root update)	Compact contract interface, Midnight tx pipeline, proof acceptance on testnet	G3	High
Workstream 4: Hardening	Turn "works in a lab" into "survives production fault"	Monitoring, rate limits, abuse controls, chaos tests, incident playbooks, endurance test	G4	High

This unit defines the gate model and the feasibility phase. The detailed per-workstream tasks are specified in §3.2–3.5; gate criteria are defined in §3.1.7 and in each workstream's validation matrix.

3.1.2 What a Readiness Gate Is (and What It Is Not)

A **readiness gate** is a bounded decision point with explicit evidence requirements. Passing a gate means we can safely commit to downstream engineering work without re-litigating foundational assumptions.

A gate has four parts:

Component	Description
Decision statement	What is being decided (e.g., "Path A vs Path B" for the proof boundary)
Evidence package	The artifacts required to justify the decision
Pass criteria	Objective checks (tests, benchmarks, reproducible builds)
Failure consequence	What we do if it fails (fallback design, scope change, or stop)

A gate is **not** a milestone like "we wrote code." It is a claim we can defend under adversarial review.

To keep sequencing honest, we enforce one rule:

> **No downstream build work may depend on an open gate.** > If a component's correctness depends on a gate outcome, implementation is limited to scaffolding until the gate closes.

3.1.3 Feasibility Phase: Close the Unknowns First

The feasibility phase exists for one reason: to avoid building a beautiful system on an impossible constraint set.

It closes **Gate G0.5**, which covers the two foundational unknowns identified in Parts I–II:

- **Signing compatibility:** Midnight requires BIP-340 Schnorr; the NEAR Chain Signatures codebase does not produce BIP-340.
- **Proof boundary and curve strategy:** Midnight verifies Halo2/PLONKish proofs on BLS12-381 + JubJub; candidate folding stacks (e.g., Sirius/ProtoGalaxy as commonly deployed) are curve-mismatched unless ported.

G0.5 Deliverables (Evidence Package)

D1 — Signing acceptance proof (Midnight).

Produce an end-to-end test that demonstrates what Midnight will accept, using real formats and real verification paths:

- A BIP-340 key + signature generation path
- A transaction (or test vector) that the Midnight stack accepts
- A negative test that fails under ECDSA (to prevent "encoding confusion" regressions)

This converts "we think Midnight uses BIP-340" into "the chain accepted these signatures and rejected these others."

D2 — MPC latency and availability measurements (Ethereum path). Measure the NEAR codebase fork on NearFall-operated nodes under representative load:

- p50/p95 signing latency
- Sustained throughput
- Failure behavior under partial node loss
- Policy binding (attestation-gated sessions, replay controls)

These measurements determine whether the user-facing UX assumptions in later sections are realistic.

D3 — Proof boundary decision (Path A vs Path B). Freeze the proof boundary defined in §2.1:

- **Path A (Native Contract)** if Midnight-native verification/aggregation can meet the target throughput/latency envelope
- **Path B (Verifier Gadget)** only if Path A cannot meet the envelope and verifier-gadget cost is acceptable

The output of D3 is not prose; it is a *frozen interface*: what the "batch proof" is, what its public inputs are, and what on-chain acceptance checks it satisfies.

D4 — Curve strategy decision (native vs port). Commit to one of:

- A Midnight-native path (preferred), or
- A concrete port plan with an explicit risk register (if folding requires a non-native stack)

A "maybe we port later" outcome is not acceptable. If a port is required, the scope and complexity must be reflected immediately.

D5 — Proof size constraints. Measure batch proof artifact size at representative batch sizes. Confirm that proof artifacts fit within Midnight transaction size limits. If proof size exceeds limits, document chunking strategy or batch size ceiling before proceeding.

G0.5 Pass Criteria

G0.5 closes only if all of the following are true:

- **Midnight signing reality is verified:** we have accepted test vectors and failed counterexamples.
- **Ethereum signing performance is measured:** latency/throughput targets are supported by real benchmarks, not estimates.
- **Proof boundary is frozen:** Path A or Path B is selected with an explicit artifact definition.
- **Curve strategy is frozen:** we have either a native plan that works or a port plan with explicit complexity and risk posture.
- **Proof size is bounded:** batch proof fits within transaction limits, or a mitigation strategy is documented.

If any of these fail, we do not "push forward and hope." The failure consequences are concrete:

Failure	Consequence
Signing reality unclear (D1 incomplete)	Escalate to Midnight team; gate blocked until resolved.
Ethereum signing too slow (D2 fails latency targets)	Evaluate alternative MPC networks (Lit Protocol, Entropy). If no alternative meets targets, reduce throughput SLO and document user-facing impact.
Path A infeasible, Path B unacceptable (D3 deadlock)	Escalate to IOG for native aggregation support. If IOG cannot commit, evaluate whether NearFall is viable without recursive verification (simpler architecture, weaker guarantees).
Curve strategy unresolved (D4 incomplete)	If native path blocked AND port complexity exceeds Workstream 2 scope, pause program and re-scope. A port that exceeds the IVC foundation scope is a different project.
Proof size exceeds limits (D5 fails)	Document chunking strategy or reduce batch size ceiling. If neither is acceptable, escalate to IOG for transaction size guidance.

The worst outcome is not "we picked wrong"—it's "we didn't know we picked wrong until late integration." G0.5 exists to make that impossible.

3.1.4 The FROST Track (If BIP-340 Confirmed)

If D1 confirms BIP-340 (as expected), NEAR Chain Signatures cannot sign Midnight transactions. This is not a surprise—it is the baseline assumption. The response is a dedicated FROST workstream that runs parallel to Workstreams 1–2 but begins after G0.5 closes (since D1 confirmation is a prerequisite).

Complexity: Medium-High (threshold cryptography integration, key ceremony design)

Sequencing: FROST workstream is blocked by G0.5 (requires D1 confirmation). Once unblocked, it runs concurrently with Workstreams 1–2 and must complete before Workstream 3 requires Midnight signing.

Milestone	Deliverable	Exit Criteria
FROST POC	FROST library selection + POC	Threshold Schnorr signature on secp256k1
Key Ceremony	Key generation ceremony design	DKG protocol specified; key share distribution
Signing Integration	Signing protocol integration	TEE can request FROST signature via API
Attestation Binding	Attestation binding	FROST nodes verify TEE attestation before signing
Integration Testing	Integration testing	End-to-end: TEE → FROST → valid Midnight signature

This is NOT a deferred fallback. It is a funded, staffed parallel workstream that completes before Workstream 3 needs it. The hybrid architecture uses the NEAR fork for Ethereum (ECDSA) and FROST for Midnight (BIP-340 Schnorr). Both are NearFall-operated—no external dependencies.

Open question (to be resolved during experiment):

> **Who operates the FROST MPC network?** > > The FROST workstream specifies *what* gets built, not *who* runs it. The operator model has three candidates: > > | Model | Pros | Cons | > |-----|-----|-----| > | **Midnight validators** | Aligned incentives; existing infrastructure | Adds operational burden; may not have MPC expertise | > | **Dedicated NearFall operators** | Specialized; can optimize for signing latency | New trust assumption; who selects them? | > | **Hybrid (validators + operators)** | Diversity; no single-party control | Coordination complexity; split incentives | > > This decision does not block the FROST build. It must be resolved before Workstream 3 integration, when the signing path goes live. The experiment should surface operational requirements (latency, availability, key ceremony participation) that inform the operator model choice.

3.1.5 Workstream Discipline: What We Build Before G0.5 Closes

Before G0.5 closes, engineering work is limited to items that are **gate-agnostic scaffolding**:

- Skeleton intent ingest and sealed queue format (no binding to a final proof scheme)
- Attestation plumbing and identity measurement capture
- Build system pinning (reproducible versions, deterministic artifacts)
- Test harness infrastructure (so D1–D4 can be run and repeated)

Anything that locks in the proof boundary, accumulator format, or Midnight signing path before G0.5 closes is treated as rework by definition.

3.1.6 Sirius Fork Productization (Parallel Track)

The Sirius codebase is research-grade. Before production deployment, the following gaps must be closed. This work runs in parallel with Workstreams 1–2.

Complexity: High (cryptographic implementation, audit surface). Requires familiarity with PLONKish arithmetization and Rust unsafe patterns.

Level	Required For	Scope	Estimated Effort
MVP	Workstream 3 start	Priority 1: protocol correctness, panic elimination, domain separation	Medium
Engineering Complete	Integration testing	+ Priority 2–3: serialization, logging hygiene, negative tests	Medium
Audit Ready	External audit	+ Priority 4: API stability, test vectors, documentation	Medium

Priority 1 (Critical):

- Transcript domain separation (add fixed tags, write golden vector tests)
- Point-at-infinity handling (trace all `.coordinates().unwrap()` paths, implement `BigUintPoint::identity()`)
- Eliminate `.unwrap()` in reachable paths (audit all non-test code, convert to `Result`, add input validation)

Priority 2 (Engineering):

- Replace unsafe commitment key serialization
- Remove sensitive debug logging
- Pin dependencies + add `cargo-deny/cargo-audit` to CI

Priority 3 (Testing):

- Negative/adversarial tests (mutate `poly_F`, `poly_K`, commitments; verify rejection)
- Golden transcript vectors
- Stable serialization with version tags

Priority 4 (API):

- Public API documentation
- Versioned formats with migration path
- Performance regression tests in CI

3.1.7 Readiness Gate Schedule

Beyond G0.5, the implementation has additional gates at workstream boundaries and for critical subsystems:

Gate	Scope	Pass Criteria	Failure Action
G0.5	Feasibility (signing, curves, proof boundary)	D1–D5 complete with evidence	Pause or fallback
G1	TEE + Attestation	Attested runtime, checkpoint recovery works	Debug TEE integration
G2	Ethereum + IVC	ECDSA < 2s, IVC fold < 100ms, E2E test passes	Optimize or descope
G3	Midnight Integration	Proof accepted on testnet, 100 consecutive BIP-340 signatures accepted	Debug Compact integration
G3.5	Verification Mode Selection	Mode A or B selected with implementation plan; Mode C not acceptable for mainnet	Extend G3 or accept higher risk with TVL caps
G4	Production Hardening	Endurance test at 99.9%, audit findings resolved, chaos suite passes	Extend remediation

G3.5 Cross-Chain Verification Mode Selection (Launch Prerequisite):

G3.5 covers verification in **both directions**:

- **Midnight→Ethereum (Phase 3)**: How Ethereum verifies Midnight finality before releasing funds
- **Ethereum→Midnight (Phase 4)**: How Midnight verifies Ethereum release before committing transfers

Direction	Verification Options	Mode Selection Required
Midnight→Ethereum	Mode A (BEEFY light client), Mode B (optimistic oracle), Mode C (trusted attestation)	Yes — at G3.5
Ethereum→Midnight	Mode A' (Ethereum light client on Midnight), Mode B' (optimistic oracle), Mode C' (trusted attestation)	Yes — at G3.5

Criterion	Evidence Required
Midnight→Ethereum mode selected	Documented decision with rationale (A vs B)
Ethereum→Midnight mode selected	Documented decision with rationale (A' vs B' vs C')
Implementation plan	Timeline, resource allocation, audit scope for chosen modes in both directions
Fallback policy	If Mode A/A' implementation fails, what is the contingency?
TVL policy	Maximum TVL under Mode C/C' (testnet only); TVL expansion gates for A/B
Symmetry justification	If different modes chosen for each direction, document why

Ethereum→Midnight verification options:

Mode	Mechanism	Trust Assumption	Complexity
Mode A'	Ethereum light client (sync committee proofs)	Cryptographic (Ethereum consensus)	High — requires Ethereum light client on Midnight
Mode B'	Optimistic oracle with dispute window	Economic (challenger incentives)	Medium — requires oracle + dispute mechanism
Mode C'	Trusted NearFall attestation	Operational (NearFall honesty)	Low — testnet only

This gate ensures verification mode selection is a deliberate engineering decision for **both directions**, not a deferred "pick later" item. Mode C/C' (trusted attestation) is acceptable for testnet but not for mainnet launch.

G4 Hardening Requirements: G4 is the production readiness gate. Requirements are detailed in [§3.5](#) (Workstream 4: Production Hardening). Summary criteria:

Category	Requirement
Performance	P99 < 2s, throughput ≥10 sig/sec sustained
Availability	99.9% uptime, 0 safety violations
Chaos suite	400 runs (4 scenarios × 100 iterations), 0 failures
Audit	All Critical/High findings resolved
Runbooks	Executed in drills with documented results

Full chaos protocol, observability requirements, and validation matrix are specified in [§3.5.4–§3.5.9](#).

Runbook requirements (Gate G4 artifact):

Runbook	Contents
Incident response	Escalation path, contact list, severity classification
Measurement revocation	How to disable compromised TEE code measurement
Key rotation	MPC key rotation procedure (no downtime)
Break-glass activation	When and how to invoke fallback signing committee

Subsystem gates (run within workstreams):

Gate	Blocked By	Criteria
Atomicity mechanism	G1 complete	Both chains reject on mismatch
Anti-rollback	G1 complete	MPC rejects replay after enclave rollback
Session binding	G1 complete	MPC authenticates TEE attestation + ephemeral key
Nonce sync	G2 complete	No gaps or conflicts under load test

3.1.8 Production Deliverables

Two deliverables are required for production but are not part of the core gate sequence:

Intent Schema Standard (Blocked by: G2 complete)

- Versioned intent schema with canonical serialization
- Domain separation to prevent cross-protocol replay
- 10+ test vectors including edge cases

Break-Glass Recovery Mechanism (Blocked by: G2 complete)

The break-glass mechanism provides sovereign fallback signing if the NearFall-operated MPC becomes unavailable (node failures, key compromise, operational issues). This is insurance, not an alternative operating mode.

Why break-glass matters: Without sovereign fallback, a NearFall MPC failure (node compromise, key loss, operational failure) leaves user funds stranded indefinitely. With break-glass, pending settlements complete under committee authority. The design balances two risks: (1) funds stranded indefinitely (mitigated by break-glass existence), (2) break-glass becomes default path (mitigated by timelock, scope limitation, reputation stake). Neither risk is eliminated; both are bounded.

Committee selection (illustrative default configuration):

- Members drawn from: Midnight validators, NearFall operators, independent security council
- Selection criteria: geographic diversity (no majority in one jurisdiction), infrastructure diversity (no majority on one cloud), reputation stake
- Members are public and auditable; this is not a secret committee

Key generation ceremony:

- Performed once during initial deployment, in-person or via audited MPC ceremony protocol
- Uses Shamir Secret Sharing with configurable threshold
- Each member receives one share; no member ever sees more than their share
- Shares are stored in hardware security modules (HSMs) or hardware wallets controlled by each member
- The combined key is *never reconstructed* during ceremony—only the shares exist

Activation protocol (prevents break-glass from becoming default):

- **Trigger detection:** Automated monitoring detects NearFall MPC unavailable beyond configured threshold OR key compromise detected
- **Activation request:** Any committee member can propose activation; requires on-chain governance proposal
- **Timelock:** Configurable delay before activation (allows MPC recovery or false-positive detection)
- **Threshold assembly:** Threshold of members must provide shares within a bounded signing window
- **Scope limitation:** Break-glass can only sign *pending settlements* (intents already in the TEE queue); cannot sign new intents

Why this won't become the default path:

- Timelock makes it slower than normal MPC for any non-emergency
- Requires human coordination within bounded window
- All break-glass activations are logged on-chain and trigger incident review
- Committee members have reputation stake; frivolous activation damages credibility

3.1.9 Audit Scope and Requirements

What's in scope for the external audit (after G3 complete):

Component	In Scope	Rationale
Sirius fork (folding/IVC)	■ Yes	Core cryptographic machinery; highest risk
FROST MPC integration	■ Yes	Must be stable before audit engagement
TEE enclave code	■ Yes	Trust boundary; attestation logic
Compact settlement contract	■ Yes	On-chain verifier; funds at risk if wrong

Component	In Scope	Rationale
NEAR codebase fork	■ ■ Partial	Interface + our modifications; base code audited by NEAR Foundation
Ethereum custody contract	■ Yes	Funds at risk
Intent schema + serialization	■ Yes	Canonicalization bugs cause signing failures

What's NOT in scope (externally audited or vendor responsibility):

- NEAR Chain Signatures base code (audited by NEAR Foundation; we audit our modifications)
- GCP Confidential Space / AWS Nitro (platform security; vendor responsibility)
- Midnight Compact runtime (audited by IOG)

Cryptographic operations requiring test vectors:

Operation	Test Vector Count	Coverage
ProtoGalaxy folding	20+	Valid folds, invalid witness, identity-point edge cases
Accumulator serialization	10+	Round-trip, version migration, malformed input rejection
BIP-340 signing (FROST)	15+	Valid signatures, wrong-key rejection, nonce reuse detection
ECDSA signing (NEAR fork)	10+	Valid signatures, digest binding, replay rejection
TEE attestation verification	10+	Valid attestation, expired attestation, wrong measurement

Computed Vector Requirements (Audit Readiness Rule)

The biggest audit failure mode is **placeholder test vectors**—hand-written "expected" values that were never computed by the actual system. NearFall enforces:

> **No placeholder vectors.** Every test vector must be *computed* by the implementation, verified independently, and checked into CI with regeneration capability.

Required vector sets:

Vector Type	What It Proves	CI Requirement
envelope → digest	Serialization is canonical	Regenerate from inputs; fail if hash changes
digest → signature → verify	Signing produces valid signatures (ECDSA + BIP-340)	Round-trip verification passes
one-byte mutation → rejection	Verification rejects tampered inputs	Negative tests fail as expected
full tx → testnet acceptance	End-to-end correctness	At least 5 vectors accepted by Midnight testnet

Vector repository requirement:

CI must include a vector regeneration capability that fails if any vector differs from checked-in values. This catches:

- Serialization drift between versions
- Non-deterministic implementations
- "It worked on my machine" regressions

Audit sequence (relative ordering):

Milestone	Blocked By	Deliverable
Audit branch cut	G3 complete	Code freeze on audit branch
Auditor engagement	Audit branch ready	Contract signed; kickoff
Audit execution	Engagement complete	Auditor review + Q&A
Findings delivery	Audit execution	Audit report received
Remediation	Findings delivery	Fix critical/high findings

Audit branch artifacts (required before audit engagement):

- Test vectors for all cryptographic operations listed above
- Threat model document (Part V)
- Adversarial test results (all gates)
- API documentation (stable serialization formats)
- Dependency audit report (cargo-deny + cargo-audit clean)

3.1.10 Complexity and Risk Assessment

Note on NEAR Chain Signatures: NearFall forks the NEAR Chain Signatures codebase; it does not depend on the live NEAR MPC network. This eliminates dependency risks (API changes, economic changes, governance actions) but creates maintenance responsibility. The fork is our code to maintain.

Complexity by workstream:

Workstream	Complexity	Primary Drivers	Key Risks
Feasibility (G0.5)	Medium	Midnight signing verification, curve compatibility analysis	Signing incompatibility discovered late
TEE + Attestation (G1)	Medium	GCP integration, checkpoint design, attestation binding	Platform-specific issues; MPC integration harder than expected
External Chain + IVC (G2)	High	Custody contract security, ECDSA latency optimization, IVC fold performance	Latency misses target; fold memory pressure
Midnight Integration (G3)	High	Compact contract interface, FROST signing, proof acceptance	BIP-340 integration issues; proof rejection
Hardening (G4)	High	Chaos engineering, operational tooling, endurance validation	Audit findings require significant remediation

Risk register:

Risk	Probability	Impact	Mitigation
BIP-340 confirmed, FROST required	70%	Low (parallel workstream)	Parallel workstream already planned
Path B required (verifier too large)	30%	High	Path B design ready before G0.5
BLS12-381 folding slower than target	25%	Medium	Accept smaller batches; optimize iteratively
Audit findings require remediation	40%	Medium	Pre-audit internal review; budget remediation

The architecture is designed so that worst-case scenarios result in "scope adjustment," not "impossible." No single risk is existential once G0.5 passes.

3.1.11 ERC-7683 Roundtrip Test Specification (Workstream 2 Deliverable)

NearFall must demonstrate compatibility with the Ethereum intent ecosystem. The roundtrip test validates all four CAKE layers in production configuration using the ERC-7683 standard.

Concrete trace (Alice's ETH→NIGHT swap via ERC-7683):

- Alice signs a `CrossChainOrder` on Ethereum: sell 1 ETH, buy ≥ 95 NIGHT, expires in 1 hour
- NearFall TEE decrypts, finds Bob's matching order (sell 100 NIGHT, buy ≥ 0.95 ETH)
- TEE folds both into the accumulator, generates decider proof
- FROST signs Midnight settlement; NEAR fork signs Ethereum repayment release
- Midnight verifies proof, anchors state root, finalizes
- Ethereum releases 1 ETH to filler (repayment); Alice receives 100 NIGHT

This validates all four CAKE layers: Application (intent encrypted), Permission (MPC threshold signing), Solver (TEE matching), Settlement (Midnight proof verification).

Test objective: Execute a complete cross-chain intent lifecycle with Midnight as settlement layer.

Test flow:



Test cases (minimum viable matrix):

Test ID	Origin	Destination	Asset	Settlement	Pass Criteria
RT-1	Ethereum Sepolia	Arbitrum Sepolia	ETH	Midnight testnet	Fill confirmed, proof verified, repayment released
RT-2	Arbitrum Sepolia	Ethereum Sepolia	ETH	Midnight testnet	Reverse direction works
RT-3	Ethereum Sepolia	Base Sepolia	USDC	Midnight testnet	ERC-20 path works
RT-4	Multi-origin	Multi-destination	Mixed	Midnight testnet	Batch settlement (ERC-8001 coordination)

ERC-7683 interface requirements (illustrative pseudocode, non-normative):

SOLIDITY

```
// NearFall implements IDestinationSettler
interface IDestinationSettler {
    function fill(
        bytes32 orderId,
        bytes calldata originData,
        bytes calldata fillerData
    ) external;
}

// Settlement proof structure (posted to Midnight)
struct SettlementProof {
    bytes32 orderId;
    bytes32 originChainId;
    bytes32 destinationChainId;
    bytes32 fillTxHash;
    bytes proof; // PLONK proof artifact
}
```

Gate G2 pass criteria (intent-related):

- RT-1 and RT-2 complete successfully on testnets
- Fill latency < 30 seconds (excluding Midnight finality)
- Settlement proof accepted by Midnight testnet verifier
- Filler repayment mechanism operational (optimistic or proof-based)

Dependencies:

- Workstream 1 complete (TEE operational)
- NEAR fork MPC signing ECDSA transactions

- Midnight testnet verifier deployed
- ERC-7683 OriginSettler deployed on test networks (use Across/UniswapX reference implementations)

3.1.12 What This Unit Commits To

This plan commits the project to an evidence-first approach:

- We close the signing and curve/proof-boundary unknowns before building irreversible components.
- We treat gates as hard technical commitments with pass/fail outcomes.
- We structure later workstreams so that each ends with an external verification artifact, not an internal claim.

With G0.5 closed, the rest of Part III becomes executable: Workstream 1 establishes the TEE boundary and checkpointing, Workstream 2 enables external chain coordination (Ethereum custody + proof pipeline), Workstream 3 anchors settlement on Midnight, and Workstream 4 makes the system survivable under fault and adversarial conditions.

Cross-references:

- G0.5 detailed criteria: [§3.1.3](#) and [§2.1.7](#)
- FROST MPC integration: [§3.4](#)
- Sirius productization: [§3.6](#)
- Go/No-Go gate definitions: [§3.1.7](#)
- Security audit scope: [§5.8](#)
- Risk register: [§6.1](#)

What comes next: This section defined the gate model and feasibility phase. The next section specifies Workstream 1 in detail: TEE bootstrap, NEAR fork attestation flow, and the GCP Confidential Space setup.

3.2 Workstream 1: TEE + Attestation Bootstrap

Blocked by: G0.5 complete

Readiness Gate: G1

Complexity: Medium

Workstream 1 establishes two boundaries NearFall relies on. The **confidentiality boundary**: intent plaintext and witness material remain opaque to operators, hosts, and the MPC layer. The **attestation boundary**: NearFall-operated MPC nodes authorize signing only for a measured enclave identity, not for a generic "OIDC token accepted" signal. At the end of Workstream 1, the system is either a real confidential co-processor with enforceable authorization, or it is not.

NearFall's ordering principle applies immediately. Midnight is the settlement layer and is treated as authoritative. Workstream 1 does not attempt full settlement; it prepares the environment in which later settlement claims can be made honestly.

3.2.1 Workstream 1 Objectives

Workstream 1 has two objectives that must both be true for later workstreams to be meaningful. First, NearFall must be deployable as an attested TEE service such that encrypted intents can be ingested and processed without plaintext egress through logs, metrics, storage, or debugging pathways. Second, NearFall must integrate measurement-bound authorization into its MPC fork so that signing is gated on code hash and session binding, not on a generic "TEE token accepted" rule.

Success is defined by falsifiable criteria. A workstream that is "mostly working" is treated as failing if the criteria are not met.

Objective	Success Condition (Falsifiable)	Test Condition
Confidential execution boundary exists	A host operator with root access cannot recover intent plaintext or witness material from memory snapshots, logs, metrics, traces, or sealed storage artifacts	Run ingestion + processing workload; collect host-level snapshots and all telemetry; confirm only ciphertext and commitments appear
Attestation is measurement-bound	MPC signing layer authorizes requests only when enclave measurement hash matches an allowlist entry and request is bound to that measurement via ephemeral session key	Attempt signing under valid measurement (accepted); attempt under different measurement (rejected); attempt with replayed session binding (rejected)
Crash recovery is safe	Restart does not create a second signing path for the same logical work, and does not permit rollback to an earlier authorization state	Induce crashes at defined phases; restart; confirm monotonic replay prevention holds and duplicate signing is rejected

3.2.2 Infrastructure Requirements

Workstream 1 runs on GCP Confidential Space as the primary platform. The baseline threat model assumes a hostile host OS and treats cloud operator visibility as untrusted. The platform choice determines what can be attested and what can be sealed.

The deployment environment is a private VPC with a private subnet. The NearFall TEE runtime must not be reachable from the public internet except through an explicit ingress boundary. Internal traffic between the TEE runtime and NearFall-operated MPC nodes stays on private networking. Workstream 1 does not require multi-cloud active/active, but it keeps the disaster-recovery path open; AWS Nitro is the designated DR platform and is explicitly non-blocking for Gate G1.

Category	Requirement	Workstream 1 Minimum	Rationale
TEE platform	AMD SEV-SNP or equivalent	Required	Primary measured runtime for Gate G1
Compute resources	≥8 vCPU, ≥32GB RAM	Required	Sirius accumulator operations require 8-12GB peak; 32GB provides headroom
Network	VPC with private subnet	Required	TEE not internet-reachable; ingress limited to intent gateway
Storage	Encrypted + measurement-bound sealing	Required	Checkpoints bound to enclave identity; host cannot unseal
Observability	Commitment-level telemetry only	Required	Metrics safe under hostile host; no plaintext sampling
Multi-cloud DR	Secondary provider interface compatibility	Not required for G1	Planning required; activation deferred to post-G1

Why 32GB RAM? Sirius accumulator operations and witness processing require 8-12GB peak. The 32GB floor provides headroom for batch size growth and prevents OOM during load spikes.

3.2.3 NEAR Fork Attestation Integration

NearFall operates its own MPC nodes by forking the NEAR Chain Signatures codebase. Workstream 1 integrates attestation into that fork in the only way that matters: authorization must be measurement-bound. An OIDC pass/fail signal may still exist as a transport mechanism, but it is not the gate. The gate is a code measurement hash and a session binding that proves the request originates from a runtime whose measurement is allowlisted.

The toy model: Think of the MPC nodes as a vault that will sign a message only when it receives two proofs at once: proof of identity (the attestation measurement hash) and proof of possession (an ephemeral session key whose public key is bound to that attestation). Without both, the vault refuses.

Why OIDC-only fails:

Attack scenario:

1. Operator deploys modified TEE code (backdoor that leaks secrets)
2. Modified TEE obtains valid OIDC token (it's still a TEE)
3. OIDC-only MPC sees valid token, signs request
4. Secrets leaked, unauthorized transactions signed

Defense (measurement-bound):

1. Operator deploys modified TEE code
2. Modified TEE obtains valid OIDC token
3. MPC checks `image_digest` claim against allowlist
4. Modified code's digest NOT in allowlist → signing rejected

Session binding is a protocol requirement, not an implementation detail. The enclave generates an ephemeral session keypair and produces an attestation-bound bundle stating that the measured runtime created that keypair. The enclave signs each signing request with the ephemeral private key. MPC nodes verify the attestation, verify the measurement allowlist membership, verify the binding to the session public key, and then verify the request signature under that session public key. Only then can the request reach the typed-signing policy layer.

Step	Action	Output
1	TEE boots, requests attestation from GCP metadata service	OIDC token with <code>image_digest</code> , <code>hwmodel</code> claims
2	TEE generates ephemeral keypair (<code>Ks_pub</code> , <code>Ks_priv</code>)	Session key valid for attestation lifetime
3	TEE creates attestation bundle: token + <code>Ks_pub</code> + signature	Bundle proves: "this measurement created this session key"
4	Client verifies bundle before sending encrypted intent	Client assured of TEE identity
5	MPC verifies bundle before signing	MPC assured request came from attested TEE

Rejection conditions (all falsifiable):

Mechanism	Requirement	Rejection Condition
Measurement-bound authorization	MPC accepts signing only if <code>measurement_hash ∈ allowlist</code>	Valid attestation with non-allowlisted measurement → rejected
Session binding	Requests authenticated by ephemeral key bound to attestation	Request signed by unbound key → rejected
Freshness	Attestation and session have bounded lifetime	Expired attestation or session → rejected
Replay resistance	Session identifiers and nonces prevent replay	Reuse of prior bundle or nonce → rejected

Measurement allowlist governance is part of security, not operations. Workstream 1 establishes the governance rule and demonstrates enforcement. The allowlist must support add, remove, and emergency revoke, and the MPC nodes must apply revocations promptly enough that compromised measurements are not usable during an incident window.

Governance Action	Authority Model	Enforcement Requirement
Add measurement	Multisig approval (3-of-5)	New measurement accepted only after allowlist update distributed to MPC nodes
Remove measurement	Multisig approval (3-of-5)	Removed measurement rejected within propagation SLA
Emergency revoke	Any single operator (break-glass)	Revoked measurement rejected immediately; requires multisig to re-add

3.2.4 Checkpoint and Recovery Validation

Checkpointing is a correctness feature, not an operational convenience. Recovery is not defined as "the service came back up." Recovery is defined as: the service resumed without duplicating signing, without losing phase coherence, and without allowing replay of a prior authorization state.

The key invariant is **monotonicity**: once the system has advanced the authorization state for a given key scope, it cannot go backwards after restart.

Crash scenarios to validate:

Scenario	Trigger Point	Required Recovery Outcome	Invariant Checked
Crash during ingest	After ciphertext accepted, before sealed queue commit	Intent present once in queue OR absent with stable rejection	No duplicate ingestion; no plaintext egress
Crash during checkpoint write	Mid-write of sealed checkpoint record	Recovery rejects partial checkpoint; resumes from last valid	Checkpoint integrity and atomicity
Crash after session registration	Session bundle issued, before signing requests	Recovery does not accept replay of prior session beyond lifetime	Session freshness and replay resistance
Crash after signing request	Request submitted to MPC, response not received	No double-signing; re-request rejected or reconciled deterministically	Monotonic replay prevention at MPC boundary

State consistency invariants:

Invariant	Statement	Violation Signal
Sealed-state integrity	Modified sealed checkpoint is detected and rejected	Checkpoint authentication fails
Measurement binding	Checkpoint sealed under one measurement cannot load under different measurement	Unseal fails or measurement mismatch flagged
Monotonic authorization	Recovered runtime cannot cause MPC to sign two different payloads at same monotonic position	MPC rejects replays; audit log shows no duplicates
Phase coherence	Phase transitions advance in defined order; cannot skip backwards	Recovery refuses invalid phase transitions

3.2.5 Workstream 1 Deliverables

Deliverable	Owner	Acceptance Criteria	Gate
GCP Confidential Space deployment	Infrastructure	Runtime deploys on SEV-SNP with pinned measurement and reproducible build	G1
Encrypted intent ingestion boundary	Runtime	Intents accepted as ciphertext only; telemetry contains no plaintext	G1
Measurement-bound attestation in MPC fork	Cryptography	MPC rejects non-allowlisted measurements; accepts allowlisted under same workload	G1
Session binding protocol	Runtime + MPC	Requests accepted only when signed by attestation-bound ephemeral session key	G1
Sealed checkpoint format	Runtime	Checkpoints sealed, integrity-protected, measurement-bound, recoverable after crash	G1
Recovery harness	QA	All specified crash scenarios execute reproducibly with required outcomes	G1
AWS Nitro DR interface plan	Infrastructure	Interface compatibility documented and validated at protocol level	Not G1

3.2.6 Workstream 1 Validation Matrix

Workstream 1 is validated by functional tests and adversarial tests. Functional tests show the happy path works. Adversarial tests show that security claims fail closed under wrong measurement, replay attempts, and expiry conditions.

Test ID	Category	Test Condition	Pass Condition
G1-F1	Functional	Deploy TEE runtime on GCP Confidential Space with pinned measurement	Attestation verifies; produces expected measurement hash
G1-F2	Functional	Submit encrypted intents through ingest and sealed queue	Ciphertext persists; no plaintext in logs/metrics/traces

Test ID	Category	Test Condition	Pass Condition
G1-F3	Functional	Register signing session with MPC using valid measurement	Session accepted; requests authenticated under bound session key
G1-F4	Functional	Crash and restart during defined phases	Recovery resumes without duplicate ingestion or invalid phase transition
G1-F5	Functional	Run sustained period with token expiry every hour	Zero signing gaps; refresh seamless
G1-A1	Adversarial	Attempt signing under enclave with different measurement hash	Rejected due to measurement mismatch
G1-A2	Adversarial	Replay previously valid session binding bundle after expiration	Rejected due to freshness failure
G1-A3	Adversarial	Replay previously valid signing request with same monotonic position	Rejected due to replay prevention
G1-A4	Adversarial	Present valid OIDC transport token but wrong measurement	Rejected; OIDC is not sufficient
G1-A5	Adversarial	Corrupt sealed checkpoint bytes and attempt recovery	Recovery rejects corrupted checkpoint
G1-A6	Adversarial	Token with wrong aud claim	Rejected
G1-A7	Adversarial	Token past exp claim	Rejected
G1-A8	Adversarial	Request signed with key not bound to attestation bundle	Rejected

Gate G1 pass criteria: NearFall passes Gate G1 only if the measured runtime is verifiable (G1-F1), the confidentiality boundary is demonstrated under hostile-host inspection (G1-F2), the MPC fork enforces measurement-bound authorization with session binding (G1-F3, G1-A1–A8), and crash recovery preserves monotonic authorization invariants (G1-F4).

Failure consequences:

Failure Type	Action
Functional tests fail	Debug GCP setup or MPC integration; re-test
1-2 adversarial tests fail	Fix promptly; re-test; proceed with caution
3+ adversarial tests fail	Stop; evaluate attestation model; consider alternative MPC
Recovery tests fail	Debug checkpoint logic; do not proceed until fixed

3.2.7 Workstream 1 Dependencies and Risks

Dependencies:

Dependency	Source	Risk if Unavailable
GCP Confidential Space	GCP	Switch to AWS Nitro (complexity increase)
NEAR fork codebase	Internal	Cannot proceed without fork
Measurement allowlist design	Security team	Blocks attestation integration

Risks:

Risk	Probability	Impact	Mitigation
GCP attestation format changes	Low	Medium	Pin to known-working SDK version
MPC attestation integration harder than expected	Medium	High	Budget additional effort; have FROST fallback
Checkpoint recovery edge cases	Medium	Medium	Extensive chaos testing before G1

Cross-references:

- Attestation binding protocol: [§2.2.5](#)
- Checkpoint model: [§2.2.7](#)
- G1 criteria: [§3.1.7](#)
- FROST parallel track (if MPC integration fails): [§3.1.4](#)

What comes next: With G1 closed, the TEE can run trusted computation and request signatures. Workstream 2 connects this capability to Ethereum custody operations and establishes the IVC/folding pipeline.

3.3 Workstream 2: External Chain Coordination

Blocked by: G1 complete

Readiness Gate: G2

Complexity: High

Workstream 2 makes NearFall operable on external chains without changing the system's settlement posture. Ethereum is a coordination target that can hold and release assets; Midnight is the settlement layer where correctness is proved and state is anchored. The technical consequence is that Ethereum custody is not allowed to "decide" when funds move. Ethereum custody is allowed to escrow and release only when it can verify that a Midnight-finalized settlement exists for the requested release.

Workstream 1 proved the confidentiality and identity boundary: an attested runtime exists, checkpoint recovery works, and MPC authorization is measurement-bound. Workstream 2 connects that boundary to external chains by building (i) an Ethereum custody interface that mirrors Midnight state and enforces Midnight-first finality, (ii) a typed ECDSA signing interface on the NearFall-operated MPC fork, and (iii) the IVC/folding foundation needed to batch work efficiently.

3.3.1 Workstream 2 Objectives

Workstream 2 has three objectives: external custody coordination on Ethereum, typed ECDSA authorization via the NearFall MPC fork, and an IVC/folding foundation with measured performance. Success is defined by falsifiable criteria tied to Gate G2.

Objective	Success Condition (Falsifiable)	Test Condition
Ethereum custody can escrow and release under Midnight authority	Release is rejected unless a valid Midnight finality proof is presented; deposits and settlement execution update the mirrored <code>stateRoot</code> monotonically	Execute end-to-end: deposit → settlement execution → release without proof (rejected) → release with proof (accepted)
NearFall MPC fork provides typed ECDSA signing with performance targets	MPC signs only typed requests whose digest reconstruction matches inputs, rejects arbitrary-bytes attempts, achieves P99 < 2s and throughput ≥5 sig/sec (G2 interim target; G4 requires ≥10 sig/sec)	Run 100 sequential requests for P99; 1-hour sustained load for throughput; verify rejection on malformed requests
IVC/folding foundation meets step performance target	Fold operation completes in < 100ms per step; accumulator state transitions are checkpointable and replay-safe	Run fold benchmarks; confirm per-step latency and memory footprint; confirm accumulator recovery without duplication
ERC-7683 roundtrip completes under Midnight-first ordering	RT-1 and RT-2 from §3.1.11 pass with escrow on Ethereum and settlement authority on Midnight	Execute roundtrip tests; confirm finality-gated release behavior is enforced

Workstream 2 does not claim production-grade custody security. It claims that custody is shaped correctly around Midnight-first finality and that the performance envelope is real enough to justify Workstream 3.

3.3.2 Ethereum Custody Contract Requirements

The Ethereum custody contract is a coordination primitive. It escrows assets, records a mirror of the Midnight settlement state root, accepts settlement execution updates authorized by the MPC fork, and releases funds only when it is presented with a proof of Midnight finality for the relevant settlement.

Contract state:

Field	Type	Mutability	Purpose
<code>mpcSigner</code>	addresses	Immutable	Prevents signer substitution attacks
<code>stateRoot</code>	bytes32	Mutable	Mirrors Midnight-authoritative root
<code>settlementNonce</code>	uint256	Mutable	Prevents replay and reordering of settlement executions
<code>deposits</code>	mapping	Mutable	Tracks escrowed balances per user per token
<code>releases</code>	mapping	Mutable	Tracks executed releases; enforces idempotency

Contract interface:

Method	Parameters	Required Checks	Required State Updates
<code>deposit</code>	token, amount	Amount positive; transfer succeeds	Increase escrow balance; emit Deposited event
<code>executeSettlement</code>	newRoot, transitionCount, signature	MPC signature valid under typed request; <code>settlementNonce</code> monotonic; chain domain correct	Update <code>stateRoot</code> ; increment <code>settlementNonce</code> ; emit SettlementExecuted event
<code>release</code>	recipient, token, amount, batchNonce, membershipProof, finalityProof	Escrow sufficient; release not previously executed; finality proof validates Midnight-finalized settlement; membership proof shows release in committed batch	Mark release executed; transfer funds; emit Released event

Security clarification for `executeSettlement`:

`executeSettlement` updates a mirrored state root on Ethereum. By itself, it **cannot cause fund release**. The security model is:

Question	Answer
Can <code>executeSettlement</code> alone release funds?	No — <code>release</code> is the only method that transfers funds
What does <code>executeSettlement</code> control?	It updates <code>stateRoot</code> , which <code>release</code> checks against the finality proof
What if a malicious <code>stateRoot</code> is written?	<code>release</code> will fail because the finality proof won't match the fake root
What if TEE is compromised but MPC is honest?	MPC policy enforcement prevents signing arbitrary <code>executeSettlement</code> calls (typed requests, nonce monotonicity)
What if MPC is compromised?	Attacker can write any <code>stateRoot</code> , but still cannot release without a valid Midnight finality proof

The invariant: `release` succeeds only if *both* (a) `stateRoot` matches the finality proof, *and* (b) the finality proof demonstrates Midnight committed the batch containing this release. A compromised `executeSettlement` call that writes a fake root breaks invariant (a); the release will fail.

Residual risk: If *both* MPC and cross-chain verification (Mode A/B/C) are compromised, an attacker could write a fake root and supply a fake finality proof. This is the residual risk floor acknowledged in §6.5.

The critical difference in NearFall: Settlement execution on Ethereum is not sufficient to unlock funds. Release requires proof that Midnight finalized the settlement that authorizes the release. The finality proof mechanism follows §2.2.6: BEEFY proof path is preferred when available; measurement-bound attestation fallback is the bootstrap path.

Concrete example (custody flow):

1. User calls `deposit(USDC, 1000)`
 - 1000 USDC transferred to contract
 - `Deposited(user, USDC, 1000)` emitted
2. NearFall processes intent batch, generates proof
 - Midnight accepts proof, updates state root (included)
 - Midnight block finalizes (~6s later)
3. NearFall calls `executeSettlement(newRoot, 50, signature)`
 - Contract verifies MPC signature under typed request format
 - `stateRoot` updated; `settlementNonce` incremented
 - `SettlementExecuted(42, oldRoot, newRoot)` emitted
4. NearFall obtains finality proof (BEEFY or attestation)
5. NearFall calls `release(recipient, USDC, 1000, finalityProof)`
 - Contract verifies finality proof references current `stateRoot`
 - Contract verifies release not already executed
 - 1000 USDC transferred to recipient
 - `Released(recipient, USDC, 1000)` emitted

If step 4 fails (no finality proof available), step 5 cannot execute. A release attempt that supplies only an Ethereum transaction hash or only the updated `stateRoot` must be rejected.

3.3.3 NEAR Fork ECDSA Integration

NearFall operates its own MPC nodes by forking the NEAR Chain Signatures codebase. In Workstream 2, those nodes provide ECDSA signing for Ethereum coordination transactions. The fork does not expose a "sign arbitrary bytes" interface. It exposes a typed request interface that forces the MPC layer to reconstruct the digest from declared components and reject if the reconstruction does not match.

Typed request format:

Field	Type	Purpose	MPC Validation Rule
<code>scheme</code>	enum	Signature scheme	Must be <code>ecdsa_secp256k1</code> for Ethereum
<code>derivation_path</code>	string	Key derivation path	Enforce allowlisted prefixes and indices
<code>domain</code>	bytes	Chain and contract binding	Recompute from <code>chainId</code> and contract address
<code>batch_nonce</code>	u64	Replay prevention	Reject unless <code>== last_signed_nonce + 1</code> (strict sequential)

Field	Type	Purpose	MPC Validation Rule
tx_shape_id	bytes32	Allowed method identifier	Reject if not in method allowlist
tx_params_hash	bytes32	Hash of typed parameters	Reject if parameter hashing does not match schema
digest32	bytes32	Final hash to sign	Recompute from components; reject if mismatch

The digest reconstruction rule is the core falsifiable claim. The MPC nodes must recompute `digest32` from the other fields and reject any request whose claimed digest does not match reconstruction. A request that supplies an allowed method id but mismatched parameters must fail the reconstruction check.

Concrete example (ETH release SigningRequest):

```
SigningRequest {
  scheme:          "ecdsa_secp256k1"
  derivation_path: "ethereum/mainnet/custody/0"
  domain:          0x01 || 0xCustodyContractAddress
  batch_nonce:     42
  tx_shape_id:     keccak256("release(address,address,uint256,bytes32)")
  tx_params_hash:  keccak256(recipient || token || amount || finalityProofHash)
  digest32:        keccak256(domain || tx_shape_id || tx_params_hash || 42)
}
```

Rejection cases:

Scenario	Rejection Reason
tx_shape_id = keccak256("transfer(address,uint256)")	Method not in allowlist
batch_nonce = 41 (already used; last signed was 41)	Replay attempt; must be 42
batch_nonce = 43 (last signed was 41)	Skip attempt; must be 42 (strict sequential)
Claimed digest32 ≠ reconstructed digest32	Tampered request
derivation_path = "ethereum/mainnet/attacker/0"	Path not in allowlisted prefixes

Performance targets:

Metric	Target	Validation Method
P50 signing latency	< 1.5 seconds	100 sequential requests
P99 signing latency	< 2 seconds	100 sequential requests
Sustained throughput	≥5 signatures/second (G2); ≥10 sig/sec (G4)	1-hour load with partial node failure
Signature validity	100%	All signatures recoverable to expected address on-chain

Performance targets are enforced at the boundary, not assumed. Custody coordination cannot tolerate long-tail stalls without cascading queue growth.

3.3.4 IVC Pipeline Foundation

Workstream 2 establishes the IVC/folding foundation used to keep proving sustainable under real volume. Workstream 3 performs full settlement submission and verification on Midnight; Workstream 2 builds the pipeline elements and measures whether the fold operation is fast enough to justify the architecture.

What Workstream 2 delivers:

Component	Requirement	Validation
Fold operation	Single fold completes in < 100ms	Benchmark on target hardware
Accumulator state	Correctly tracks intent membership across folds	Unit tests comparing accumulator to intent list
Batch boundary logic	Accumulator finalized at configurable batch size	Integration test with batch size = 10
Memory management	Accumulator fits in TEE memory (< 8GB peak)	Memory profiling under load
Checkpoint integration	Accumulator state recoverable after restart	Recovery test without duplication

Fold integration with TEE:

```

For each intent in batch:
  1. TEE decrypts witness
  2. TEE validates intent against settlement rules
  3. TEE calls Sirius fold(accumulator, witness)
  4. TEE updates accumulator state
  5. TEE checkpoints after each fold

At batch boundary:
  6. TEE calls Sirius finalize(accumulator) → decider proof
  7. TEE checkpoints proof artifact
  8. Workstream 3: Submit proof to Midnight

```

The fold target is concrete and falsifiable: fold latency must be < 100ms per step on the reference circuit in the target runtime profile. If the fold exceeds the target materially, Workstream 2 must surface the constraint early because Workstream 3 cannot "optimize away" a non-viable fold.

Performance targets:

Metric	Target	Gate
Fold latency	< 100ms per step	G2
Batch size	50-100 intents per batch	G2 (configurable)
Memory footprint	< 8GB peak during fold	G2
Decider proof time	< 60s for full batch	G3 (not G2)

Workstream 2 validates fold latency and accumulator correctness. Decider proof generation targeting Midnight's verifier is a Workstream 3 deliverable. Sirius/ProtoGalaxy provides the folding model; §3.6 provides full treatment.

3.3.5 ERC-7683 Integration

Workstream 2 validates NearFall's compatibility with the Ethereum intent ecosystem through the ERC-7683 roundtrip tests specified in §3.1.11. The integration work is to support CrossChainOrder ingestion, destination fill execution through the destination settler interface, and filler repayment release under Midnight-first ordering.

The Workstream 2 completion condition is not "we support the entire ERC-7683 surface." It is that the roundtrip can be executed end-to-end on testnets in a way that respects NearFall's settlement posture. The fill may occur on an external chain, but release of repayment funds from Ethereum custody must remain gated on Midnight finality proof. Workstream 3 completes the full settlement submission and verification path on Midnight; Workstream 2 ensures the intent interface and coordination flows are compatible and testable.

3.3.6 Workstream 2 Deliverables

Deliverable	Owner	Acceptance Criteria	Gate
Custody contract deployed (Sepolia)	Smart Contract	Deposit works; <code>executeSettlement</code> updates <code>stateRoot</code> only under MPC signature; <code>release</code> rejects without valid Midnight finality proof	G2
Typed ECDSA request validator in MPC fork	Cryptography	MPC reconstructs digest from typed fields; rejects mismatches; rejects non-allowlisted shapes; enforces monotonic nonce	G2
ECDSA performance benchmark	Cryptography + QA	P99 < 2s and throughput ≥5 sig/sec (G2 interim; G4 requires ≥10) under load including partial node failure	G2
IVC fold benchmark	Cryptography	Fold < 100ms per step; memory footprint measured; accumulator checkpoint/recovery demonstrated	G2
ERC-7683 roundtrip (RT-1, RT-2)	Runtime + QA	Both test cases pass with Midnight-first ordering and finality-gated release	G2

3.3.7 Workstream 2 Validation Matrix

Test ID	Category	Test Condition	Pass Condition
G2-F1	Functional	Deploy custody contract and perform ETH and ERC-20 deposits	Deposits escrowed; balances update correctly
G2-F2	Functional	Submit <code>executeSettlement</code> with valid typed MPC signature	<code>stateRoot</code> updates; <code>settlementNonce</code> increments; event emitted
G2-F3	Functional	Attempt <code>release</code> with correct parameters but without finality proof	Release rejected deterministically
G2-F4	Functional	Attempt <code>release</code> with valid Midnight finality proof	Release succeeds once; subsequent identical release rejected as duplicate

Test ID	Category	Test Condition	Pass Condition
G2-F5	Functional	Run 100 sequential ECDSA signing requests via typed format	P99 < 2 seconds; 100% signatures verify
G2-F6	Functional	Run sustained signing load with partial MPC node loss	Throughput ≥ 5 sig/sec sustained (G2 interim); degrades without violating correctness
G2-F7	Functional	Run fold benchmark with checkpointing enabled	Fold < 100ms per step; accumulator recovers after restart without duplication
G2-F8	Functional	Execute ERC-7683 RT-1 and RT-2 as in §3.1.11	Orders fill; settlement authority on Midnight; repayment release finality-gated
G2-A1	Adversarial	Submit signing request with mismatched <code>digest32</code> versus reconstruction	MPC rejects; no signature produced
G2-A2	Adversarial	Submit signing request with non-allowlisted <code>tx_shape_id</code>	MPC rejects
G2-A3	Adversarial	Replay prior signing request with same <code>batch_nonce</code>	MPC rejects under monotonicity rule
G2-A4	Adversarial	Attempt <code>executeSettlement</code> with signature not bound to typed fields	Rejected at MPC boundary or contract
G2-A5	Adversarial	Attempt <code>release</code> with forged or stale finality proof	Release rejected deterministically
G2-A6	Adversarial	Attempt to advance <code>settlementNonce</code> out of order	Contract rejects; MPC refuses to sign for stale nonce

Gate G2 pass criteria: NearFall passes Gate G2 only if the performance targets are met and the ordering posture is enforced by construction. Specifically: P99 ECDSA latency must satisfy the bound demonstrated by G2-F5, fold latency must satisfy the bound demonstrated by G2-F7, and ERC-7683 roundtrip must satisfy the

end-to-end criteria demonstrated by G2-F8, with release remaining conditional on Midnight finality rather than Ethereum confirmation.

Failure consequences:

Failure Type	Action
Custody contract tests fail	Debug contract logic; redeploy; re-test
ECDSA latency exceeds target	Profile MPC communication; optimize networking; consider reducing threshold
Fold latency exceeds target	Profile Sirius; check memory allocation; consider hardware upgrade
Roundtrip tests fail	Debug fill handler or repayment logic; re-test
Adversarial tests fail	Fix validation logic; do not proceed until all adversarial tests pass

3.3.8 Workstream 2 Dependencies and Risks

Item	Type	Description	Mitigation
Workstream 1 complete	Dependency	Attested runtime, checkpoint recovery, measurement-bound MPC authorization	G2 tests assume G1 properties; fail fast if missing
Ethereum testnet availability	Dependency	Custody contract and ERC-7683 tests require stable testnets	Use multiple testnets; record chain parameters
Sirius MVP readiness	Dependency	Folding benchmarks depend on usable Sirius/ProtoGalaxy path	Keep fold benchmark circuit minimal; defer architecture depth to §3.6
ECDSA tail latency	Risk	P99 < 2s may fail under realistic load	Benchmark under fault and load; optimize MPC networking early
IVC memory footprint	Risk	Fold may be fast but memory-heavy, causing runtime instability	Measure memory under checkpointing; treat large footprint as scope-impacting
Custody contract audit surface	Risk	Release gating and finality proof verification are security-critical	Keep interface small; require negative tests (G2-A5); audit required before G4

Cross-references:

- Typed request format: §1.4
- Midnight-first finality: §2.2.6
- Checkpoint model: §2.2.7
- ERC-7683 roundtrip test: §3.1.11
- Sirius architecture: §3.6
- Workstream 1 deliverables: §3.2.5

What comes next: With G2 closed, NearFall can hold assets, sign transactions, and produce folded proofs. Workstream 3 connects this to Midnight: proof submission, state root anchoring, and FROST BIP-340 signing for Midnight transactions.

3.4 Workstream 3: Midnight Integration

Blocked by: G2 complete

Readiness Gate: G3

Complexity: High

Workstream 3 is the payoff. Without it, NearFall is infrastructure with no settlement finality. Workstreams 1–2 built the boundary and the machinery: an attested TEE runtime (§3.2), typed MPC authorization and Ethereum custody coordination (§3.3), and the Sirius decider proof path (§3.6). Workstream 3 connects those components to Midnight's on-chain authority so that correctness is anchored where it belongs: on the chain that verifies proofs and stores the canonical settlement root (§2.4).

Workstream 3 proves one core claim under testnet conditions: NearFall can submit a batch to Midnight, Midnight verifies it under the Compact contract acceptance rule, NearFall observes finality, and only then do downstream coordination actions become eligible to complete. Midnight does not trust the TEE; Midnight verifies the proof. The contract is the system's external memory; if it accepts the wrong thing once, everything downstream is contaminated. Signing is policy enforcement, not a cryptographic vending machine; the signing layer enforces measurement, replay prevention, and typed transaction shape. Finality is a state machine input; "included" and "finalized" are distinct, and NearFall does not mark a batch settled until the configured finality condition is met.

3.4.1 Workstream 3 Objectives

Workstream 3 establishes Midnight integration as a falsifiable, repeatable pipeline. Production hardening remains Workstream 4 (§3.5); Workstream 3 focuses on correctness anchoring, signature validity, and finality observation on Midnight testnet.

Objective	Success Condition (Falsifiable)	Test Condition
Contract-anchored settlement works	Midnight accepts a valid batch, updates <code>currentStateRoot</code> , and emits an acceptance event; the same batch cannot corrupt state under replay attempts	Submit two consecutive valid batches; verify root continuity and monotonic nonce; verify events and on-chain state
Proof acceptance is real	Proofs generated by the selected proof path (Path A: native; Path B: Sirius decider per §3.6) are accepted by the settlement contract and invalid proofs are rejected	Submit 100 valid proofs; submit 10 invalid proofs spanning wrong root, wrong manifest, and malformed proof; observe deterministic rejection
BIP-340 signing is correct	Midnight ledger transactions are signed with BIP-340 via a FROST network and are accepted on testnet	Execute 100/100 signing requests; confirm 100/100 transactions accepted by a Midnight node and included on-chain
Attestation gates signing	Signing requests without the approved enclave measurement are rejected; replay attempts are rejected	Attempt signing with wrong measurement; attempt signing with stale nonce; attempt signing with wrong transaction shape
Finality is observed and checkpointed	NearFall distinguishes "included" from "finalized," checkpoints both states, and resumes after restart without double submission	Induce crashes at defined points; recover; observe no duplicate signing and no incorrect "settled" marking

What Workstream 3 does NOT do: Production hardening. Workstream 3 operates on testnet with controlled load. Rate limiting, abuse controls, monitoring dashboards, and chaos testing are Workstream 4 deliverables. Workstream 3 proves the mechanism works; Workstream 4 proves it survives.

3.4.2 The Settlement Contract

The settlement contract is NearFall's external memory on Midnight. It stores the current state root, tracks pending settlements awaiting Ethereum confirmation, enforces monotonic batch ordering, and emits events for reconciliation. If it accepts the wrong thing once, everything downstream is contaminated.

Workstream 3 deploys and exercises the Compact settlement contract interface defined in §2.4.4. The contract implements the **pending/commit model** from §6.2:

Contract State	Type	Role
currentStateRoot	bytes32	Anchors the canonical settlement state
lastBatchNonce	uint64	Enforces monotonic replay prevention
pendingSettlements	map	Tracks batches awaiting Ethereum release confirmation

Entry points (pending/commit model):

Entry Point	Parameters	Acceptance Rule
recordPending	(proof, batchNonce, oldRoot, newRoot, batchCommitment, expiresAt)	Accept if oldRoot == currentStateRoot, batchNonce > lastBatchNonce, and proof verifies

Nonce policy (two layers):

- **MPC signing layer:** Strictly sequential (batchNonce == lastSignedNonce + 1). Gaps rejected.
- **Contract layer:** Monotonic (batchNonce > lastBatchNonce). Allows retry if a signed transaction fails on-chain.

commit	(batchNonce, ethReleaseProof)	Accept if batch is PENDING , not expired, and Ethereum release proof verifies
expirePending	(batchNonce)	Accept if batch is PENDING and <code>currentBlock > expiresAt</code>

State transitions:

Action	Trigger	Effect
recordPending	Proof verified	<code>currentStateRoot = newRoot</code> ; assets reserved; emit <code>BatchPending(...)</code>
commit	Ethereum release confirmed	Transfers finalized; emit <code>BatchCommitted(...)</code>
expirePending	Timeout without Ethereum release	Assets returned; emit <code>BatchExpired(...)</code>

Why pending/commit: Single-step settlement (`submitBatch` → immediate transfer) creates the atomicity vulnerability in §6.2.8: Midnight commits before Ethereum releases, leaving one party with assets on both chains. The pending/commit separation ensures Midnight transfers finalize only after Ethereum release is confirmed.

Idempotency is part of correctness, not a convenience. The contract must treat a repeated submission for an already-processed `batchNonce` as a deterministic rejection. Workstream 3 tests idempotency directly by re-submitting after each phase transition.

3.4.3 Proof-to-Contract Interface

> **Note:** This section describes the proof binding mechanism. The specific proof source depends on Gate G0.5 outcome:

> - **Path A (Native Contract):** Proof artifact is a standard Midnight contract execution proof > - **Path B (Verifier Gadget):** Proof artifact is a Sirius decider proof verified via gadget > > Workstream 3 criteria apply to whichever path G0.5 selects. References to "Sirius decider proofs" below apply only if Path B is chosen.

If Path B (Sirius) is selected at G0.5:

Workstream 3 binds Sirius decider proofs to the contract's acceptance rule. Sirius produces a decider proof at the batch boundary (§3.6). That proof is the bridge between NearFall's private computation and Midnight's public settlement authority: it turns "the TEE says it ran the batch" into "the chain verified the batch."

If Path A (Native Contract) is selected at G0.5:

Workstream 3 binds batch execution proofs to the contract's acceptance rule using Midnight's native proof verification. The proof artifact is a standard Midnight-verifiable proof without folding infrastructure dependency.

Canonical commitment model (both paths):

The contract uses `batchCommitment` as the single canonical commitment (see §2.4.3). The proof validates this commitment; the contract stores only this commitment; sub-commitments are reconstructible from batch data.

Proof Public Input	Meaning	Stored On-Chain?
<code>oldRoot</code>	Settlement state before batch	No (checked against <code>currentStateRoot</code>)
<code>newRoot</code>	Settlement state after batch	Yes (becomes new <code>currentStateRoot</code>)
<code>batchCommitment</code>	Merkle root over header leaf + fill leaves (see §2.4.3)	Yes (in <code>pendingSettlements</code> map and events)

What `batchCommitment` binds (Merkle structure):

Leaf Type	Contents	Purpose
<code>headerLeaf</code>	<code>batchNonce</code> , <code>oldRoot</code> , <code>newRoot</code> , <code>count</code> , <code>witnessDigest</code>	Binds batch metadata; always first leaf
<code>fillLeaf</code>	<code>lock_id</code> , <code>intent_id</code> , <code>asset</code> , <code>amount</code> , <code>recipient_eth</code> , <code>recipient_midnight</code>	Per-fill data; enables membership proofs

Why Merkle root matters: Ethereum release requires `verify_merkle_proof(proof, batchCommitment, lock_id)` to confirm a specific lock is included in the batch. The fill leaves enable this membership verification without revealing the entire batch.

The contract verification logic treats the proof as a standard Midnight-verifiable proof artifact under Midnight's PLONK/KZG verifier on BLS12-381 (§2.4). Curve compatibility is resolved before Workstream 3 starts (Gate G0.5); Workstream 3 does not carry curve uncertainty.

Workstream 3 tests binding failures as first-class adversarial cases. A proof that verifies for a different `batchCommitment` must fail. A proof that verifies for the wrong `oldRoot` must fail. These failures demonstrate

that acceptance is anchored in the statement, not in the transport.

3.4.4 FROST BIP-340 Signing

Midnight ledger transactions require BIP-340 Schnorr signatures (§2.4.7). The NEAR MPC fork provides ECDSA for Ethereum coordination (§3.3). These are different algorithms on the same curve; one cannot substitute for the other. NearFall therefore operates two MPC networks:

Chain	Signature Scheme	MPC Backend
Ethereum	ECDSA secp256k1	NEAR codebase fork
Midnight	BIP-340 Schnorr	FROST MPC (separate network)

Workstream 3 brings up the FROST-based MPC signing network for BIP-340 and integrates it as a policy boundary in the same style as Workstream 1: measurement-bound attestation gates signing, replay prevention blocks duplicates, and typed transaction shapes prevent "sign arbitrary bytes."

Policy Requirement	Enforced By	Rejection Condition
Measurement-bound authorization	Attestation check against allowlist (§3.2)	Wrong enclave measurement rejects signing request
Monotonic replay prevention	batchNonce monotonicity for the Midnight signing scope	Stale or repeated nonce rejects signing request
Digest-format validation	Typed transaction shape policy	A digest that does not match an allowlisted Midnight transaction shape is rejected

Key derivation paths:

Purpose	Derivation Path	Network Scope
Midnight testnet settlement	midnight/testnet/nearfall/settlement/{index}	Workstream 3
Midnight mainnet settlement	midnight/mainnet/nearfall/settlement/{index}	Workstream 4

Signing flow: The TEE constructs the `midnight_tx` payload for `recordPending` (or `commit`), derives the typed digest under the network domain separator, and submits a signing request containing `(measurement, batchNonce, txShapeId, txParamsHash, digest32)`. The FROST network reconstructs the digest from typed components and signs only if reconstruction matches and the nonce is fresh. The TEE attaches the signature and submits the transaction.

Workstream 3 demonstrates 100/100 successful signing and acceptance and demonstrates rejection on wrong measurement, wrong nonce, and wrong transaction shape.

3.4.5 Transaction Pipeline

Workstream 3 implements the Midnight transaction pipeline from proof artifact to finalized on-chain state. The pipeline is built for correctness first and records enough checkpoint state to survive crashes without double submission.

Pipeline Phase	Input	Output	Checkpoint Record
Build	(proof, batchNonce, oldRoot, newRoot, batchCommitment)	midnight_tx, digest32, txShapeId, txParamsHash	READY_TO_SIGN with digest and params hash
Sign	digest32 + attestation + nonce	bip340_signature	SIGNED with signature and signing receipt
Submit	signed midnight_tx	tx_hash, inclusion status	SUBMITTED with tx hash
Observe	node status and chain events	inclusion and finality markers	INCLUDED then FINALIZED markers
Commit	finality observed	batch marked settled	SETTLED with block height and event reference

Finality is a state machine input. NearFall distinguishes "included" and "finalized" explicitly. Workstream 3 defines settlement completion as finality, not inclusion. A batch is not treated as settled for downstream coordination until finality is observed and checkpointed.

Checkpoint contents:

Field	Purpose
batch_id	Unique batch identifier
batch_nonce	Monotonic ordering key
proof_artifact	Decider proof (if generated)
signature	BIP-340 signature (if signed)
tx_hash	Transaction hash (if submitted)
block_height	Block of inclusion (if included)
finality_marker	Finality status (if finalized)

Field	Purpose
phase	Current pipeline phase (READY_TO_SIGN, SIGNED, SUBMITTED, INCLUDED, FINALIZED, SETTLED)

Recovery loads the checkpoint and resumes from the recorded phase.

3.4.6 Testnet Validation Requirements

Workstream 3 runs on Midnight testnet. The testnet requirement is not "it works once"; it is repeatable acceptance and repeatable rejection under defined error conditions.

Proof acceptance tests:

Test	Condition	Pass Criteria
Valid proof acceptance	Submit 100 valid decider proofs	100/100 accepted; state roots updated
Invalid proof rejection	Submit 10 proofs with mutated coefficients	10/10 rejected with stable failure behavior
Wrong oldRoot rejection	Submit proof with stale root	Rejected with root mismatch error
Stale nonce rejection	Re-submit prior batch	Rejected with nonce error
Manifest mismatch rejection	Submit valid proof with wrong batchCommitment	Rejected; state unchanged

Signing tests:

Test	Condition	Pass Criteria
Valid signature	Sign 100 transactions with FROST	100/100 signatures accepted by Midnight node
Attestation enforcement	Request signature without valid attestation	Rejected by FROST network
Replay enforcement	Request signature for prior nonce	Rejected by FROST network
Wrong transaction shape	Request signature for disallowed tx type	Rejected by FROST network

Latency targets:

Metric	Target	Measurement
Proof submission → inclusion	< 10s	Median over 100 submissions
Inclusion → finality	< 10s	Median (GRANDPA ~6s expected)
End-to-end (submit → finality)	< 30s	P95 over 100 submissions
FROST signing latency	< 2.5s	P95 over 100 signing requests

3.4.7 Failure Modes and Recovery

Workstream 3 defines recovery as state-machine correctness under crash and partial outage, not as "the process restarted." Recovery is checkpoint-based and must preserve the monotonic signing and settlement invariants established in Workstreams 1–2.

Failure Mode	Crash Point	Required Recovery Outcome	Invariant Preserved
Crash after proof generation, before signing	READY_TO_SIGN	Resume and request signing once; no duplicate signing for the same nonce	Monotonic nonce and typed digest binding
Crash after signing, before submission	SIGNED	Recover signed transaction and submit; no second signature for the same digest	No double-sign and no digest drift
Crash after submission, before finality observation	SUBMITTED or INCLUDED	Resume status polling; do not resubmit if tx hash is already included; mark settled only at finality	Inclusion/finality distinction
MPC node failure during signing	READY_TO_SIGN	Retry on quorum restoration; refuse downgrade to weaker policy	Attestation gate and nonce monotonicity
Midnight node unavailable	SIGNED or SUBMITTED	Switch to backup RPC endpoint; preserve tx hash and do not rebuild transaction	Idempotent submission and stable digest
Chain reorg under inclusion	INCLUDED	Continue observation until finality condition; if reverted, resubmit the same signed tx if still valid under nonce policy	Finality as completion condition

Workstream 3 validates these failures through scripted crashes and restarts and records evidence that duplicates do not occur.

3.4.8 Deliverables and Acceptance Criteria

Deliverable	Acceptance Criteria	Gate
Settlement contract deployed	Contract deployed on Midnight testnet; callable; emits acceptance events	G3
Decider proof integration	Sirius decider proof accepted by contract for 100/100 valid batches; rejects invalid proofs	G3
FROST signing operational	100/100 BIP-340 signatures correct and accepted by Midnight node	G3
Midnight transaction pipeline	Build → sign → submit → finality observed; checkpoints record tx hash and finality marker	G3
End-to-end roundtrip	Intent → TEE → fold (§3.6) → decider proof → sign → submit → finality	G3

Gate G3 pass criteria: Gate G3 passes when a batch of 100 intents can be folded in the TEE, compressed to a decider proof, signed by the FROST network, submitted to Midnight testnet, verified by the settlement contract, and observed as final—with the entire flow surviving a TEE restart at any checkpoint.

Gate G3 failure actions:

Failure Condition	Action
Proof acceptance fails	Investigate proof format or verifier compatibility; escalate to G0.5 re-evaluation if curve mismatch
FROST signing correctness fails	Review FROST implementation selection; extend audit scope
FROST signing meets correctness but misses latency	Optimize network topology; if structural, accept degraded latency with documented tradeoff
Finality observation unreliable	Investigate RPC provider; implement redundant finality observation; add fallback attestation path
Contract deployment fails	Debug Compact contract; verify testnet compatibility
3+ adversarial tests fail	Stop and remediate before proceeding

3.4.9 Validation Matrix

Test ID	Category	Test Condition	Pass Condition
S4-F1	Functional	Submit a valid batch with correct <code>oldRoot</code> , <code>newRoot</code> , <code>batchCommitment</code>	Contract accepts; root updates; acceptance event emitted
S4-F2	Functional	Submit a second valid batch with incremented <code>batchNonce</code> and correct <code>oldRoot</code>	Contract accepts; nonce advances; root continuity holds
S4-F3	Functional	Query contract state after acceptance	<code>currentStateRoot</code> equals expected <code>newRoot</code> ; <code>lastBatchNonce</code> equals expected nonce
S4-F4	Functional	Observe transaction lifecycle	Included and finalized states recorded; batch marked settled only at finality

Test ID	Category	Test Condition	Pass Condition
S4-A1	Adversarial	Submit with wrong <code>oldRoot</code>	Contract rejects; state unchanged
S4-A2	Adversarial	Submit with stale <code>batchNonce</code>	Contract rejects; state unchanged
S4-A3	Adversarial	Submit an invalid proof for correct inputs	Contract rejects; state unchanged
S4-A4	Adversarial	Submit a valid proof with mismatched <code>batchCommitment</code>	Contract rejects; state unchanged
S4-A5	Adversarial	Request FROST signing without approved attestation measurement	Signing request rejected; no signature produced
S4-A6	Adversarial	Replay a signing request with the same <code>batchNonce</code> and digest	Signing request rejected under monotonicity rule
S4-P1	Performance	Measure submit → finality latency for 20 consecutive batches	Finality observed within 30 seconds per batch under testnet conditions
S4-R1	Recovery	Crash after signing and restart	Recovery submits the signed tx once; no double signing; finality observed

Gate G3 passes when the functional tests S4-F1 through S4-F4 pass, the adversarial tests S4-A1 through S4-A6 pass, the performance test S4-P1 meets the bound, and the recovery test S4-R1 demonstrates checkpoint correctness. These tests operationalize the requirements in §2.4, depend on the boundary established in §3.2, assume the coordination and typed-signing foundation in §3.3, and consume the Sirius decider proof path in §3.6. The FROST MPC analysis and policy details are consolidated in §4.2.

3.5 Workstream 4: Production Hardening

Blocked by: G3 complete

Readiness Gate: G4

Complexity: High

Workstream 4 is the difference between "works in a lab" and "survives production." Workstreams 1–3 proved the mechanism is correct: the TEE boundary exists (§3.2), typed MPC signing drives external-chain coordination (§3.3), Sirius produces decider proofs (§3.6), and Midnight verifies and finalizes settlement (§3.4). Workstream 4 proves the system survives: sustained load, crash loops, network partitions, malicious inputs, operator mistakes, and long-running operation without safety violations. Mainnet deployment is the outcome, not the test; the test is that the system can pass an endurance test under production-equivalent conditions and remain safe (§3.1).

Workstream 4 is structured around Gate G4 as defined in §3.1.7 and the production deliverables in §3.1.8. Chaos scenarios exercise the failure modes defined in §3.4.7. Sirius hardening deliverables are tracked under §3.6.8. Break-glass recovery and the committee mechanism are specified in §3.1.8; Workstream 4 operationalizes them through drills rather than restating their design.

3.5.1 Workstream 4 Objectives

Workstream 4 proves one core claim under defined conditions: the system can pass an endurance test at production-equivalent load and fault profile without violating safety invariants, while meeting latency and throughput targets, and while producing operator-visible signals and runbook actions that work under pressure.

Objective	Success Condition (Falsifiable)	Test Condition
Survival under production conditions	Endurance test at 99.9% uptime with no safety violations and no data loss	Continuous endurance test with production-equivalent workload; failures counted by definitions in §3.5.6
Performance under load	P99 signing latency < 2s at 10 req/sec load and > 10 sig/sec sustained for 1 hour (§3.1.7)	Controlled load windows during endurance test with stable measurement and recorded percentiles
Fault tolerance under chaos	Chaos suite passes with 0 failures across 400 runs (4 scenarios × 100 iterations), preserving invariants	Inject chaos scenarios under 10 req/sec load; verify invariant set after each injection
Operator control and observability	Alerts page within 5 minutes and runbook actions succeed in drills (§3.1.7)	Execute alert and drill scenarios; measure page time and action completion time
Audit readiness	All Critical and High audit findings resolved before G4 passes	Audit findings closure evidenced by retest and signoff (§3.5.7)

Production conditions mean: Load, faults, and adversarial inputs that reflect how the system will be attacked and how it will fail in reality. Workstream 4 treats these as test inputs, not as background anxiety.

What Workstream 4 does NOT do: Mainnet deployment is the outcome, not the test. Workstream 4 validates on testnet with production conditions. Mainnet contract deployment, MPC key ceremony for mainnet, and go-live happen after G4 passes.

3.5.2 Observability Infrastructure

Workstream 4 requires observability that is both complete and privacy-preserving. The system must support a per-request trace from intent ingress to final settlement observation, and it must do so without leaking witness material, intent plaintext, or other confidential state through logs or traces. The confidentiality boundary established in §3.2 must survive the addition of monitoring.

Distributed tracing is required at the level of system phases and identifiers, not payloads. Each request is traced by `intent_id`, `batch_id`, and `batch_nonce` through TEE execution, MPC signing, transaction submission, and finality observation (§3.4.5). Traces must allow an operator to answer, for a specific batch: is the system waiting on proving, signing, submission, or finality?

Metrics are required for the variables that control stability and economics: latency percentiles for signing and proving, throughput, error rates, queue depth, checkpoint recovery frequency, and finality observation delays.

Operator dashboard requirements:

Signal	Metric	Required Operator View
Signing performance	P50/P95/P99 latency, throughput	Time-series percentiles with tail-latency alarms
Proving and folding	Fold latency, decider time, memory peak	Batch-level distribution and regression alarms
Safety invariants	Nonce monotonicity, replay rejections, duplicate detection	Counters plus event logs for anomaly triage
Pipeline health	Queue depth, batch cadence, checkpoint recoveries	Backpressure indicators and saturation warnings
Chain health	Submission success, inclusion time, finality time	Included vs finalized separation and delays

Alerting requirements:

Alert	Condition	Response Time
Latency spike	P99 > 2s for 5 min	Page within 5 min
Signing failure	3 consecutive failures	Page within 5 min
Attestation rejection	Any rejection	Page within 5 min
TEE restart	Enclave restarted	Page within 5 min
MPC quorum degraded	< threshold nodes available	Page within 5 min

Logs are treated as a hostile exfiltration surface. Logs must contain only identifiers, commitments, and structured error codes. Sensitive fields are replaced with hashes or redacted markers. Log aggregation is encrypted at rest and in transit. Workstream 4 validates this through log audits during endurance testing and during chaos injection.

3.5.3 Rate Limiting and Abuse Controls

Workstream 4 introduces rate limiting and abuse controls as safety mechanisms, not as traffic shaping. NearFall must fail fast under abuse and must not queue unbounded work that converts adversarial inputs into resource exhaustion. The rate limiting policy is enforced at three choke points: intent ingestion, proof generation, and signing request issuance.

Intent ingestion limits protect the TEE from becoming a memory and queue sink. Proof generation limits protect CPU and memory so that one workload cannot starve checkpointing and recovery. Signing request limits protect the MPC cluster from becoming the system-wide bottleneck that cascades into timeouts and retried submissions.

Rate limit tiers:

Resource	Limit	Scope	Rejection Behavior
Intent ingestion	100/min per user	User ID	429 response; retry after backoff
Intent ingestion	1000/min global	System	503 response; queue full
Signing requests	20/min per batch scope	Batch nonce namespace	Reject; log abuse
Proof generation	10 concurrent	TEE instance	Queue; timeout after 120s

Abuse patterns and responses:

Control Plane	Abuse Pattern	Required Response
Intent ingress	Malformed schema, expired intents, high-rate floods	Reject at boundary; constant-time reject path; bounded memory growth
Proving/folding	Pathological batches, memory pressure triggers	Shed load by deferring or rejecting; preserve checkpoint integrity
Signing	Replay attempts, shape mismatches, nonce stalls	Reject at policy layer; do not degrade to signing arbitrary bytes

Principle: Fail fast, do not queue unboundedly. The system must not "try harder" when an attacker floods it. Legitimate users retry with backoff; attackers hit rate limits. Workstream 4 validates that legitimate traffic at 80% of rate limit maintains P99 < 2s while concurrent abuse traffic at 10× rate limit is rejected at the boundary with deterministic 429 responses.

3.5.4 Chaos Testing Protocol

Workstream 4 executes the chaos protocol defined in §3.1.7 and uses it to exercise the failure modes described in §3.4.7. Chaos testing is not optional and not performed at idle. Failures are injected under baseline load and the system must recover within the defined bounds while preserving invariants.

The invariants are fixed across all chaos scenarios: No duplicate signatures, no lost requests that were accepted, and no state corruption at the settlement authority. Safety violations terminate the test run and trigger

incident response.

Chaos scenarios:

Scenario	Failure Surface	Invariant Set
TEE crash mid-fold	Sirius accumulator continuity (§3.6) and checkpoint durability (§3.2)	No state rollback, no lost accepted intents, no duplicate signing
TEE crash post-sign	Signing idempotency (§3.4.7)	Signed transaction retrieved, no re-signing, eventual submission
MPC node failure	Quorum behavior under partial loss (§3.1.7)	Signing continues or fails closed; no incorrect authorization
Network partition	Coordination between TEE and MPC (§3.1.7)	No partial authorization; recovery resumes without replay

Protocol (per §3.1.7):

- Start load test at 10 req/sec baseline
- Inject failure at random point in request lifecycle
- Verify: no duplicate signatures, no lost requests, no state corruption
- Verify: system recovers to baseline within configured recovery window
- Repeat 100 times per scenario

Pass criteria: 0 failures across 400 runs (4 scenarios × 100 iterations). A single failure requires root cause analysis and re-run of the full suite after fix. Each run records pre-injection state, injection timestamp, post-recovery state, and verification results. The evidence package is a G4 artifact.

Workstream 4 measures outcomes as pass/fail against the chaos protocol requirements. Any deviation is treated as a correctness failure, not as an operational nuisance.

3.5.5 Incident Response Framework

Workstream 4 operationalizes incident response as a practiced capability. Runbooks are required by §3.1.7 and the break-glass mechanism is specified in §3.1.8; Workstream 4 makes them executable through drills.

Severity classification is enforced so that the system does not rely on ad hoc judgment during a crisis. A safety violation is always treated as the highest severity.

Severity classification:

Severity	Definition	Response Time	Example
S1 (Critical)	Safety violation or funds at risk	Immediate; all hands	Double-sign detected; key compromise
S2 (High)	Service unavailable	< 30 min	TEE cannot restart; MPC quorum lost
S3 (Medium)	Degraded performance	< 4 hours	Latency elevated; partial node failure
S4 (Low)	Minor issue	Next business day	Dashboard not loading; log gap

Escalation path:

Time Since Detection	Action
0 min	On-call engineer paged
15 min	Secondary engineer joined
30 min (S1/S2)	Engineering lead notified
1 hour (S1)	Executive notified; break-glass evaluation

Runbook requirements (G4 artifacts):

Runbook	Trigger	Required Outcome
Measurement revocation	Suspected TEE compromise or untrusted build	Compromised measurement blocked by MPC policy within response window
Key rotation	Suspected key exposure or scheduled rotation	MPC keys rotated without downtime and without breaking nonce monotonicity
Break-glass activation	Safety risk requiring immediate halt	Committee mechanism invoked per §3.1.8; system transitions to safe mode

Workstream 4 requires drills, not documents. Operators must execute revocation and rotation drills and record time-to-completion and correctness. Post-incident review is required after drills and after any real incident; the output is an action list with owners and closure criteria.

3.5.6 Endurance Test

The endurance test is the final exam. It runs on production-equivalent infrastructure and uses production-equivalent load while targeting Midnight testnet for settlement observation. Multi-cloud deployment is included: GCP remains primary, AWS serves as secondary, and Azure remains tertiary per §3.2.

Test conditions:

Parameter	Value
Duration	Defined before test begins; not adjustable once test starts
Infrastructure	Multi-cloud (GCP primary, AWS secondary, Azure tertiary per §3.2)
Load	Baseline 5 req/sec; peak 15 req/sec during stress windows
Target chain	Midnight testnet (real network, real finality)
Chaos injection	None during endurance test (chaos suite runs separately)

The test is not "no alerts." The test is "alerts fire when they should, operators respond correctly, and the system remains safe."

Metrics tracked:

Metric	Target	Failure Definition
Uptime	99.9%	Cumulative downtime exceeds 0.1% of test duration
Data loss	0	Any intent accepted but not settled
Safety violations	0	Duplicate signature, state corruption, or unauthorized signing
Latency regression	< 20% vs baseline	P99 > 2.4s sustained

Downtime definition: The system is "down" if it cannot accept new intents OR cannot progress existing intents toward settlement. Planned maintenance windows (announced in advance, bounded duration) do not count against uptime if no intents are lost.

Early termination triggers:

Trigger	Action
Safety violation detected	Stop endurance test; investigate; restart clock after fix
Data loss detected	Stop endurance test; investigate; restart clock after fix
Uptime drops below 99%	Evaluate root cause; continue if recoverable within bounded window

The endurance test clock restarts after any early termination. G4 requires the full endurance period to complete with clean results. An endurance test that continues through safety violations is not an endurance test; it is denial.

3.5.7 Audit Integration

Workstream 4 integrates audit work as a deployment gate, not as an afterthought. Audit runs in parallel with the endurance test for efficiency, but its findings gate G4. The audit scope covers the boundaries that define NearFall's security: TEE boundary and attestation gating (§3.2), typed MPC signing policy (§3.3, §3.4), Sirius folding and decider proof (§3.6), and the settlement contract acceptance rule (§3.4, §2.4).

Audit scope (per §3.1.9):

Component	In Scope	Risk Level
Sirius fork (folding/IVC)	■	Critical
FROST MPC integration	■	Critical
TEE enclave code	■	High
Compact settlement contract	■	Critical
Ethereum custody contract	■	High
Intent schema + serialization	■	Medium

Finding severity and resolution requirements:

Severity	Definition	Requirement Before G4
Critical	Exploitable vulnerability; funds at risk	Resolved and retested
High	Significant vulnerability; requires specific conditions	Resolved and retested
Medium	Moderate risk; defense in depth concern	Tracked with owner and closure plan
Low	Minor issue; best practice deviation	Documented and accepted

Sequence:

Milestone	Activity
Audit start	Audit begins; initial findings
Remediation	Remediation of Critical/High findings
Re-audit	Re-audit of fixes; final report
Closure	Medium findings tracked; G4 evaluation

Re-audit triggers: Any Critical finding requires re-audit of the affected component after fix. High findings require auditor sign-off on the fix. Material changes to signing policy, settlement contract, or proof interface trigger re-audit of affected components.

Audit package artifact bundle (Gap 2.7 resolution):

The "Audit Package" deliverable for G4 is a concrete bundle with the following completeness checklist:

Artifact	Description	Required
Threat model document	Documented threat vectors, trust assumptions, and mitigations	■
Attack vector repository	Test cases for each identified threat vector	■
Reproducibility scripts	Scripts that reproduce audited build from pinned dependencies	■
Dependency audit output	Vulnerability scan results for all dependencies	■
Gate evidence bundles	Collected evidence from G0.5, G1, G2, G3	■
Final audit report	External auditor report with all findings	■
Remediation evidence	Fixes for Critical/High findings with test results	■

Artifact	Description	Required
Acceptance signatures	Sign-off from security lead and project lead	■

G4 cannot pass unless all audit package artifacts are complete and archived.

3.5.8 Deliverables and Acceptance Criteria

Deliverable	Acceptance Criteria	Gate
Observability stack	Tracing, metrics, alerting operational; paging within 5 min on defined signals (§3.1.7)	G4
Rate limiting and abuse controls	Abuse patterns blocked; bounded queues; legitimate traffic within SLO	G4
Chaos suite	400 runs (4 scenarios × 100 iterations), 0 failures under protocol in §3.1.7	G4
Runbooks and drills	Revocation, rotation, break-glass executed in drills with documented results	G4
Endurance test	99.9% uptime, 0 safety violations, 0 data loss, performance bounds met (duration defined before test begins and recorded in gate evidence)	G4
Audit resolution	All Critical/High findings resolved and retested	G4
Mainnet deployment plan	Mainnet deployment artifacts prepared as outcome after G4, aligned to §3.1.8	G4

Gate G4 pass criteria: Gate G4 passes when the chaos suite completes with 0 failures, the endurance test achieves 99.9% uptime with no safety violations, all Critical/High audit findings are resolved and retested, and operators have executed each runbook drill with documented results.

Sirius is not treated as optional infrastructure; it is hardened under the same survival requirements as the rest of the system. Sirius hardening deliverables are tracked in [§3.6.8](#).

3.5.9 Validation Matrix

Test ID	Category	Test Condition	Pass Condition
S5-F1	Functional	Sustained operation for endurance period (duration defined before test begins; recorded in gate evidence with workload profile)	99.9% uptime, 0 data loss, 0 safety violations
S5-F2	Functional	Process 1000 intents under normal conditions	All intents settle correctly; reconciled to on-chain events
S5-F3	Functional	Trigger alert on signing latency spike	Page received within 5 min and acknowledged
S5-C1	Chaos	TEE crash mid-fold, 100 iterations	0 duplicate signatures, 0 lost accepted intents, no accumulator corruption
S5-C2	Chaos	TEE crash post-sign, 100 iterations	Signed tx retrieved and submitted; no re-signing
S5-C3	Chaos	MPC node failure, 100 iterations	Signing continues or fails closed; no incorrect authorization
S5-C4	Chaos	Network partition, 100 iterations	No partial authorization; recovery resumes without replay
S5-A1	Adversarial	Flood with malformed intents during baseline load	Rate-limited; bounded resource use; legitimate traffic within envelope
S5-A2	Adversarial	Replay attack on signing requests	Rejected by monotonic nonce check
S5-P1	Performance	P99 signing latency under 10 req/sec load	< 2 seconds
S5-P2	Performance	Sustained throughput for 1 hour	> 10 sig/sec
S5-R1	Runbook	Execute measurement revocation drill	Compromised measurement blocked within response window (§3.1.7)
S5-R2	Runbook	Execute key rotation drill	Keys rotated without downtime; nonce monotonicity preserved

Gate G4 passes when S5-F1 demonstrates the endurance requirements, S5-C1 through S5-C4 demonstrate chaos survival under protocol conditions, S5-P1 and S5-P2 meet the performance bounds defined in [§3.1.7](#), S5-R1 and S5-R2 demonstrate operator control, and the audit requirement in [§3.5.7](#) is satisfied. A system that

passes G3 and fails G4 is not deployable to mainnet because it has not demonstrated survival of the boundaries established in §3.2, §3.3, §3.4, and §3.6.

3.6 Sirius: Folding Infrastructure

Sirius is core infrastructure. NearFall's economic model assumes that proving cost is amortized across many intents; without folding, proving cost scales linearly with intent count and the service becomes a throughput tax rather than a product. With folding, NearFall turns many intent-validity checks into one decider proof and pays the verification cost once per batch, not once per intent. This section explains what Sirius does, where it sits in the TEE pipeline, what it must output for Midnight verification, and what must be proven true before Workstream 3 can depend on it.

3.6.1 Why Folding Is Essential

NearFall offers proof outsourcing as a product capability: the user does not run a prover locally; the TEE does. In that model, the unit of cost is not "one proof," it is "one intent." If each intent requires a full independent proof, the proving service cost scales linearly and quickly dominates. Folding changes the shape of the cost curve: the system does small per-intent work to fold each step into an accumulator, and then pays for one decider proof per batch.

The economics, concretely:

Scenario	Proving Work	On-Chain Verifications	Cost Structure
Per-intent (100 intents)	$3s \times 100 = 300s$	100	Linear in N
Folded (100 intents)	$30ms \times 100 + 5s$ compress = 8s	1	Sublinear in N

At 1,000 intents per day, per-intent proving consumes 50 minutes of TEE compute and pays for 1,000 separate verifications. Folded proving consumes ~35 seconds and pays for one verification. At 10,000 intents, folding is 100× cheaper in verification cost alone.

Toy model (why this matters for product): Suppose a "full proof + verification" path costs a fixed amount per proof, and a batch is sized to 100 intents. Without folding, 100 users each pay the full per-proof cost. With folding, the batch pays one decider proof cost, and each additional intent pays only the marginal fold cost, which is designed to be small. This is the difference between 100 users each paying 1 cent and 100 users splitting 1 dollar: both can add to the same total, but only one of them scales to millions of users without exploding the operator cost.

Concrete example: A batch of 100 intents is processed inside the TEE. Each intent contributes one fold step that updates the accumulator. The fold steps are checkpointed. At the batch boundary, the TEE generates one

decider proof that is compatible with Midnight's verifier. Midnight verifies once and anchors the resulting root. The batch becomes accountable and final; custody releases gated on Midnight finality can proceed.

3.6.2 Sirius Architecture

Sirius is NearFall's folding infrastructure based on ProtoGalaxy-style accumulation of multiple instances. It implements an accumulator-based IVC model in which each folding step combines the current accumulator with one or more new instances, producing a new accumulator whose correctness implies correctness of everything folded so far. ProtoGalaxy's design goal is to keep the verifier's marginal work small, especially when folding multiple instances per step, so that batching trades off cleanly against decider cost.

Why ProtoGalaxy: ProtoGalaxy (Eagen & Gabizon, 2024) uses Lagrange-base folding instead of monomial-base, eliminating group operations from the verifier entirely. The Protostar paper (Bünz & Chen, 2023) describes a generic accumulation compiler where the verifier performs $k+2$ elliptic curve multiplications. ProtoGalaxy improves on this: the verifier performs only $d + \log(n)$ field operations and hash calls, where d is gate degree and n is witness size.

Scheme	Verifier Work	Lookup Support	PLONKish Native
Nova	$2 G + O(1) F$	■	■ (R1CS)
Protostar	$2 G + d + O(1) F$	■ ($\log U_p$)	■
ProtoGalaxy	$d + \log(n) F$	■	■

Sirius components and falsifiable requirements:

Component	Role	What Must Be True (Falsifiable)
Fold operation	Incorporates new intent instances into accumulator	Updated accumulator can be decided successfully if and only if all folded instances were valid
Accumulator state	Running object that commits to folded history	Commits to ordered history and current state root used by settlement
Decider circuit	Produces succinct proof that accumulator is valid	Decider proof implies validity of all folded instances and is verifiable by Midnight
Verifier	On-chain verification mechanism	Verification cost is bounded per batch and does not scale with intent count
Compatibility boundary	"What Midnight can verify"	Final proof must be PLONK/KZG over BLS12-381

The key architectural relationship is strict. Folding happens inside the TEE; the decider proof is what goes to Midnight. Midnight does not "trust the TEE." Midnight verifies the decider proof. The decider proof is therefore the bridge between NearFall's private computation and Midnight's public settlement authority.

3.6.3 IVC Pipeline Integration

Sirius is not an isolated library. It is the mechanism inside the TEE that makes the end-to-end pipeline sustainable. Workstream 2 establishes the fold foundation; Workstream 3 uses the decider proof to drive Midnight settlement.

Pipeline phases and persisted artifacts:

Pipeline Phase	Sirius Role	Persisted Artifact
Decrypt and validate	No folding yet; establish admissible instance	Admissible intent record and commitment hash
Fold	Update accumulator with one or more instances	Updated accumulator state and history commitment
Checkpoint	Make accumulator durable across restart	Sealed accumulator snapshot, batch nonce, phase marker
Batch boundary	Stop folding and run decider	Decider proof artifact and state root it attests to

Checkpoint requirements: Accumulator state management must preserve two properties simultaneously: it must be cheap to update at fold time, and it must be durable across restarts. The durability requirement is not optional; if the accumulator cannot survive a TEE restart, the system cannot safely run long enough to amortize anything.

Workstream 2's checkpoint mechanism therefore becomes a Sirius requirement:

- Accumulator serialization must be deterministic and sealed
- Checkpoint includes: accumulator state, fold count, batch ID, last intent hash
- Recovery loads checkpoint, verifies integrity, resumes folding
- Recovery must not create a path to double-sign or double-settle

3.6.4 Proof Outsourcing Service Model

Sirius exists to support NearFall's outsourced proving model without sacrificing confidentiality. The user submits an encrypted witness payload to the attested TEE runtime. The TEE decrypts and validates, then uses Sirius to fold that instance into the accumulator. At the batch boundary, the TEE produces a decider proof that NearFall submits to Midnight. The user never runs a prover locally; that is not a convenience feature, it is the product value for retail devices that cannot reliably produce Midnight proofs.

The falsifiable claim is operational: The only public artifacts should be ciphertexts, commitments, and the decider proof. If plaintext witness data can be recovered from logs, telemetry, or storage, the service fails its confidentiality model even if the math is correct. Workstream 1 establishes the boundary; Sirius must remain inside it.

3.6.5 Performance Requirements

Performance requirements are gate-tracked because they determine feasibility. Fold speed and memory footprint are tested in Workstream 2; decider proof generation and Midnight verification cost are tested in Workstream 3.

Metric	Target	Gate	Test Condition
Fold latency per step	< 100ms	G2	Measured on reference circuit inside target TEE profile
Batch size	50–100 intents	G2/G3	Configurable; must not degrade fold stability or memory safety
Decider proof time per batch	< 60s	G3	Measured for full batch under same TEE profile
Peak memory during fold	< 8GB	G2	Measured with checkpointing enabled and realistic witness sizes
Midnight verification cost	Bounded per batch	G3	Measured on Midnight testnet as "one verification per batch"

Performance comparison (from literature):

Scheme	Reported Fold Time	Accumulator Size
Nova	10-50ms	~1 KB
Protostar	8-30ms	~0.5 KB
ProtoGala xy	10-40ms	~0.6 KB

These targets are falsifiable. If fold latency exceeds the bound materially, NearFall does not get amortization and Workstream 3 cannot pretend otherwise. If decider proof time exceeds the bound, the system cannot produce settlement proofs on a cadence consistent with product requirements.

3.6.6 Curve Compatibility Strategy

Midnight verifies PLONK/KZG proofs on BLS12-381. ProtoGalaxy-style folding frameworks are commonly instantiated on BN254-family curves in reference implementations. This mismatch is not cosmetic: a proof produced under BN254 parameters cannot be verified by a BLS12-381 verifier.

Requirement	Sirius (Current)	Midnight	Status
Pairing curve	BN254	BLS12-381	■ Incompatible
Embedded curve	Grumpkin	JubJub	■ Incompatible
KZG SRS	BN254 powers of tau	BLS12-381 powers of tau	■ Incompatible
Arithmetization	PLONKish	PLONKish	■ Compatible

Why porting is non-trivial: Sirius uses CycleFold for efficient non-native curve operations. CycleFold relies on BN254/Grumpkin forming a curve cycle (each curve's scalar field equals the other's base field). BLS12-381/JubJub do not form a true cycle—JubJub is embedded in BLS12-381's scalar field, but the relationship is asymmetric.

Strategy space:

Strategy	Description	Decision Criteria (Falsifiable)
Native BLS12-381 port	Port Sirius to produce BLS12-381-compatible proofs end-to-end	Fold and decider performance meet gates; audit surface is contained; effort consistent with Workstream 3
Proof composition layer	Keep folding on one curve but produce BLS12-381 decider proof	Composition overhead is bounded and measurable; trust assumptions are explicit; verification compatible with Midnight

The selection is made by measurement, not preference. If composition adds unbounded proof size or verification cost, it fails the core purpose. If a port cannot reach the performance and memory targets, it fails the deployment constraints.

Stakes: The wrong choice here wastes months. If we commit to native port and performance fails, we've spent porting effort on an unusable artifact. If we commit to composition and verification cost is unbounded, batching

provides no economic benefit. G0.5 forces this decision by measurement: benchmark both paths, pick the one that meets targets, document the tradeoff.

3.6.7 Productization Work

Sirius is research-shaped infrastructure that must be made production-shaped. The work is not "polish," it is the delta between a protocol that works on a developer workstation and a mechanism that can survive long-running operation inside a constrained TEE with crash recovery and auditability.

Productization Area	Required Property	Acceptance Condition
Curve compatibility	Proofs verifiable by Midnight's BLS12-381 verifier	End-to-end proof accepted by Midnight testnet
Deterministic accumulator serialization	Stable byte encoding across versions	Identical accumulator state produces identical serialization and hash
Checkpoint and recovery integration	Accumulator survives restart without replay window	Restart tests show no duplicate signing and no state rollback
Memory optimization	Peak memory within TEE constraints	Benchmarks demonstrate < 8GB peak under representative loads
Benchmarking harness	Measured performance is reproducible	Independent runs reproduce fold and decider timings within tolerance

Ownership is workstream-aligned rather than treated as a side track. Sirius deliverables are explicitly tied to G2, G3, and G4 because those gates fail without them. See [§3.1.6](#) for detailed patch list and effort estimates.

3.6.8 Deliverables by Workstream

Workstream	Deliverable	Acceptance Criteria	Gate
Workstream 2	Fold benchmark on reference circuit	Fold < 100ms per step; memory footprint measured; accumulator checkpoint works	G2
Workstream 2	Deterministic accumulator serialization	Serialization stable; recovery yields identical accumulator state	G2
Workstream 3	Decider proof generation	Decider proof produced for a full batch within target time	G3
Workstream 3	Midnight verifier integration	Proof accepted on Midnight testnet and anchors expected root	G3
Workstream 4	Hardening and scale validation	Sustained operation under load; fault injection does not violate invariants	G4

3.6.9 Validation Matrix

Test ID	Category	Test Condition	Pass Condition
S-F1	Functional	Fold one instance into a fresh accumulator	Updated accumulator passes local decision and commits to instance
S-F2	Functional	Fold N instances sequentially	Final decider proof implies all N instances were valid
S-F3	Functional	Fold k instances per step (ProtoGalaxy mode)	Marginal verifier work per instance remains bounded; correctness holds
S-F4	Functional	Checkpoint accumulator, crash, restart, resume folding	Post-restart accumulator equals expected; no duplicate authorization state
S-A1	Adversarial	Attempt to reorder instances without changing final proof	Decider rejects or history commitment mismatch is detected

Test ID	Category	Test Condition	Pass Condition
S-A2	Adversarial	Attempt to provide invalid instance with valid-looking commitments	Decider rejects with the invalid instance included
S-A3	Adversarial	Replay an old accumulator checkpoint to force rollback	Recovery detects rollback attempt via monotonic state invariant and refuses
S-P1	Performance	Measure fold latency on reference circuit in TEE profile	Fold < 100ms per step and within memory bound
S-P2	Performance	Measure decider proof time for a full batch	Decider proof < 60s and verification cost on Midnight is bounded

This validation matrix is the falsifiable contract for Sirius. If these tests do not pass, NearFall does not have an economically viable proving path, and Workstream 3 must not proceed as if it does.

Cross-references:

- Proof outsourcing rationale: [§1.3.1](#)
- Sirius patch details: [§3.1.6](#)
- Workstream 2 IVC foundation: [§3.3.4](#)
- Workstream 3 Midnight integration: [§3.4](#)
- Curve compatibility: [§3.1.6](#), Gate G0.5

Part IV: MPC and Signing Infrastructure

Status: Deferred to implementation phase. This section will specify the MPC architecture, key ceremony protocols, and signing policy enforcement in detail.

What this part will contain when complete:

Section	Content	Status
§4.1	MPC Architecture Overview	Deferred
§4.2	BIP-340 FROST Implementation	Deferred — see §4.2.1 below for selection criteria
§4.3	ECDSA Threshold Signing (NEAR Fork)	Deferred
§4.4	Key Ceremony Protocol	Deferred — see §4.4.1 below for requirements
§4.5	Signing Policy Enforcement	Deferred
§4.6	Key Rotation and Recovery	Deferred

Forward references in this document: References to [§4.2](#) (BIP-340 analysis) and [§4.3](#) (MPC wrapper contract) in other sections refer to content that will appear here. The architectural decisions are established in Part II ([§2.2.5](#)) and Part III ([§3.1.4](#)); Part IV will provide implementation-level specification.

4.2.1 FROST Implementation Selection Criteria

FROST is required for BIP-340 Schnorr threshold signing. No production-scale FROST deployment exists comparable to threshold ECDSA systems. Selection criteria for implementation:

Criterion	Requirement	Rationale
BIP-340 compatibility	Must produce signatures verifiable as BIP-340	Midnight requires BIP-340; incompatible Schnorr variants rejected
Audit status	At least one third-party cryptographic audit completed	Unaudited implementations are not acceptable for custody
License	Permissive open source (MIT, Apache 2.0, or equivalent)	Enables forking and modification without legal encumbrance
Language	Rust preferred; C/C++ acceptable with FFI	Matches TEE enclave toolchain; memory safety preferred
Threshold flexibility	Configurable t-of-n with $n \geq 5$	Supports production quorum requirements
DKG support	Distributed key generation protocol included or compatible	Avoids single-dealer key generation vulnerability

Candidate implementations (evaluation pending):

Implementation	Status	Notes
ZF FROST (Zcash Foundation)	Leading candidate	Rust, audited, BIP-340 compatible fork exists
secp256kfun FROST	Under evaluation	Rust, designed for BIP-340, newer
FROST reference (IETF)	Reference only	Not production-hardened

Selection gate: G1 includes FROST implementation selection with documented rationale. G3 validates 100 consecutive signatures on Midnight testnet.

4.4.1 Key Ceremony Requirements

Key generation for threshold signing requires a ceremony that:

Requirement	Specification
No single dealer	Distributed key generation (DKG) protocol; no party sees the complete private key
Verifiable shares	Each participant can verify their share is consistent with the public key
Threshold enforcement	Ceremony fails if fewer than n participants complete; no "partial key" state
Ceremony transcript	Cryptographic transcript of ceremony for audit; proves no collusion
Air-gapped option	Ceremony can be conducted with air-gapped machines for high-security deployments
Recovery path	Documented procedure for share recovery if participant loses share (requires t participants)

Ceremony phases:

- **Setup:** Participants verify identities; communication channels established
- **Commitment:** Each participant commits to random contribution
- **Share distribution:** Participants exchange encrypted shares
- **Verification:** Each participant verifies received shares against commitments
- **Public key derivation:** Aggregated public key computed and verified by all
- **Transcript finalization:** Ceremony transcript signed by all participants

Operational constraints:

- Ceremony requires synchronous participation (all n participants online)
- Estimated duration: 2-4 hours depending on n and communication latency
- Ceremony failure requires restart from phase 1

Gate: G3 includes successful key ceremony for testnet deployment.

Part V: Security Analysis

Status: Deferred to audit preparation phase. This section will consolidate security analysis, threat modeling, and audit scope.

What this part will contain when complete:

Section	Content	Status
§5.1	Threat Model (Consolidated)	Partial — threat model snapshot in §1.4; full analysis deferred
§5.2	Attack Surface Analysis	Deferred
§5.3	Cryptographic Security Arguments	Deferred
§5.4	Formal Verification Scope	Deferred
§5.5	Audit Preparation Checklist	Partial — audit scope in §3.1.9

Relationship to Part VI: Part V provides the security *analysis*; Part VI provides the security *mechanisms*. Part VI (Risk Management) is complete because the mechanisms must be specified for implementation. Part V (analysis) is deferred because it consolidates findings from implementation and audit.

Forward references: References to §5.1 (security analysis) in other sections refer to content that will appear here. The architectural security model is established in Part II (§2.2.3 Trust Boundaries); Part V will provide formal treatment.

Part VI: Risk Management

How to read Part VI: This part is a threat-informed design document. It names the failure modes that could destroy NearFall, explains why standard mitigations are insufficient, and specifies the mechanisms that address each threat. Read §6.1 for the taxonomy; §6.2–§6.4 for the detailed protocols; §6.5 for residual risk acknowledgment.

Organizing principle: NearFall combines three technologies—TEEs, MPC threshold signing, and ZK proofs—each with distinct failure modes. The risk framework addresses where these technologies interact, because that's where novel vulnerabilities emerge. A TEE rollback that wouldn't matter in isolation becomes catastrophic when it desynchronizes MPC signing state.

6.1 Risk Overview and Framework

Question this section answers: What can go wrong, how badly, and what's the mitigation strategy?

Cross-chain settlement systems concentrate capital and expose large, permissionless attack surfaces. In 2022, Chainalysis reported \$3.1 billion stolen from DeFi protocols, with cross-chain bridges accounting for 64% of that total—bridges are not a niche risk category; they are a primary loss driver. The failures cluster into recognizable patterns: Wormhole (~\$320M) lost funds to a signature verification bypass; Ronin (~\$624M) fell to validator key compromise where 4 of 9 keys were held by one entity; Nomad (~\$190M) was drained after an upgrade bug made the contract accept any message as valid. These weren't exotic attacks—they were predictable failures in well-understood risk categories.

NearFall's architecture inherits the standard bridge risk categories, plus new ones created by combining TEE confidentiality, MPC threshold authorization, and ZK proof-based validity. This section establishes the risk taxonomy, maps each category to NearFall's specific exposure, and references the detailed mitigation protocols in §6.2–§6.4.

6.1.1 Risk Categories

NearFall faces six primary risk categories. Each combines a failure mode with an attack surface specific to the architecture.

Category	Failure Mode	NearFall-Specific Exposure	Impact	Detailed Treatment
Smart Contract Vulnerabilities	Logic bugs in verification or state handling	Settlement contract on Midnight; custody contract on Ethereum	■ Critical — funds drained	§6.3 (Risk Register)
Key Management / MPC Collusion	Threshold compromise or validator collusion	FROST (3-of-5 for Midnight); NEAR fork (3-of-5 for Ethereum)	■ Critical — unauthorized signing	§6.3 (Risk Register)
Finality and Reorganization	Acting on unfinalized state; reorg after release	Midnight finality (~6s); Ethereum finality (~15 min)	■ Critical — double-spend	§6.2 (Atomicity Protocol)
TEE Security	Rollback attacks; enclave forking; attestation bypass	Confidential VM hosts intent plaintext and witnesses	■ Critical — state desync, replay	§6.4 (Anti-Rollback)
Economic / Liquidity	Wrapped asset depeg; liquidity drain; griefing	Custody contract TVL; filler/relayer incentives	■■ High — partial fund loss	§6.3 (Risk Register)
Operational / Governance	Admin key compromise; malicious upgrade; process failure	Contract upgradeability; MPC node administration	■■ High — system compromise	§6.3 (Risk Register)

Stakes: The Ronin hack went undetected for 6 days; Nomad was drained within hours of the first exploit by dozens of independent copycats. Speed of detection is itself a risk factor—custody pools don't survive slow response to Critical failures.

Scope constraint: This taxonomy is derived from historical bridge failures and known TEE/MPC attack literature. Attack categories not yet demonstrated in production may exist; [§6.5](#) addresses residual risks including unknown-unknowns.

6.1.2 Historical Precedent

Four bridge incidents directly inform NearFall's risk model:

Wormhole (February 2022, ~\$320M) A deprecated Solana contract function allowed an attacker to bypass guardian signature verification and mint 120,000 wETH without backing. The root cause was insufficient input validation—the contract accepted a spoofed system account as a legitimate signer.

NearFall relevance: The settlement contract's proof acceptance logic and the custody contract's release conditions are analogous attack surfaces. Mitigation: formal verification of acceptance rules; multiple independent audits ([§3.5.7](#)).

Ronin (March 2022, ~\$624M) Attackers compromised 5 of 9 validator keys in a multisig where 4 keys were already controlled by one entity (Sky Mavis). A dormant Axie DAO permission provided the 5th signature. The breach went undetected for 6 days.

NearFall relevance: MPC threshold security depends on key distribution. If 3 of 5 MPC nodes are operated by the same entity or share infrastructure, the threshold provides false assurance. Mitigation: geographic and administrative diversity requirements; real-time signing anomaly detection (§3.5.2).

Nomad (August 2022, ~\$190M) An upgrade bug set the trusted root to `0x00`, making the contract accept any message as valid. Once the first attacker demonstrated the exploit, dozens of copycats drained the remaining funds within hours.

NearFall relevance: Contract upgrades are a critical risk surface. A single faulty deployment can compromise the entire system. Mitigation: timelock on upgrades; multi-signature governance; upgrade simulation on testnet before mainnet (§6.3).

Poly Network (August 2021, ~\$610M; July 2023, ~\$42B notional / ~\$10M realized) In 2021, an attacker exploited cross-chain message handlers to become temporary contract owner and approve fraudulent transfers (~\$610M moved, later returned). In July 2023, an attacker was able to *issue* assets on the order of \$42B notional, but realized substantially less (~\$10M) due to liquidity limits and token freezes—the "notional" magnitude still matters because it tested operational controls (pauses, freezes) and governance response speed.

NearFall relevance: Cross-chain message authentication is a fundamental attack surface. NearFall's TEE-attested signing requests must be unforgeable—an attacker who can construct a valid-looking signing request without TEE involvement breaks the security model. Mitigation: attestation binding with measurement verification; MPC policy enforcement (§6.4).

6.1.3 NearFall-Specific Risk Interactions

Standard risk categories don't capture the novel interactions in NearFall's architecture. Three interaction patterns require dedicated protocols.

TEE concepts for risk analysis: A Trusted Execution Environment (TEE) is hardware-isolated memory where code runs such that even the machine's operator cannot inspect its contents. The operator can run the TEE but cannot read what's inside—like a locked box that computes. *Attestation* is the mechanism by which external parties verify the TEE is running expected code: the hardware produces a cryptographic quote containing the *measurement* (hash of the loaded code), which a verifier checks against an allowlist. For deeper background, see §1.3.1 (TEE solution) and §2.2.5 (attestation binding).

Terminology (used throughout §6.2–§6.4):

Term	Definition
Batch	A set of matched intents processed atomically; the unit of settlement

Term	Definition
batch_nonce	Monotonically increasing identifier for each batch; used for replay prevention
Signing request	A message from TEE to MPC requesting threshold signature over a payload
Authorization policy	Rules enforced by MPC independent of TEE: nonce ordering, payload constraints, attestation validity
Rollback attack	Attacker restores TEE to earlier state (e.g., VM snapshot), causing it to "forget" recent operations
Enclave forking	Attacker runs multiple TEE instances from the same state, causing parallel execution that breaks assumptions of single-threaded operation
Measurement	Cryptographic hash of code loaded into TEE; used to verify expected software is running
Attestation quote	Hardware-signed document containing measurement and other TEE state; proves TEE integrity to external verifiers

Interaction 1: TEE Rollback → MPC Desync → Double-Signing

A malicious cloud host snapshots the VM at time T, allows the enclave to process batches through T+N, then restores the T snapshot. The enclave loses all memory of batches processed after T—it believes batch N hasn't happened yet. The enclave requests MPC signature for batch N (again). If the MPC doesn't track processed batches independently, it signs the same batch twice—enabling double-spend.

Why standard TEE mitigations are insufficient: Hardware monotonic counters (SGX) are slow (~60ms per update) and limited in quantity. AMD SEV-SNP protects runtime memory but not persisted state—an attacker can restore an entire VM snapshot. The enclave cannot detect rollback alone.

NearFall mitigation: Monotonic `batch_nonce` enforced at MPC layer. The MPC rejects any signing request with $\text{nonce} \leq \text{last-seen nonce}$, regardless of attestation validity. Detailed protocol: [§6.4](#).

Interaction 2: Finality Race → Atomicity Violation → Fund Loss

Ethereum custody is released based on Midnight settlement inclusion. Before Midnight finalizes, a reorg replaces the settlement with a different batch. The Ethereum release is now orphaned—funds released for a settlement that no longer exists.

Why standard finality handling is insufficient: Waiting for Midnight finality before Ethereum release is necessary but not sufficient. If Ethereum release fails after Midnight finality, funds are locked on Midnight with no corresponding release—violating atomicity in the opposite direction.

NearFall mitigation: Two-phase commit with timeout-bounded abort. Ethereum custody is locked (not released) until Midnight finality is confirmed. If release fails, compensating refund executes within bounded window. Detailed protocol: [§6.2](#).

Interaction 3: Attestation Bypass → Unauthorized Signing → Theft

An attacker constructs a signing request that passes MPC policy checks without originating from a legitimate TEE. If attestation verification is bypassable (as in Wormhole's guardian check), the attacker can sign arbitrary transactions.

Why identity-only proofs are insufficient: A request provenance proof must bind MPC authorization to (a) a hardware-verifiable quote and (b) an expected measurement/configuration. Identity tokens alone (e.g., OIDC credentials) reduce the policy to "someone with the right identity asked"—which is not a security boundary if the identity can be compromised or the token replayed.

NearFall mitigation: Measurement-bound attestation. MPC accepts signing requests only when the attestation quote includes the expected enclave measurement hash and freshness properties. Identity-only tokens are insufficient; hardware attestation with measurement binding is required. Detailed requirements: [§3.2.3](#), enforcement: [§6.4](#).

6.1.4 Risk Assessment Framework

NearFall uses a severity × likelihood matrix for risk prioritization. Severity reflects maximum fund exposure; likelihood reflects attack complexity and prerequisite conditions.

Severity Levels:

Level	Definition	Example
■ Critical	Total custody at risk; system-wide compromise possible	Settlement contract accepts invalid proofs
■■ High	Significant fund loss possible; bounded by rate limits or caps	Single MPC node key compromise
■ Medium	Limited fund exposure; operational disruption	TEE availability loss requiring failover
■ Low	No direct fund risk; reputational or efficiency impact	Latency SLO violation

Likelihood Levels:

Level	Definition	Characteristics
High	Actively exploited in similar systems; low attacker cost	Known vulnerability pattern; script-kiddie accessible
Medium	Demonstrated in research; requires moderate resources	Novel but documented attack; skilled attacker required
Low	Theoretical or requires extraordinary resources	Nation-state level; hardware supply chain compromise
Very Low	No known path; requires multiple independent failures	Cryptographic break of multiple primitives

Priority Matrix:

	High Likelihood	Medium Likelihood	Low Likelihood	Very Low Likelihood
■ Critical	P1 — Immediate	P1 — Immediate	P2 — Pre-launch	P3 — Tracked
■■ High	P1 — Immediate	P2 — Pre-launch	P3 — Tracked	Accept
■ Medium	P2 — Pre-launch	P3 — Tracked	Accept	Accept
■ Low	P3 — Tracked	Accept	Accept	Accept

P1 items block launch. P2 items must be mitigated before mainnet. P3 items are documented with monitoring.

6.1.5 Defense-in-Depth Principle

Design goal: No single component compromise should be sufficient for fund theft.

To make this claim auditable, NearFall requires independent enforcement at multiple layers:

Layer	Enforcement Requirement	Fails If...
On-chain correctness	Settlement/custody contracts reject invalid proofs and unauthorized releases independent of off-chain state	Contract accepts malformed proof or release without valid finality evidence
MPC authorization policy	MPC enforces payload constraints and monotonic <code>batch_nonce</code> independent of TEE state (§6.4)	MPC signs based solely on TEE request without independent anti-replay
Confidentiality boundary	TEE protects plaintext intent/witness from operators, reducing targeted theft under partial compromise	Plaintext leaks to non-TEE environment
Operational containment	Upgrade timelocks, circuit breakers, and detection SLOs prevent slow-bleed failures from becoming total-loss	No pause capability; detection latency exceeds drain rate

This framework does **not** assume TEEs are invulnerable. It assumes TEEs can fail (rollback, side-channel, attestation bypass) and requires the MPC and on-chain layers to remain theft-resistant under that failure. A compromised TEE can *request* signatures—the MPC must independently constrain *what* can be signed and *when*.

The Risk Register (§6.3) documents each identified risk with severity, likelihood, mitigations, residual risk, owner, and validation gate.

6.1.6 Part VI Section Map

Section	Content	Status
§6.1	Risk Overview and Framework	■ This section
§6.2	Cross-Chain Atomicity Protocol	Detailed protocol for finality/reorg risk
§6.3	Risk Register	Enumerated risks with mitigations
§6.4	Anti-Rollback Mechanism	Detailed protocol for TEE rollback risk
§6.5	Residual Risk Acknowledgment	Risks accepted after mitigation

6.1.7 What This Section Establishes

Claim	Status
Six risk categories cover NearFall's attack surface	■ Established
Historical bridge failures inform the risk model	■ Established (4 incidents analyzed)
Three novel interaction patterns require dedicated protocols	■ Established
Severity × likelihood matrix provides prioritization	■ Established
Defense-in-depth is enforced by independent layer requirements	■ Design goal with auditable criteria
Taxonomy scope acknowledged (unknown-unknowns possible)	■ Stated

What comes next: §6.2 specifies the Cross-Chain Atomicity Protocol—the mechanism that prevents finality races and ensures atomic-or-compensating settlement across Midnight and Ethereum.

6.2 Cross-Chain Atomicity Protocol

Question this section answers: How does NearFall ensure that cross-chain settlements either complete on both chains or revert on both chains—despite the chains having no native two-phase commit?

True simultaneity across independent blockchains is impossible. Midnight and Ethereum have different finality mechanisms, different block times, and no shared transaction coordinator. Yet NearFall must guarantee atomicity: Alice's ETH and Bob's NIGHT must either both transfer, or neither transfers. A system that can leave Alice with no ETH *and* no NIGHT—or Bob with both—is not a bridge; it's a theft vector.

The 60-second answer: When does Bob's NIGHT become irreversibly Alice's, and what evidence is required?

> NIGHT is **reserved** when the lock transaction is included on Midnight (Phase 1). > NIGHT is **pending** when the batch proof is verified and recorded (Phase 2). > NIGHT becomes **final** only at **commit** (Phase 4), which requires an **Ethereum release proof** showing Alice's ETH was released to Bob. > Ethereum release (Phase 3) requires a **Midnight finality proof** and **batch membership proof** showing the settlement was included in the finalized batch.

The circular dependency is intentional: neither side commits until it has proof the other side committed. Timeouts break deadlocks—if any step fails to complete within bounds, both sides can reclaim their original assets.

This section specifies the protocol that achieves atomicity through **escrow on both chains**, **conditional release**, and **timeout-bounded abort**. The design draws from production systems (UniswapX, Across, IBC) but adapts them to NearFall's TEE + MPC + ZK architecture.

6.2.1 The Atomicity Problem

Concrete scenario: Alice wants to swap 1 ETH for 100 NIGHT. Bob wants the inverse. NearFall matches them.

Without atomicity guarantees, the following failure modes are possible:

Failure Mode	What Happens	Who Loses
Orphaned release	Ethereum releases Alice's ETH to Bob; Midnight reorgs and the settlement disappears	Alice loses ETH, gets no NIGHT
Stranded funds	Midnight settles; Ethereum release transaction fails or times out	Alice has NIGHT on Midnight but ETH is locked forever on Ethereum
Double-pay	Midnight settles (Alice gets NIGHT); Ethereum release fails; Alice refunds ETH	Alice gets both NIGHT + ETH; Bob loses NIGHT, gets no ETH
Double-spended	Attacker triggers release on Ethereum, then causes Midnight reorg to a different settlement	Protocol loses funds; one party paid twice
Griefing	Attacker repeatedly locks funds and forces timeouts without completing	Honest users' capital is tied up; system unusable

Stakes: Nomad's exploit was a direct atomicity failure—releasing funds without valid settlement proof. The other incidents in [§6.1.2](#) (key compromise, signature bypass) created conditions where atomicity checks could be circumvented. Atomicity isn't a feature—it's the difference between a bridge and a honeypot.

The key insight: Atomicity must protect *both* parties, not just the depositor. A protocol that guarantees "Alice gets swap OR refund" but allows "Bob loses funds" is not atomic—it just shifts the victim.

6.2.2 Protocol Overview

NearFall implements atomicity through **escrow on both chains** with **conditional commit**. Neither chain finalizes transfers until evidence arrives that the other chain has progressed.

Mental model (two safes with timed locks):

- **Safe E (Ethereum):** Holds Alice's ETH. Opens to Bob only if a valid "commit ticket" (Midnight finality proof) arrives. Otherwise, timer expires and Alice reclaims it.
- **Safe M (Midnight):** Holds Bob's NIGHT in pending state. Commits to Alice only if proof of ETH release arrives. Otherwise, timer expires and Bob reclaims it.

Atomicity is achieved because **neither safe opens permanently without evidence that the other side has progressed**.



The four phases:

Phase	Location	Action	Success Condition	Failure Response
1. Lock	Both chains	Alice escrows ETH; Bob's NIGHT reserved	Both locks confirmed	Either party cancels before Phase 2
2. Settle (Pending)	Midnight	TEE generates proof; Midnight records pending settlement	Pending record created; finality reached	Batch aborted; both sides unlock
3. Release	Ethereum	Custody releases ETH to Bob	Release transaction confirmed	Retry; eventual timeout → both sides refund
4. Commit	Midnight	Pending settlement commits; Alice receives NIGHT	Commit transaction confirmed	Retry; eventual timeout → Bob reclaims NIGHT

Critical ordering:

- Phase 3 (Ethereum release) requires Phase 2 finality proof
- Phase 4 (Midnight commit) requires Phase 3 release proof
- This creates a **commit chain**: neither side finalizes without evidence the other progressed

6.2.3 Phase 1: Lock (Both Chains)

Both parties escrow their assets before settlement begins.

Ethereum side (Alice's ETH):

Illustrative contract logic (non-normative):

```
BEFORE: custody_contract.locks[alice] = ∅

ACTION: alice.deposit(1 ETH, intent_hash, deadline)

AFTER:  custody_contract.locks[alice] = {
    amount: 1 ETH,
    intent_hash: 0xabc...,
    deadline: block.timestamp + LOCK_DURATION,
    status: LOCKED
}
```

Constraint: Lock must bind deposited amount to specific intent; lock must have bounded expiration; lock status must prevent double-spend.

Midnight side (Bob's NIGHT):

Bob's NIGHT is reserved when his intent is accepted into a batch. The settlement contract marks the balance as encumbered.

Illustrative state transition (non-normative):

```
BEFORE: settlement_contract.balances[bob] = 100 NIGHT (available)

ACTION: TEE includes bob's intent in batch #42

AFTER: settlement_contract.balances[bob] = 100 NIGHT (reserved for batch #42)
       settlement_contract.reservations[batch_42] = {
         participants: [bob],
         amounts: [100 NIGHT],
         expires_at: midnight_block + RESERVATION_BLOCKS
       }
```

Constraint: Reservation must encumber balance without transferring ownership; reservation must expire if settlement does not complete; reserved funds must not be spendable in other transactions.

Lock parameters:

Parameter	Constraint	Rationale
LOCK_DURATION	$\geq 2 \times$ expected settlement time; configurable	Must accommodate proof generation + finality + release
intent_hash	Binds lock to specific intent	Prevents substitution attacks
deadline	Absolute timestamp (Ethereum) or block height (Midnight)	Chain-native time representation; on-chain enforceable

Note: Specific timeout values are deployment configuration. The spec constrains relationships (e.g., Ethereum timeout > Midnight pending timeout) rather than absolute values.

Lock invariants (both chains):

- **Binding:** Lock is bound to specific intent/batch; cannot be released for different settlement
- **Non-transferable:** Only the protocol can move escrowed funds
- **User-refundable:** After timeout, user can reclaim without protocol cooperation

6.2.4 Phase 2: Settle (Pending)

The TEE matches intents, generates a batch proof, and submits to Midnight. Midnight creates a **pending settlement record**—it does NOT finalize transfers yet.

Settlement flow:

- TEE decrypts Alice's intent (sell 1 ETH, buy ≥ 95 NIGHT)
- TEE decrypts Bob's intent (sell 100 NIGHT, buy ≥ 0.95 ETH)
- TEE verifies match: Alice gets 100 NIGHT, Bob gets 1 ETH (both satisfy minimums)
- TEE computes `batch_commitment` binding all settlement details
- TEE generates `proof_artifact` proving match correctness
- MPC signs Midnight transaction (BIP-340 Schnorr)
- TEE broadcasts to Midnight

Midnight creates PendingSettlement (not final transfers):

Illustrative record structure (non-normative):

```
pending_settlements[batch_nonce] = {
  batch_commitment: H(batch_nonce, state_root_prev, state_root_next, fills_merkle_root),
  fills: [
    { from: bob, to: alice, asset: NIGHT, amount: 100 },
    // Note: ETH transfer is on Ethereum, referenced by lock_id
  ],
  eth_locks: [alice_lock_id], // Ethereum locks that must release for commit
  status: PENDING,
  created_at: current_block,
  expires_at: current_block + PENDING_TIMEOUT_BLOCKS
}
```

Constraints:

- Pending settlement must record batch commitment binding all settlement details
- Pending settlement must reference corresponding Ethereum locks
- Pending settlement must have bounded expiration
- Status must distinguish pending from committed/expired

Batch commitment structure:

Illustrative commitment (non-normative):

```

batch_commitment = H(
  batch_nonce,
  state_root_prev,
  state_root_next,
  merkle_root(
    for each fill: H(lock_id, intent_hash, amount, recipient_eth, recipient_midnight)
  )
)

```

Constraint: Commitment must cryptographically bind batch_nonce, state transition, and all fills such that Ethereum can verify a specific lock was included in a specific finalized batch.

Pending settlement invariants:

- **No premature transfer:** Balances are reserved, not transferred; Bob still owns his NIGHT (encumbered)
- **Commitment binding:** `batch_commitment` cryptographically binds all settlement details
- **Expiration:** Pending settlements expire if not committed within timeout
- **Proof validity:** Midnight only accepts batches with valid ZK proofs

Failure handling:

Failure	Detection	Response
Proof generation fails	Proof circuit returns error or timeout exceeds 60s	Batch aborted; reservations released
Midnight rejects proof	On-chain transaction reverts with verification error	Batch aborted; investigate root cause
MPC refuses to sign	Signing request returns policy violation code	Re-attest or abort batch

6.2.5 Phase 3: Release (Ethereum)

Once the pending settlement reaches Midnight finality, Ethereum can release Alice's ETH to Bob.

Why finality matters (unsafe vs. safe timeline):

Timeline (UNSAFE: release on inclusion)

```

████████████████████████████████████████████████████████████████████████████████
t=0:  Alice's ETH locked on Ethereum
t=5:  Pending settlement included in Midnight block 1000
t=6:  NearFall releases ETH to Bob          ← TOO EARLY
t=8:  Midnight reorg; block 1000 replaced by 1000'
t=9:  Pending settlement gone; Bob has ETH but Alice has no claim to NIGHT
      → ATOMICITY VIOLATED

```

Timeline (SAFE: release on finality)

```

████████████████████████████████████████████████████████████████████████████████
t=0:  Alice's ETH locked on Ethereum
t=5:  Pending settlement included in Midnight block 1000
t=6:  NearFall waits...
t=11: Block 1000 finalized (GRANDPA)
t=12: NearFall releases ETH to Bob with finality proof ← SAFE
t=15: Release confirmed; proceed to Phase 4
      → ATOMICITY PRESERVED (so far)

```

Release flow:

- TEE obtains finality proof for Midnight block containing pending settlement
- TEE constructs release transaction with Merkle proof for Alice's lock
- MPC signs Ethereum transaction (ECDSA secp256k1)
- TEE broadcasts to Ethereum
- Custody contract verifies and releases

Custody contract release rule:

Illustrative contract logic (non-normative):

```

release(lock_id, recipient, amount, batch_nonce, batch_commitment, merkle_proof, finality_pr...
requires:
    // Lock validity
    locks[lock_id].status == LOCKED
    AND locks[lock_id].amount == amount

    // Replay protection (per-batch, not global)
    AND NOT released_batches[batch_nonce]

    // Cross-chain verification
    AND verify_finality_proof(finality_proof, batch_commitment)
    AND verify_merkle_proof(merkle_proof, batch_commitment, lock_id)

EFFECT:
    transfer(amount, recipient)
    locks[lock_id].status = RELEASED
    released_batches[batch_nonce] = true
    emit ReleaseExecuted(lock_id, batch_nonce, recipient, amount)

```

Release constraints:

- Release must verify lock exists and is in LOCKED state
- Release must verify amount matches locked amount exactly
- Release must verify batch has not already been released (per-batch replay protection)
- Release must verify finality proof binds to batch commitment
- Release must verify lock was included in the committed batch (Merkle proof)
- Release must atomically update lock status and batch tracking

Cross-chain verification modes:

The custody contract must verify that a pending settlement actually reached finality on Midnight. Three modes with different trust assumptions:

Mode	Mechanism	Trust Assumption	Status	TVL Appropriate
A: On-chain light client	BEEFY proofs verified by Ethereum contract	Cryptographic (Midnight validator honesty >2/3)	■■ Requires Midnight BEEFY	Unlimited
B: Optimistic oracle	Bonded relayers post commitments; dispute window	Economic (bond > profit from fraud)	■■ Design pending	Up to bond amount
C: TEE attestation (bootstrap)	TEE observes finality, signs claim	TEE integrity + MPC honesty	■ Available	Capped (see §2.4.6)

Note: Midnight uses Substrate's GRANDPA finality and BEEFY light-client protocol (see [§2.2.1](#)). Mode A is the target end state. Mode C is explicitly trusted and requires conservative TVL caps and rapid circuit breakers.

Bootstrap verification (Mode C):

Illustrative verification logic (non-normative):

```

verify_finality_proof_bootstrap(attestation, batch_commitment) :=
  // Verify TEE signature
  recover_signer(attestation.signature) == known_tee_pubkey
  AND attestation.measurement == approved_measurement

  // Verify content
  AND attestation.batch_commitment == batch_commitment
  AND attestation.finality_block >= required_finality_depth
  AND attestation.timestamp > block.timestamp - MAX_ATTESTATION_AGE

```

Bootstrap verification constraints:

- Attestation must be signed by known TEE key
- Attestation must include approved enclave measurement
- Attestation must bind to the batch commitment being released
- Attestation must reference sufficient finality depth
- Attestation must be fresh (bounded age)

Release invariants:

- **Finality-gated:** Release requires valid finality proof (mode-appropriate)
- **Amount-exact:** Released amount must match locked amount
- **Single-release per lock:** `locks[lock_id].status` prevents double-spend
- **Per-batch replay protection:** `released_batches[batch_nonce]` prevents batch replay without global coupling

Failure handling:

Failure	Detection	Response
Release transaction reverts	Transaction receipt shows revert	Retry with gas adjustment
Finality proof rejected	Contract emits <code>ProofVerificationFailed</code>	Investigate; do not retry same proof
Ethereum congestion	Transaction pending beyond configured threshold	Retry with higher gas; escalate if persistent
Release timeout	No release within <code>LOCK_DURATION</code>	Alice can refund; Midnight pending expires

6.2.6 Phase 4: Commit (Midnight)

The pending settlement on Midnight only commits (finalizing Alice's NIGHT receipt) after proof that Ethereum released Bob's ETH.

Why this phase exists: Without it, the protocol has the "double-pay" vulnerability: Midnight settles, Ethereum release fails, Alice refunds ETH → Alice has both NIGHT and ETH.

Commit flow:

- TEE observes Ethereum release transaction confirmed
- TEE constructs release proof (Ethereum tx inclusion + block header)
- TEE submits commit transaction to Midnight
- Midnight verifies release proof and finalizes transfers

Midnight commit rule:

Illustrative contract logic (non-normative):

```
commit_settlement(batch_nonce, eth_release_proof) requires:
  pending_settlements[batch_nonce].status == PENDING
  AND verify_eth_release_proof(eth_release_proof, pending_settlements[batch_nonce].eth_locks)
  AND current_block < pending_settlements[batch_nonce].expires_at

EFFECT:
  FOR each fill IN pending_settlements[batch_nonce].fills:
    transfer(fill.amount, fill.from, fill.to)
  pending_settlements[batch_nonce].status = COMMITTED
  emit SettlementCommitted(batch_nonce)
```

Commit constraints:

- Commit requires pending settlement in PENDING state
- Commit requires valid proof that corresponding Ethereum locks were released
- Commit must occur before pending settlement expires
- Commit must atomically execute all fills in the batch

Ethereum release proof verification:

Illustrative verification (non-normative):

```
verify_eth_release_proof(proof, expected_locks) :=
  // Verify Ethereum block header (via light client or oracle)
  verify_eth_block_header(proof.block_header)

  // Verify release transaction inclusion
  AND verify_tx_inclusion(proof.tx_receipt, proof.merkle_proof, proof.block_header)

  // Verify release matches expected locks
  AND proof.released_locks == expected_locks
  AND proof.release_event.batch_nonce == batch_nonce
```

Release proof constraints:

- Proof must demonstrate Ethereum block validity
- Proof must demonstrate release transaction inclusion in that block
- Proof must demonstrate the released locks match the expected locks for this batch

Commit invariants:

- **Release-gated:** Commit requires proof that Ethereum released
- **Timeout-bounded:** Pending settlements expire; Bob can reclaim if commit never happens
- **Single-commit:** Each pending settlement commits exactly once

6.2.7 Abort and Refund Protocol

When any phase fails, both chains must be able to revert to safe state **independently**. This is critical: refunds must not require cross-chain coordination.

Abort triggers and responses:

Trigger	Phase	Ethereum Response	Midnight Response
User cancellation	1	Alice calls <code>cancel()</code> if intent not batched	Bob's reservation released
Settlement failure	2	No action needed (ETH still locked)	Reservation released; pending not created
Release timeout	3	Alice calls <code>refund()</code> after deadline	Pending settlement expires → Bob reclaims
Commit timeout	4	Already released (Bob has ETH)	Pending expires → Bob reclaims NIGHT

Ethereum refund mechanism:

Illustrative contract logic (non-normative):

```
refund(lock_id) requires:
  locks[lock_id].status == LOCKED
  AND block.timestamp > locks[lock_id].deadline

EFFECT:
  transfer(locks[lock_id].amount, locks[lock_id].depositor)
  locks[lock_id].status = REFUNDED
  emit Refunded(lock_id)
```

Refund constraints:

- Refund requires lock in LOCKED state (not already released or refunded)
- Refund requires deadline to have passed

- Refund must not require protocol cooperation (user-callable)
- Refund must atomically update lock status

Midnight pending expiration:

Illustrative contract logic (non-normative):

```
expire_pending(batch_nonce) requires:
    pending_settlements[batch_nonce].status == PENDING
    AND current_block > pending_settlements[batch_nonce].expires_at

EFFECT:
    FOR each reservation IN pending_settlements[batch_nonce].reservations:
        release_reservation(reservation) // Bob's NIGHT becomes available again
    pending_settlements[batch_nonce].status = EXPIRED
    emit PendingExpired(batch_nonce)
```

Expiration constraints:

- Expiration requires pending settlement past its deadline
- Expiration must release all reservations (return encumbered funds to owners)
- Expiration must be callable without protocol cooperation

Timeout constraints:

Parameter	Constraint	Rationale
LOCK_DURATION	$\geq 2 \times$ expected settlement time	Must accommodate worst-case latency
RESERVATION_BLOCKS	$< \text{LOCK_DURATION}$ equivalent	Midnight reservation should expire before Ethereum lock
PENDING_TIMEOUT_BLOCKS	$> \text{RESERVATION_BLOCKS}$	Pending must outlast reservation to allow commit
RELEASE_RETRY_LIMIT	Bounded; configurable	Prevents infinite retry loops

Note: Specific timeout values are deployment configuration, not architectural constraints. The spec requires relationships between timeouts (e.g., `PENDING_TIMEOUT > RESERVATION_BLOCKS`) but does not mandate specific durations.

Refund invariants:

- **Independent refunds:** Each chain's refund works without the other chain's cooperation

- **Bounded wait:** Maximum lock time is `LOCK_DURATION` (Ethereum) or `PENDING_TIMEOUT_BLOCKS` (Midnight)
- **No double-claim:** Refund impossible if release/commit already succeeded
- **User-callable:** Refunds don't require NearFall liveness after timeout

6.2.8 Failure Mode Analysis

Every failure mode has a defined recovery path. The protocol protects **both** parties.

Failure Mode	Alice (ETH depositor)	Bob (NIGHT provider)	Protocol State
Happy path	Gets 100 NIGHT	Gets 1 ETH	Both committed
TEE crash before Phase 2	Refunds ETH after timeout	Reservation released	Both refunded
Midnight rejects proof	Refunds ETH after timeout	Reservation released	Both refunded
Midnight reorg pre-finality	Refunds ETH after timeout	Reservation released	Both refunded
Ethereum release fails	Refunds ETH after timeout	Pending expires; reclaims NIGHT	Both refunded
Commit proof delayed	Has no claim yet	Pending may expire; reclaims NIGHT	Release happened but commit didn't → Bob has ETH, reclaims NIGHT (protocol loss, see below)
All systems down	Refunds after <code>LOCK_DURATION</code>	Reclaims after <code>PENDING_TIMEOUT</code>	Both refunded

Edge case: Release succeeds but commit fails

If Ethereum releases (Bob gets ETH) but the commit proof never arrives on Midnight (pending expires, Bob reclaims NIGHT), then Bob ends up with both assets. This is a **protocol loss**, not a user loss—Bob is made more than whole at NearFall's expense.

Mitigations:

- `PENDING_TIMEOUT` >> expected commit time (provides large buffer)
- Commit retries with escalation before timeout

- Circuit breaker: pause new settlements if commit backlog grows
- Economic: this scenario requires NearFall operational failure, not user action

6.2.9 Validation Gates

The following tests validate atomicity guarantees:

Test	Scenario	Expected Result
Reorg test	Reorg Midnight pre-finality	Ethereum release blocked; no finality proof available
Release-failure test	Force Ethereum release failure	Midnight pending expires; Bob reclaims NIGHT; Alice refunds ETH
Nonce replay test	Replay identical release call	Fails via <code>released_batches[batch_nonce]</code>
Lock replay test	Attempt second release for same lock	Fails via <code>locks[lock_id].status == RELEASED</code>
Partial outage test	MPC unavailable beyond lock deadline	User-callable refunds work after deadline
Commit timeout test	Commit proof never submitted	Pending expires; Bob reclaims; Alice refunds
Cross-chain race test	Submit release and refund simultaneously	Exactly one succeeds based on timing

6.2.10 Protocol Parameters

Parameter constraints (not specific values):

Parameter	Constraint	Governance
LOCK_DURATION	$\geq 2 \times$ expected settlement time	Configurable; changes require timelock
RESERVATION_BLOCKS	$<$ Ethereum lock equivalent	Configurable
PENDING_TIMEOUT_BLOCKS	$>$ RESERVATION_BLOCKS	Configurable; changes require timelock
RELEASE_RETRY_LIMIT	Bounded positive integer	Operational config
MAX_ATTESTATION_AGE	Bounded; $<$ expected finality time	Configurable

Required relationships:

- $PENDING_TIMEOUT > RESERVATION$ (pending must outlast reservation)
- $LOCK_DURATION > expected_settlement_time \times 2$ (buffer for worst-case)
- $MAX_ATTESTATION_AGE < expected_finality_propagation$ (freshness)

Governance constraints:

- Critical timeout changes require bounded timelock (defined at deployment)
- Parameter changes must emit events for off-chain monitoring
- Parameter ranges must be validated on-chain (e.g., cannot set $PENDING_TIMEOUT < RESERVATION$)

6.2.11 What This Section Establishes

Claim	Status
Four-phase commit-abort protocol with escrow on both chains	■ Established
Midnight-first settlement prevents orphaned releases	■ Established
Pending/commit separation prevents double-pay vulnerability	■ Established

Claim	Status
Finality-gated release prevents reorg-based attacks	■ Established
Per-batch replay protection avoids liveness coupling	■ Established
Independent refunds on each chain (no cross-chain dependency)	■ Established
Cross-chain verification modes explicit (bootstrap vs. target)	■ Established
Validation gates defined	■ Established

Design invariant (under stated assumptions): Assuming contract correctness, TEE integrity (for bootstrap mode), and MPC honest-majority thresholds are not violated, every lock either commits to a completed swap or can be refunded by the owner after bounded timeouts.

What comes next: §6.3 specifies the Risk Register—the enumerated list of identified risks with severity, likelihood, mitigations, and owners. §6.4 details the Anti-Rollback Mechanism that prevents TEE state manipulation from breaking the atomicity guarantees established here.

6.3 Risk Register

Question this section answers: What specific risks threaten NearFall, how severe are they, and what evidence demonstrates that mitigations are effective?

A risk register is not a worry list. It is an enumerated set of failure modes with three properties: (1) each risk has a condition that makes it **realized**—something observable went wrong; (2) each risk has a **mitigation** with validation criteria—how we know the mitigation works; (3) each risk has **severity** and **likelihood** that determine prioritization. A risk without these properties is speculation, not risk management.

This section enumerates risks across the six categories established in §6.1.1, assigns severity/likelihood ratings, specifies mitigations, and defines the validation criteria that demonstrate mitigation effectiveness. Risks addressed by dedicated protocol sections (§6.2 Atomicity, §6.4 Anti-Rollback) are referenced but not duplicated.

6.3.1 Risk Register Structure

Each risk entry follows this format:

Field	Content
ID	Unique identifier (e.g., SC-1 for Smart Contract risk 1)
Risk	What can go wrong
Realized When	Observable condition indicating the risk has materialized
Severity	Impact if realized (Critical / High / Medium / Low)
Likelihood	Probability of occurrence given current design (High / Medium / Low)
Mitigation	What reduces severity or likelihood
Validated By	Evidence that mitigation is effective
Owner	Functional role responsible (not named individual)
Status	Open / Mitigated / Accepted

Severity definitions (from §6.1.4):

Level	Definition	Example
■ Critical	Total fund loss possible; system compromise	Custody drained; unauthorized signing
■ High	Significant fund exposure; major operational disruption	Partial custody loss; extended downtime
■ Medium	Limited fund exposure; recoverable disruption	Single-user loss; temporary degradation
■ Low	Minimal impact; inconvenience	Delayed settlement; minor UX issue

Likelihood definitions:

Level	Definition	Basis
High	Similar attacks have succeeded against comparable systems	Historical precedent exists
Medium	Attack is theoretically feasible; no direct precedent	Requires novel combination or insider
Low	Attack requires capabilities beyond known threat actors	Nation-state or breakthrough required

6.3.2 Smart Contract Risks

Smart contract vulnerabilities are the most common bridge failure mode. Wormhole and Nomad were both contract bugs, not cryptographic breaks.

SC-1: Settlement Contract Proof Acceptance Bypass

Field	Content
Risk	Settlement contract accepts invalid or forged proofs, allowing unauthorized state transitions
Realized When	State root updated without corresponding valid batch proof; funds transferred without matching proven intents
Severity	■ Critical
Likelihood	Medium — Nomad accepted any message root due to initialization bug; similar logic errors are plausible
Mitigation	(1) Formal verification of proof acceptance logic; (2) Multiple independent audits with overlapping scope; (3) Testnet deployment with adversarial fuzzing before mainnet

Field	Content
Validated By	(1) Formal verification report covering acceptance predicate; (2) Property-based adversarial suite: defined test case count, mutation count, statement/branch coverage targets for acceptance predicate, differential tests against reference verifier, known-bad proof corpus with zero false accepts; (3) Audit findings resolved
Owner	Smart contract engineering
Status	Open — blocks mainnet

SC-2: Custody Contract Release Without Valid Cross-Chain Proof

Field	Content
Risk	Ethereum custody releases funds without a verifiable proof that the corresponding Midnight settlement is finalized and matches the lock
Realized When	<code>ReleaseExecuted(lock_id, ...)</code> occurs AND either (a) the proof verifier accepts an invalid witness, or (b) no verifiable proof mode exists in production (implicit trust path)
Severity	■ Critical
Likelihood	Medium — verification boundaries and upgrade/config errors are historically common (cf. Wormhole guardian bypass)
Mitigation	(1) Explicit proof mode selection (light client / optimistic / trusted bootstrap) with TVL caps tied to trust level; (2) Per-lock binding: release requires membership proof of <code>lock_id</code> in <code>batch_commitment</code> ; (3) Independent audit of verifier + binding separately from business logic
Validated By	(1) End-to-end test: real finalized batch proof passes on-chain; (2) Negative tests: malformed proof, wrong lock membership, wrong batch commitment all rejected; (3) Verifier + binding path coverage $\geq$ defined threshold

Field	Content
Owner	Smart contract engineering
Status	Open — blocks mainnet

SC-3: Reentrancy or State Manipulation in Custody Contract

Field	Content
Risk	Attacker exploits reentrancy or unexpected state during lock/release/refund operations
Realized When	Funds extracted via callback during custody operation; lock state inconsistent after operation
Severity	■ Critical
Likelihood	Low — well-understood attack class with established mitigations (checks-effects-interactions, reentrancy guards)
Mitigation	(1) Checks-effects-interactions pattern enforced; (2) Reentrancy guard on all external calls; (3) Static analysis for reentrancy vulnerabilities
Validated By	(1) Audit confirms pattern adherence on all state-modifying functions; (2) Static analysis tools report zero reentrancy findings; (3) Negative test: reentrancy attempt reverts
Owner	Smart contract engineering
Status	Open — requires implementation + audit

SC-4: Integer Overflow/Underflow in Amount Handling

Field	Content
Risk	Arithmetic errors in amount calculations allow theft or fund creation

Field	Content
Realized When	Locked amount $\neq$ released amount without valid settlement; negative or overflow amounts processed
Severity	■ Critical
Likelihood	Low — Solidity 0.8+ has built-in overflow protection; requires explicit unsafe operations
Mitigation	(1) Use Solidity ≥ 0.8 with default overflow checks; (2) Explicit bounds validation on all amount inputs; (3) Invariant: $\text{sum}(\text{locked}) = \text{sum}(\text{released}) + \text{sum}(\text{refunded})$ enforced
Validated By	(1) Audit confirms no unchecked arithmetic; (2) Invariant test suite passes across all state transitions
Owner	Smart contract engineering
Status	Open — requires implementation + audit

SC-5: ZK Proof System Soundness Vulnerability

Field	Content
Risk	Undiscovered soundness bug in Sirius/Halo2 stack allows invalid proofs to verify
Realized When	Invalid state transition accepted by Midnight verification logic
Severity	■ Critical
Likelihood	Low — Halo2 is production-tested; Sirius modifications are scoped and audited
Mitigation	(1) Multiple audits of Sirius fork; (2) Property-based testing with known-bad proof corpus; (3) On-chain verification provides defense-in-depth; (4) TVL caps during initial deployment

Field	Content
Validated By	(1) Audit confirms no soundness issues in modified code; (2) Known-bad proofs rejected by verifier
Owner	Cryptography engineering
Status	Accepted — elimination infeasible; full justification in §6.5.3

6.3.3 Key Management / MPC Risks

MPC threshold compromise is the Ronin failure mode. Key management risks are not about cryptography; they're about operational security and collusion resistance.

KM-1: MPC Threshold Compromise (Midnight Signing)

Field	Content
Risk	Attacker compromises $\geq t$ of n FROST signers, enabling unauthorized Midnight transaction signing
Realized When	Valid BIP-340 signature produced for transaction not authorized by TEE attestation
Severity	■ Critical
Likelihood	Low — requires compromising multiple independent operators across administrative boundaries
Mitigation	(1) Geographic and administrative diversity requirements for MPC operators; (2) Hardware security modules or TEE-protected key shares; (3) Monitoring for signing anomalies (unexpected transaction patterns)

Field	Content
Validated By	(1) Operator diversity audit: no single entity controls $\geq t$ shares, operators span $\geq$ defined jurisdiction count; (2) Key ceremony documentation with witness attestations; (3) Anomaly detection drill: inject synthetic anomaly, alert fires, escalation path exercised
Owner	MPC operations + Key ceremony lead
Status	Open — requires operator onboarding

KM-2: MPC Threshold Compromise (Ethereum Signing)

Field	Content
Risk	Attacker compromises $\geq t$ of n ECDSA signers, enabling unauthorized Ethereum custody operations
Realized When	Valid ECDSA signature produced for release not matching verified finality proof
Severity	■ Critical
Likelihood	Low — same controls as KM-1
Mitigation	Same as KM-1; additionally: (1) Custody contract enforces finality proof even if signature is valid; (2) TVL caps limit blast radius
Validated By	Same as KM-1; additionally: (1) Negative test: valid signature + invalid proof → release rejected
Owner	MPC operations + Key ceremony lead
Status	Open — requires operator onboarding

KM-3: Key Share Extraction via Side-Channel

Field	Content
Risk	Attacker extracts MPC key shares via side-channel attacks on operator infrastructure
Realized When	Key material appears in logs, memory dumps, or network traces; signing possible without operator authorization
Severity	■ Critical
Likelihood	Low — requires physical or privileged access to operator infrastructure
Mitigation	(1) Key shares stored in HSM or TEE; (2) Key material never logged; (3) Memory encrypted at rest where supported
Validated By	(1) Operator attestation of HSM/TEE usage with hardware model documented; (2) Code review confirms no key logging paths; (3) Memory protection configuration audit
Owner	MPC operations + Key ceremony lead
Status	Open — requires operator security audit

KM-4: Signing Without Valid Attestation

Field	Content
Risk	MPC signs transaction without valid TEE attestation, bypassing authorization boundary
Realized When	Signature produced for request lacking valid attestation or with non-matching measurement
Severity	■ Critical
Likelihood	Medium — complex verification boundary with multiple code paths (attestation parsing, measurement matching, freshness checks); verification bypass is historically error-prone across systems (cf. Wormhole guardian check)

Field	Content
Mitigation	(1) MPC policy enforcement requires attestation verification before signing (§3.2); (2) Measurement allowlist with explicit version binding; (3) Attestation freshness requirement; (4) Policy parser/validator separately audited
Validated By	(1) Negative test: signing request without attestation → rejected; (2) Negative test: signing request with wrong measurement → rejected; (3) Negative test: signing request with expired attestation → rejected; (4) Policy code coverage ≥ defined threshold
Owner	Confidential compute platform lead
Status	Open — requires G1 attestation binding

6.3.4 TEE Security Risks

TEE risks are addressed in detail in §6.4 (Anti-Rollback Mechanism). This section summarizes the risk register entries; see §6.4 for protocol details.

TEE-1: Enclave Rollback Attack

Field	Content
Risk	Malicious host restores TEE to previous state, causing re-signing of already-processed batches
Realized When	Same batch_nonce signed twice; MPC accepts duplicate signing request
Severity	■ Critical
Likelihood	Medium — cloud providers have snapshot capabilities; requires malicious or compromised host
Mitigation	Monotonic batch_nonce enforced at MPC layer; MPC rejects nonce ≤ last-seen regardless of attestation validity (§6.4)

Field	Content
Validated By	Negative test: replay signing request with previous nonce → rejected by MPC; test repeated across enclave restart
Owner	Confidential compute platform lead
Status	Open — requires §6.4 protocol implementation
Detailed Treatment	§6.4 Anti-Rollback Mechanism

TEE-2: Enclave Forking Attack

Field	Content
Risk	Attacker runs multiple TEE instances from same state, causing parallel signing requests
Realized When	Multiple valid signing requests with same <code>batch_nonce</code> from different enclave instances
Severity	■ High
Likelihood	Low — requires both host compromise and defeating instance binding
Mitigation	(1) Nonce monotonicity at MPC prevents accepting second request; (2) Session binding in attestation links signing to specific instance
Validated By	Negative test: parallel signing requests with same nonce from different sessions → only first accepted; second rejected with specific error
Owner	Confidential compute platform lead
Status	Open — requires §6.4 protocol implementation
Detailed Treatment	§6.4 Anti-Rollback Mechanism

TEE-3: Attestation Forgery

Field	Content
Risk	Attacker constructs valid-looking attestation without legitimate TEE
Realized When	Signing request accepted with forged attestation quote
Severity	■ Critical
Likelihood	Low — requires compromising attestation root of trust (cloud provider PKI)
Mitigation	(1) Attestation verification against cloud provider root certificates; (2) Measurement binding (not just identity); (3) Freshness requirement prevents replay
Validated By	(1) Verification implementation matches cloud provider specification (audit confirms); (2) Negative test: expired attestation → rejected; (3) Negative test: malformed attestation → rejected; (4) Negative test: valid attestation with wrong measurement → rejected
Owner	Confidential compute platform lead
Status	Open — requires G1 attestation binding

TEE-4: Side-Channel Information Leakage

Field	Content
Risk	Attacker extracts intent plaintext or witness data via CPU side-channels (Spectre/Foreshadow class)
Realized When	Intent content or witness data observed outside TEE boundary
Severity	■ High
Likelihood	Low — requires physical proximity or co-tenancy; mitigations deployed by cloud providers

Field	Content
Mitigation	Side-channel attacks require attacker code on the same physical CPU to observe cache timing or speculative execution artifacts. Mitigations: (1) Dedicated hosts eliminate co-tenancy, removing the attack surface; (2) CPU microcode patches reduce exploitable speculation paths; (3) Minimizing sensitive data lifetime in enclave memory reduces the exposure window
Validated By	(1) Host configuration audit confirms dedicated hosting or equivalent isolation; (2) Microcode version $\geq$ vendor-recommended minimum. <i>Note: These confirm mitigation deployment; they do not prove absence of all side-channel vulnerability—novel attacks may emerge.</i>
Owner	DevOps/SRE
Status	Accepted — Side-channel leakage enables information disclosure but not fund theft: signing requires MPC threshold, which TEE compromise alone cannot bypass. An attacker who extracts intent data learns trade information but cannot sign unauthorized transactions. Residual risk acknowledged in §6.5.

6.3.5 Economic / Liquidity Risks

Economic risks affect fund availability and user experience but are generally not total-loss scenarios.

EC-1: Wrapped Asset Depeg

Field	Content
Risk	Wrapped asset loses peg to underlying due to custody incident or market perception
Realized When	Wrapped asset trades at significant discount to underlying (threshold: configurable)
Severity	■ High

Field	Content
Likelihood	Low — requires custody incident or sustained market loss of confidence
Mitigation	(1) TVL caps limit total exposure (§2.4.6); (2) Proof-of-reserves mechanism (custody balance verifiable on-chain); (3) Circuit breakers pause operations if anomalies detected
Validated By	(1) TVL caps enforced on-chain (test: deposit exceeding cap → rejected); (2) Reserves verification functional (test: query returns accurate balance); (3) Circuit breaker test: anomaly threshold → operations paused
Owner	Smart contract engineering
Status	Open — requires custody contract deployment

EC-2: Liquidity Drain via Coordinated Withdrawal

Field	Content
Risk	Coordinated withdrawal exhausts custody liquidity, stranding remaining users
Realized When	Withdrawal requests exceed available liquidity; users cannot exit positions
Severity	■ Medium
Likelihood	Low — requires coordination; affected by TVL caps
Mitigation	(1) TVL caps limit total exposure; (2) Per-address and per-period withdrawal limits; (3) Monitoring for unusual withdrawal patterns
Validated By	(1) Withdrawal limits enforced on-chain (test: withdrawal exceeding limit → rejected or queued); (2) Monitoring drill: spike in withdrawals → alert fires

Field	Content
Owner	Smart contract engineering
Status	Open — requires custody contract deployment

EC-3: Griefing via Repeated Timeout Forcing

Field	Content
Risk	Attacker repeatedly initiates settlements and forces timeouts, consuming resources and blocking capacity
Realized When	High rate of timeout-aborted settlements; legitimate users crowded out
Severity	■ Medium
Likelihood	Medium — low-cost attack if no rate limiting
Mitigation	(1) Lock deposits require minimum amount (anti-dust); (2) Rate limiting per address; (3) Timeout gas costs borne by initiator
Validated By	(1) Minimum lock enforced (test: sub-minimum lock → rejected); (2) Rate limits enforced (test: rapid repeated locks from same address → rejected after threshold); (3) Gas accounting verified
Owner	Smart contract engineering
Status	Open — requires custody contract deployment

EC-4: Intent Ordering Manipulation (MEV Extraction)

Field	Content
Risk	TEE operator extracts value by reordering, front-running, or selectively including intents within a batch
Realized When	Settlement ordering systematically benefits operator or affiliated parties; users receive worse execution than fair ordering would provide
Severity	■ Medium
Likelihood	Medium — TEE sees intent plaintext; operator controls batch construction
Mitigation	(1) Batch ordering policy enforced within TEE (e.g., FIFO by arrival time, or commit-reveal with encrypted intents until batch seal); (2) Attestation binds ordering policy to measurement—deviation detectable; (3) Execution price monitoring for systematic deviation from fair price
Validated By	(1) Ordering policy documented and bound to TEE measurement; (2) Statistical analysis of execution prices shows no systematic operator advantage; (3) Deviation beyond threshold triggers alert
Owner	Protocol engineering + Confidential compute platform lead
Status	Open — requires batch ordering policy specification

6.3.6 Operational / Governance Risks

Operational risks involve human processes and administrative controls.

OP-1: Admin Key Compromise

Field	Content
Risk	Attacker compromises contract admin key, enabling malicious upgrade or parameter change

Field	Content
Realized When	Unauthorized contract upgrade or parameter modification executed
Severity	■ Critical
Likelihood	Low — requires compromising multi-sig quorum
Mitigation	(1) Multi-sig admin with geographic diversity; (2) Timelock on critical parameter changes; (3) Monitoring for admin transactions
Validated By	(1) Multi-sig configuration audit: $\geq$ defined signer count, no single entity controls majority; (2) Timelock enforced on-chain (test: parameter change without timelock $\rightarrow$ reverts); (3) Monitoring drill: admin tx triggers alert within defined window
Owner	DevOps/SRE + Governance ops
Status	Open — requires contract deployment

OP-2: Malicious or Buggy Upgrade

Field	Content
Risk	Contract upgrade introduces vulnerability or intentional backdoor
Realized When	Upgraded contract exhibits behavior not present in audited version
Severity	■ Critical
Likelihood	Low — requires compromised governance or review failure
Mitigation	(1) All upgrades require audit of delta; (2) Timelock provides window for review; (3) Upgrade monitoring alerts community

Field	Content
Validated By	(1) Upgrade audit process documented and followed for each deployment; (2) Timelock duration verified sufficient for external review
Owner	Smart contract engineering + Governance ops
Status	Open — requires governance framework

OP-3: MPC Node Operator Collusion

Field	Content
Risk	Sufficient MPC operators collude to sign unauthorized transactions
Realized When	Unauthorized transaction signed despite no compromise of individual operators
Severity	■ Critical
Likelihood	Low — requires $\geq t$ operators acting maliciously in coordination
Mitigation	(1) Operator diversity requirements (legal jurisdiction, corporate structure); (2) Reputation/stake requirements for operators; (3) Slashing for provably malicious behavior
Validated By	(1) Diversity audit: operators span $\geq$ defined jurisdiction/entity count; (2) Operator agreements executed; (3) Slashing mechanism tested on testnet
Owner	Governance ops
Status	Open — requires operator framework

OP-4: Monitoring Blind Spot

Field	Content
Risk	Attack occurs but is not detected due to monitoring gaps (cf. Ronin's delayed detection)
Realized When	Incident discovered via external report rather than internal monitoring
Severity	■ High (multiplier on other risks)
Likelihood	Medium — monitoring is often incomplete at launch
Mitigation	(1) Define critical invariants and monitor continuously; (2) Anomaly detection on signing patterns, withdrawal patterns, proof submissions; (3) Incident response runbook with defined escalation
Validated By	(1) Monitoring coverage audit: all critical invariants have alerts; (2) Drill: inject synthetic incident → alert fires within defined window → page acknowledged → runbook executed → postmortem produced; (3) Drill repeated on defined schedule
Owner	DevOps/SRE
Status	Open — requires G4 operational readiness

OP-5: Global Liveness Coupling

Field	Content
Risk	Release gating uses a global nonce/watermark that can deadlock releases if any earlier batch gets stuck or disputed
Realized When	Later valid releases are rejected solely because an earlier batch release did not succeed
Severity	■ High (becomes ■ Critical if it forces stranded funds)

Field	Content
Likelihood	Medium — congestion and failed transactions occur in production
Mitigation	(1) Per-batch replay protection (<code>released_batches[batch_nonce]</code>) instead of global watermark; (2) Per-lock state (<code>locks[lock_id].status</code>) independent of batch ordering; (3) No global liveness dependency in release path
Validated By	Chaos test: force failure on batch k; confirm batch k+1 releases still succeed; confirm batch k can be retried or refunded independently
Owner	Protocol engineering
Status	Open — requires §6.2 implementation
Detailed Treatment	§6.2.5 (per-batch replay protection)

OP-6: Correlated Cloud Provider Failure

Field	Content
Risk	Multiple cloud providers experience simultaneous failure or coordinated action (legal seizure, correlated vulnerability)
Realized When	TEE infrastructure across multiple providers fails simultaneously; MPC quorum affected
Severity	■ High (liveness); ■ Critical if $\geq t$ shares compromised
Likelihood	Low — requires extraordinary coordination or common-mode vulnerability
Mitigation	(1) Multi-cloud deployment across AWS, GCP, Azure; (2) Geographic and jurisdictional distribution of MPC nodes; (3) Operator relationships with alternative providers; (4) Monitoring for coordinated infrastructure events

Field	Content
Validated By	(1) Deployment audit confirms multi-cloud distribution; (2) Chaos test: simulate single-provider outage → verify failover; (3) Legal review of jurisdiction diversity
Owner	Infrastructure/DevOps
Status	Accepted — primary impact is liveness, not safety; full justification in §6.5.3

OP-7: MPC Protocol Implementation Bug

Field	Content
Risk	FROST MPC implementation contains undiscovered bug causing safety or liveness failure
Realized When	Signing anomaly detected; unauthorized signature produced; or signing halts unexpectedly
Severity	■ High (liveness); ■ Critical if safety bypass
Likelihood	Low-Medium — FROST is newer than threshold ECDSA with fewer production deployments
Mitigation	(1) Multiple audits of FROST implementation; (2) Integration testing under chaos conditions; (3) Canary deployments before full rollout; (4) Signing anomaly detection and automatic pause
Validated By	(1) Audit confirms no critical issues; (2) Chaos suite passes; (3) Canary metrics within bounds before promotion
Owner	Cryptography engineering
Status	Accepted — bounded by on-chain constraints; full justification in §6.5.3

OP-8: Quarantine-based Denial of Service

Field	Content
Risk	Attacker triggers false-positive quarantines of legitimate TEE instances or MPC nodes, causing liveness degradation
Realized When	Legitimate signing requests rejected due to quarantine; throughput drops below SLA
Severity	■ High (liveness)
Likelihood	Low-Medium — requires ability to trigger quarantine mechanisms
Mitigation	(1) Rate limits on quarantine triggers; (2) Standby node promotion for quarantined primaries; (3) Re-provisioning ceremony requires operator quorum (prevents single-actor quarantine abuse); (4) Quarantine expiry with automatic reinstatement after verification
Validated By	(1) Chaos test: inject false quarantine triggers → verify standby promotion; (2) Rate limit enforcement confirmed under load; (3) Reinstatement procedure documented and tested
Owner	Operations/SRE
Status	Mitigated — rate limits and standby promotion bound impact; detailed treatment in §6.4.11

6.3.7 Cross-Chain Atomicity Risks

These risks are addressed in detail in [§6.2](#). Summary entries for completeness:

XC-1: Orphaned Release (Finality Race)

Field	Content
Risk	Ethereum releases funds before Midnight settlement is final; Midnight reorg invalidates settlement

Field	Content
Realized When	Release executed; Midnight settlement not in finalized chain
Severity	■ Critical
Likelihood	Medium — common bridge failure mode if finality not gated
Mitigation	Finality-gated release (§6.2.5); release requires verified finality proof
Validated By	Negative test: attempt release pre-finality → rejected; reorg test: Midnight reorg before finality → release blocked
Owner	Protocol engineering
Status	Open — requires §6.2 implementation
Detailed Treatment	§6.2 Cross-Chain Atomicity Protocol

XC-2: Commit Failure After Release (Protocol Loss)

Field	Content
Risk	Ethereum release (Phase 3) succeeds, but Midnight commit (Phase 4) fails—Bob receives ETH, but Alice's NIGHT remains in pending state until expiration, at which point Bob reclaims it
Realized When	ReleaseExecuted event on Ethereum; corresponding PendingSettlement on Midnight expires without SettlementCommitted event
Severity	■ Critical (protocol loss—Bob receives both ETH and reclaimed NIGHT)
Likelihood	Low — requires sustained NearFall operational failure after Ethereum release but before Midnight commit proof submission; or Midnight itself becomes unavailable

Field	Content
Mitigation	(1) Pending timeout >> expected commit proof submission time; (2) Commit proof submission with retry and escalation; (3) Circuit breaker pauses new settlements if commit backlog grows; (4) Monitoring alerts on pending settlements approaching expiration
Validated By	(1) Timeout relationship enforced on-chain (pending expiration > expected commit latency by defined margin); (2) Retry logic functional with defined max attempts and escalation path; (3) Circuit breaker test: commit backlog threshold → new settlements paused; (4) Monitoring drill: pending near expiration → alert fires
Owner	Protocol engineering
Status	Open — requires §6.2 implementation
Detailed Treatment	§6.2.6 (Phase 4: Commit)

Note: This is the "release succeeds but commit fails" edge case acknowledged in §6.2. The loss accrues to the protocol (or is socialized), not to Alice—Alice's refund path is blocked once Ethereum releases, but Bob receiving both assets is a protocol-level failure, not user error.

XC-3: Cross-Chain Verification Mechanism Mismatch

Field	Content
Risk	Custody contract relies on a finality proof that is not actually verifiable on Ethereum under the chosen deployment mode (no light client, no optimistic oracle, no trusted signer set defined)
Realized When	Release is either (a) impossible to execute due to unverifiable proof, causing system-wide stranded funds, or (b) executed via an ad-hoc trusted path not covered by audits
Severity	■ Critical

Field	Content
Likelihood	Medium — this is a common spec-to-implementation gap; verification mode selection is often deferred
Mitigation	(1) Explicitly choose Mode A/B/C verification (light client / optimistic oracle / trusted bootstrap) before deployment; (2) Codify verification mode in contract with explicit trust assumptions; (3) Tie TVL caps to trust level of chosen mode
Validated By	On-chain integration test: contract verifies a real finalized batch under the chosen mode; test covers happy path and all rejection paths
Owner	Protocol engineering + Smart contract engineering
Status	Open — requires verification mode selection
Detailed Treatment	§6.2.5 (cross-chain verification modes)

6.3.8 Risk Priority Matrix

Risks are prioritized by Severity × Likelihood. **P1** risks block launch; **P2** risks require mitigation before mainnet; **P3** risks are tracked and monitored.

Priorit y	Criteria	Action
P1	Critical severity AND (High OR Medium) likelihood	Must be mitigated before any production deployment
P2	Critical severity AND Low likelihood; OR High severity AND any likelihood	Must be mitigated before mainnet
P3	Medium or Low severity	Tracked; mitigation prioritized by resource availability
Accept	Low severity AND Low likelihood	Documented and accepted

Current P1 risks (block launch):

ID	Risk	Blocker
SC-1	Settlement proof bypass	Audit completion
SC-2	Release without valid cross-chain proof	Audit completion
KM-4	Signing without attestation	G1 attestation binding
TEE-1	Enclave rollback	§6.4 implementation
XC-1	Orphaned release	§6.2 implementation
XC-3	Cross-chain verification mismatch	Verification mode selection

Current P2 risks (before mainnet):

ID	Risk	Blocker
KM-1, KM-2	MPC threshold compromise	Operator onboarding
KM-3	Key share extraction	Operator security audit
TEE-2	Enclave forking	§6.4 implementation
TEE-3	Attestation forgery	G1 attestation binding
OP-1, OP-2	Admin/upgrade risks	Governance framework
OP-3	Operator collusion	Operator framework
OP-4	Monitoring blind spot	G4 operational readiness
OP-5	Global liveness coupling	§6.2 implementation
EC-1, EC-2, EC-3	Economic risks	Custody contract deployment

ID	Risk	Blocker
EC-4	Intent ordering manipulation (MEV)	Batch ordering policy specification
XC-2	Commit failure after release	§6.2 implementation

6.3.9 What This Section Establishes

Claim	Status
24 risks enumerated across 6 categories	■ Established
Each risk has severity, likelihood, mitigation, and validation criteria	■ Established
P1 risks identified with explicit blockers	■ Established
P2 risks identified with explicit blockers	■ Established
Cross-references to detailed treatment sections	■ Established
Risk register structure supports ongoing updates	■ Established

Design invariant (scoped): Assuming (1) contract correctness (no critical bugs in audited code), (2) MPC thresholds not compromised, (3) cross-chain proof mode functioning as specified, and (4) timeouts/refunds executable under network conditions—no failure mode results in permanent fund loss. Violations of these assumptions are explicitly enumerated as risks in this register.

Risk register maintenance: This register is a living document. New risks identified during implementation, audit, or operation should be added following the structure in [§6.3.1](#). Risk status should be updated as mitigations are validated.

What comes next: [§6.4](#) specifies the Anti-Rollback Mechanism—the detailed protocol that mitigates TEE-1 (enclave rollback) and TEE-2 (enclave forking) risks identified in this register.

6.4 Anti-Rollback Mechanism

Question this section answers: How does NearFall prevent an attacker from reverting the TEE to a previous state to re-sign already-processed batches or fork the enclave into parallel instances?

A rollback attack exploits a fundamental property of TEEs: the enclave doesn't know its own history. If a malicious host snapshots the enclave state, processes a batch, then restores the snapshot, the enclave believes the batch was never processed. Without countermeasures, it will sign the same batch twice—or sign conflicting batches from the same starting state. This is not a theoretical concern; researchers have demonstrated practical rollback and forking attacks against SGX-based cryptocurrency wallets, enabling double-spends.

NearFall's anti-rollback mechanism places the "memory" outside the TEE. The MPC network—which the TEE cannot control—tracks what has already been signed. Even if the TEE is rolled back, the MPC refuses to sign anything it has seen before. The TEE proposes; the MPC remembers.

6.4.1 The Rollback Problem

Why TEEs are vulnerable:

TEEs provide confidentiality and integrity for code execution, but they do not provide *state continuity*. The enclave can seal data to persistent storage (encrypted with a hardware-derived key), but it cannot verify whether that sealed data is the *latest* version or an old copy.

A malicious host can:

- **Snapshot** the enclave state before a signing operation
- **Allow** the signing operation to complete
- **Restore** the snapshot, returning the enclave to pre-signing state
- **Present** a different transaction for signing

The enclave, having "forgotten" the first signing, will sign the second transaction. The attacker now has two valid signatures from the same enclave instance—potentially for conflicting operations.

Forking attack variant:

Instead of sequential rollback, the attacker runs multiple enclave instances from the same starting state. Each instance believes it is unique and processes different inputs. This is called *enclave forking* or *cloning*. The result is parallel signing of conflicting transactions.

Historical precedent:

Researchers demonstrated these attacks against SGX-based cryptocurrency wallets. By forking an enclave or replaying sealed state, they caused double-spending of the same UTXO. The root cause: the enclave had no way to know whether it was the "real" instance or a clone, and no way to verify its sealed state was current.

6.4.2 Defense Architecture

NearFall's anti-rollback defense has three layers, each addressing a different attack surface:

Layer	What It Prevents	Mechanism
L1: MPC Nonce Enforcement	Re-signing of processed batches	MPC tracks <code>last_seen_nonce</code> per <code>signing_key_id</code> ; rejects <code>nonce ≤ last-seen</code>
L2: Checkpoint Hash Chain	Forking from intermediate states	Each checkpoint includes hash of previous checkpoint; deviation detectable
L3: Attestation Freshness	Replay of old attestation quotes	MPC issues challenge nonce; attestation must include challenge

Defense-in-depth rationale: No single layer is sufficient. L1 catches rollback to before a batch; L2 catches forks from the same state; L3 catches replay of captured attestations. An attacker must defeat all three layers simultaneously.

Specification note: Code blocks in this section are marked "non-normative" to indicate that exact syntax, field names, and data types may be adapted for implementation. However, the *semantics* described are normative: the invariants, verification logic, and security properties must be preserved. For example, `batch_nonce` must be a monotonically increasing integer large enough to never overflow in practice; whether it's `u64` or `u128` is implementation choice.

6.4.3 Layer 1: MPC Nonce Enforcement

Invariant: The MPC network will not sign any request with `batch_nonce ≤ last_seen_nonce` for that signing domain.

Critical design choice: Monotonicity is tracked **per signing key**, not per enclave instance.

If replay protection were scoped to `enclave_id`, an attacker who triggers re-provisioning (new enclave identity) would reset the nonce counter while the same MPC signing key remains active. This would bypass anti-rollback entirely. Instead:

- `signing_key_id` is a stable identifier for the signing domain (e.g., "Midnight MPC key" or "Ethereum custody key")
- `batch_nonce` is monotonic per `signing_key_id`
- `enclave_id` is used for telemetry, rate-limiting, and measurement allowlist checks—but not for replay protection

Mechanism:

The TEE includes a monotonically increasing `batch_nonce` in every signing request. The MPC tracks the highest nonce it has seen for each `signing_key_id`. Before signing, the MPC verifies:

```
ACCEPT signing request IF AND ONLY IF:
    request.batch_nonce == mpc_state.last_seen_nonce[signing_key_id] + 1
```

Constraint: Nonces must be **strictly sequential**, not just monotonically increasing. `batch_nonce == last_seen_nonce + 1` is required; `batch_nonce > last_seen_nonce + 1` (skip ahead) is rejected.

Rationale for strict sequentiality: If skipping were allowed, it introduces liveness and audit problems: missing checkpoints, ambiguous recovery state, and inconsistent state root chains. Strict sequentiality makes rollback/fork handling unambiguous.

Illustrative signing request structure (non-normative; field semantics are normative):

```
SigningRequest {
    signing_key_id: KeyId,           // Which MPC key domain
    batch_nonce: u64,                // Strictly sequential per signing_key_id
    tx_digest: [u8; 32],             // Transaction to sign
    batch_commitment: [u8; 32],      // Binds to settlement details
    checkpoint_hash: [u8; 32],       // Current checkpoint (for L2)
    prev_checkpoint_hash: [u8; 32],  // Previous checkpoint (for L2 chaining)
    attestation: AttestationQuote,   // TEE attestation binding to request
}
```

Constraint: `batch_nonce` is the *only* value the MPC uses for replay protection. The MPC does not trust any other field from the TEE for ordering.

What this prevents:

Attack	Outcome
Rollback to before batch N	TEE proposes batch N again; MPC rejects (nonce $\leq$ last-seen)
Fork: two instances from state before batch N	First instance's batch N succeeds; second instance's batch N rejected
Re-provision attack (new enclave, same key)	Rejected: nonce is per <code>signing_key_id</code> , not per enclave; new enclave must continue from last nonce
Skip ahead (propose batch N+2 before N+1)	Rejected: strict sequential required

What this does NOT prevent:

- Rollback to a state that has never been signed (the "genesis" attack)—addressed by L2

- Replay of a *successful* signing response—but the transaction is already on-chain, so replaying the signature has no effect
- Epoch reset attack (operator-approved nonce reset)—addressed by re-provisioning ceremony (§6.4.8)

MPC state durability:

`last_seen_nonce[signing_key_id]` must survive MPC node restarts. Each MPC node persists its view to durable storage. On restart, the node recovers from persisted state. If persisted state is lost, the node must sync from peers before participating in signing.

Atomic update requirement: A signing round is valid only if a quorum **durably records** (`signing_key_id`, `batch_nonce`, `checkpoint_hash`, `tx_digest`) **before** releasing the signature shares. Otherwise, a crash could "forget" that a nonce was used and re-sign after restart.

Constraint: A quorum of MPC nodes must agree on `last_seen_nonce` for a signing request to proceed. A single node with stale state cannot cause a rollback vulnerability.

6.4.4 Layer 2: Checkpoint Hash Chain

Invariant: Each TEE checkpoint chains from the previous checkpoint. The MPC tracks `last_checkpoint_hash` per signing key and rejects requests that don't chain correctly.

Mechanism:

The TEE maintains a checkpoint that includes:

- Current `batch_nonce`
- State root after last processed batch
- Hash of the previous checkpoint

Illustrative checkpoint structure (non-normative; field semantics are normative):

```
Checkpoint {
    batch_nonce: u64,
    state_root: [u8; 32],
    prev_checkpoint_hash: [u8; 32],
    measurement: [u8; 48],           // TEE code measurement
}

checkpoint_hash = H(Checkpoint)
```

The signing request includes both `prev_checkpoint_hash` and `checkpoint_hash`. The MPC verifies:

Illustrative verification (non-normative; field semantics are normative):

```

ACCEPT request IF:
  // Chaining: previous checkpoint matches MPC's last known
  request.prev_checkpoint_hash == mpc_state.last_checkpoint_hash[signing_key_id]

  // Sequentiality: nonce is exactly next (L1)
  AND request.batch_nonce == mpc_state.last_seen_nonce[signing_key_id] + 1

AFTER SIGNING:
  mpc_state.last_checkpoint_hash[signing_key_id] = request.checkpoint_hash
  mpc_state.last_seen_nonce[signing_key_id] = request.batch_nonce

```

Genesis bootstrap: The first batch has no previous checkpoint to chain from. The MPC accepts a genesis request if and only if:

- No `last_seen_nonce` exists for this `signing_key_id` (key domain is uninitialized)
- Attestation is valid with approved measurement
- `batch_nonce == 1`
- `prev_checkpoint_hash == GENESIS_HASH` (well-known constant, e.g., all zeros)

Once a genesis request is accepted, the signing key domain is "initialized" and subsequent requests must chain.

Illustrative genesis check (non-normative):

```

ACCEPT genesis IF:
  signing_key_id NOT IN mpc_state.initialized_keys
  AND request.batch_nonce == 1
  AND request.prev_checkpoint_hash == GENESIS_HASH
  AND attestation_valid(request.attestation)

EFFECT:
  mpc_state.initialized_keys.add(signing_key_id)
  mpc_state.last_seen_nonce[signing_key_id] = 1
  mpc_state.last_checkpoint_hash[signing_key_id] = request.checkpoint_hash

```

What this prevents:

Attack	Outcome
Fork from state S, produce $S \rightarrow A$ and $S \rightarrow B$	Both claim <code>prev_checkpoint_hash = H(S)</code> ; only first can chain from MPC's view
Rollback to checkpoint C, produce different continuation	New checkpoint doesn't chain from MPC's last-known checkpoint
Genesis replay (re-initialize after operation)	Rejected: <code>signing_key_id</code> already in <code>initialized_keys</code>

What this does NOT prevent:

- Attacks where the attacker controls MPC quorum (assumption violation)
- Novel TEE exploits that forge attestations (assumption violation)

Constraint: The checkpoint hash chain creates a total ordering of TEE states as observed by the MPC. Forks are detectable because they produce divergent chains.

6.4.5 Layer 3: Attestation Freshness and Content Binding

Invariant: Every signing request must include an attestation quote that (1) contains a fresh challenge nonce and (2) cryptographically binds to the specific request being signed.

Critical design choice: The attestation must bind to **what is being signed**, not just "this is a fresh attestation session." Without content binding, an attacker could splice a fresh attestation onto a different `tx_digest`.

Mechanism:

Before the TEE submits a signing request, it obtains a fresh challenge from the MPC:

- TEE requests challenge, including its `enclave_id` (derived from measurement + instance key)
- MPC generates `challenge_nonce` (random, unique, time-bounded) and stores `pending_challenges[enclave_id] = challenge_nonce`
- TEE computes request binding: `request_binding = H(challenge_nonce || signing_key_id || batch_nonce || checkpoint_hash || tx_digest || batch_commitment)`
- TEE generates attestation quote with `request_binding` in user-data field
- TEE submits signing request with attestation
- MPC recomputes `request_binding` from request fields and verifies it matches `attestation.user_data`

Content binding: The attestation authenticates the *statement* ("I, this TEE, am requesting signature on this specific transaction with this nonce"), not just the *session* ("I am a valid TEE"). An attacker who captures an

attestation cannot use it to sign a different transaction.

Illustrative attestation verification (non-normative; field semantics are normative):

```

ACCEPT attestation IF:
  // Quote is valid per cloud provider verification
  verify_quote_signature(attestation.quote, cloud_provider_root)

  // Measurement matches approved code
  AND attestation.measurement IN approved_measurements

  // Extract enclave identity
  enclave_id = derive_enclave_id(attestation.measurement, attestation.instance_data)

  // Enclave is in allowlist and not quarantined
  AND enclave_id IN mpc_state.enclave_allowlist
  AND enclave_id NOT IN mpc_state.quarantined_enclaves

  // Challenge is fresh
  AND challenge_issue_time[enclave_id] > (now - MAX_CHALLENGE_AGE)

  // Content binding: attestation binds to this specific request
  expected_binding = H(
    pending_challenges[enclave_id] ||
    request.signing_key_id ||
    request.batch_nonce ||
    request.checkpoint_hash ||
    request.tx_digest ||
    request.batch_commitment
  )
  AND attestation.user_data == expected_binding

```

Constraint: `MAX_CHALLENGE_AGE` must be bounded. Challenges older than this threshold are rejected regardless of other validity. Challenges are single-use; after verification, `pending_challenges[enclave_id]` is cleared.

What this prevents:

Attack	Outcome
Capture attestation at time T, replay at time T+Δ	Challenge nonce won't match current pending challenge
Pre-generate attestations for future use	Each attestation is bound to a specific challenge

What this does NOT prevent:

- Attacks that complete within the challenge window—but combined with L1/L2, the attacker gains nothing from speed

6.4.5.1 Multi-TEE Scaling Model

Problem: NearFall may require multiple TEE instances for throughput and availability. With strict sequential nonces per `signing_key_id`, how do parallel enclaves avoid appearing indistinguishable from a forking attack?

Design choice: Single Active Leader per Signing Key

NearFall uses a **single active leader** model for each `signing_key_id`:

Property	Specification
Active TEE count	Exactly one TEE instance may submit signing requests for a given <code>signing_key_id</code> at any time
Leader election	Determined by MPC; only the enclave in <code>active_leader[signing_key_id]</code> may sign
Failover	If leader fails health checks, MPC initiates leader transition to standby
Standby TEEs	May exist for availability; cannot submit signing requests until promoted

Leader transition protocol:

- MPC detects leader failure (health check timeout, repeated attestation failures)
- MPC marks current leader as `failed` and increments `leader_epoch[signing_key_id]`
- Standby TEE requests promotion with fresh attestation
- MPC verifies standby is in allowlist and not quarantined
- MPC sets `active_leader[signing_key_id] = standby_enclave_id`
- Standby continues from `last_seen_nonce[signing_key_id] + 1`

Constraint: Leader transition requires MPC quorum agreement. A single MPC node cannot unilaterally change the active leader.

Why not nonce leasing: Nonce leasing (MPC issues ranges `[a, b]` to TEE instances) introduces complexity around range exhaustion, range collision, and audit gaps. Single-leader is simpler and sufficient for NearFall's throughput requirements.

Constraint: Signing requests from non-leader enclaves are rejected even if attestation is valid:

```
REJECT IF enclave_id != mpc_state.active_leader[signing_key_id]
```

6.4.6 Recovery vs. Attack Distinction

Problem: Legitimate restarts look similar to rollback attacks. The enclave loads persisted state and resumes operation. How does the MPC distinguish "enclave crashed and recovered" from "attacker restored old snapshot"?

Solution: Recovery is permitted if and only if the enclave resumes from the *last known checkpoint* with the *next expected nonce*.

Recovery protocol:

- TEE starts and loads sealed checkpoint from persistent storage
- TEE attests (with fresh challenge from MPC)
- TEE submits signing request with:
 - `batch_nonce = last_seen_nonce + 1`
 - `prev_checkpoint_hash = last_checkpoint_hash` (as known by MPC)
- MPC verifies chain continuity and accepts

Attack scenario (rejected):

- Attacker loads *old* sealed checkpoint (from before batch N)
- TEE attests (with fresh challenge—attacker can't avoid this)
- TEE submits signing request with:
 - `batch_nonce = N` (but MPC has already seen N)
 - `prev_checkpoint_hash = checkpoint_before_N` (but MPC expects `checkpoint_after_N`)
- MPC rejects: `nonce ≤ last-seen` AND checkpoint doesn't chain

Edge case: Crash during signing

If the TEE crashes after MPC signs but before the TEE persists the new checkpoint:

- On restart, TEE's checkpoint is one batch behind
- TEE proposes batch N again
- MPC rejects (`nonce ≤ last-seen`)
- TEE requests recovery for batch N
- MPC provides the already-signed transaction
- TEE verifies and updates checkpoint without re-signing

Crash recovery authentication: The recovery path is a potential injection vector and must be secured:

- TEE sends recovery request: `{enclave_id, recovered_nonce, expected_tx_digest}`
- MPC responds with: `{nonce, tx_digest, signature}` from `last_signed[enclave_id]`
- TEE verifies:
 - `nonce == recovered_nonce + 1` (matches expected next batch)

- `tx_digest == expected_tx_digest` (TEE recomputes from its own state; does not trust MPC's content)
- `verify_signature(signature, tx_digest, mpc_public_key)` (signature is valid)
- If all checks pass, TEE updates checkpoint to reflect the signed batch
- If any check fails, TEE aborts and alerts operators (possible attack or state corruption)

Constraint: The TEE does not blindly trust the MPC's `tx_digest`. It recomputes the expected digest from its own recovered state and verifies they match. This prevents the MPC from injecting arbitrary transactions during recovery.

Constraint: Recovery requests require fresh attestation (L3 still applies). An attacker cannot replay old recovery requests.

6.4.7 Hardware Protections (Defense in Depth)

Key insight: TEEs provide confidentiality and integrity, but **not state freshness**. An enclave cannot distinguish "first run" from "replayed run" using hardware alone.

The protocol-level defenses (L1–L3) are sufficient given the threat model (malicious host, honest MPC majority). Hardware protections provide additional defense-in-depth but are not relied upon for security.

What hardware provides and doesn't provide:

Capability	SGX	SEV-SNP	TDX	NearFall Reliance
Memory confidentiality	■	■	■	Used (intent privacy)
Memory integrity (runtime)	■	■ RMP	■	Defense-in-depth
TCB rollback protection	■	■ COM MITTE D_TCB	■	Defense-in-depth
Application state freshness	■	■	■	Not relied upon
Monotonic counters	■■ Limited	■	■■ Limited	Not relied upon

Why monotonic counters are insufficient: Intel SGX Platform Services provides monotonic counters, but they have practical limitations: high latency per increment, limited quota (finite counters, finite increments before exhaustion), and availability depends on Intel ME/CSME. SEV-SNP and TDX do not provide native

application-level monotonic counters. These platforms solve TCB version anti-rollback (preventing firmware downgrade) but **not** application state rollback.

NearFall's position: These platforms do not solve application-level rollback of persisted state; external continuity mechanisms (L1–L3 via MPC) are required. This is why recent academic work continues to address rollback protection for confidential VMs as an open problem.

Constraint: NearFall does not rely on hardware monotonic counters. The MPC-based nonce enforcement (L1) provides equivalent protection without hardware dependencies, ensuring portability across TEE platforms.

6.4.8 Failure Modes and Response

Detected rollback:

Symptom	Detection	Response
<code>batch_nonce != last_seen + 1</code>	MPC nonce check	Reject signing; alert operators; quarantine enclave identity
Checkpoint doesn't chain	MPC checkpoint verification	Reject signing; alert operators
Content binding mismatch	MPC attestation verification	Reject signing; likely splice attack
Attestation measurement mismatch	MPC measurement allowlist	Reject signing; possible code tampering or downgrade
Request from non-leader enclave	MPC leader check	Reject signing; possible fork attempt

Constraint: On any rollback detection, the MPC must:

- Refuse to sign
- Log the incident with full request details
- Alert operators
- Quarantine the enclave identity (require manual re-provisioning before accepting further requests)

Quarantine as DoS target:

An attacker controlling the host can intentionally trigger rollback detection to force quarantine. This is not theft, but it is a liveness attack. Mitigations:

Control	Purpose
Challenge rate limit	Bounded challenge issuance per <code>enclave_id</code> per time window
Signing attempt rate limit	Bounded signing requests per <code>enclave_id</code> per time window
Backoff on failure	Exponential backoff after repeated failures from same <code>enclave_id</code>
Standby promotion	If leader is quarantined, standby can be promoted (§6.4.5.1)

Re-provisioning ceremony:

Re-provisioning a quarantined enclave or resetting `last_seen_nonce` requires an explicit operator-approved ceremony:

Step	Requirement
1. Investigation	Operator reviews quarantine cause; confirms no attack evidence
2. Approval	Quorum of authorized operators approves re-provisioning
3. State freeze	New settlements paused for <code>signing_key_id</code> during ceremony
4. Continuity	New enclave must continue from <code>last_seen_nonce + 1</code> ; nonce reset is never permitted without epoch change
5. Epoch change	If nonce continuity is impossible (catastrophic state loss), a new <code>signing_key_id</code> epoch is created; on-chain state must be migrated

Constraint: Automated re-provisioning is prohibited. An attacker who can trigger quarantine cannot exploit the re-provisioning process without operator collusion.

Constraint: Nonce reset (returning to nonce 1) for an existing `signing_key_id` is **never** permitted. If state is unrecoverably lost, a new epoch with a new key must be created.

6.4.9 Protocol Boundaries

What crosses the TEE → MPC boundary:

Field	Source	Verification
<code>signing_key_id</code>	TEE request	MPC checks key exists and enclave is leader
<code>batch_nonce</code>	TEE internal counter	MPC checks <code>== last_seen + 1</code>
<code>checkpoint_hash</code>	TEE computed	MPC checks chains from last-known
<code>prev_checkpoint_hash</code>	TEE state	MPC checks <code>== last_checkpoint_hash</code>
<code>tx_digest</code>	TEE settlement engine	MPC does not verify content (see trust boundary below)
<code>batch_commitment</code>	TEE computed	Bound in attestation; MPC does not verify semantics
<code>attestation</code>	TEE + hardware	MPC verifies signature, measurement, content binding

Least-privilege signing key requirement:

The MPC signing keys must be **purpose-limited**. Specifically:

Constraint	Rationale
MPC signing keys cannot be used for contract admin/upgrade	Prevents signed tx from modifying contract logic
MPC signing keys cannot modify protocol parameters	Prevents signed tx from changing timeouts, caps, etc.
Signed transactions are still gated by on-chain verifiers	Midnight: ZK proof verification; Ethereum: finality proof + batch membership

Why this matters: The MPC trusts the TEE's `tx_digest` without verifying its content. This is acceptable **only** because:

- A malicious `tx_digest` for Midnight cannot steal funds without a valid ZK proof (which requires correct computation)
- A malicious `tx_digest` for Ethereum cannot release custody without valid finality proof and batch membership proof
- The signing keys cannot perform admin actions that would bypass these verifiers

Constraint: If MPC signing keys could perform admin actions, a compromised TEE could sign an upgrade transaction that removes the on-chain verifiers, then sign arbitrary theft transactions. Least-privilege scoping prevents this escalation.

What the MPC persists:

State	Scope	Purpose	Durability Requirement
<code>last_seen_nonce[signing_key_id]</code>	Per signing key	Replay protection	Survives node restart; syncs across quorum
<code>last_checkpoint_hash[signing_key_id]</code>	Per signing key	Fork detection	Survives node restart; syncs across quorum
<code>active_leader[signing_key_id]</code>	Per signing key	Leader election	Survives node restart; syncs across quorum
<code>enclave_allowlist</code>	Global	Measurement authorization	Survives node restart
<code>quarantined_enclaves</code>	Global	Blocked enclaves	Survives node restart
<code>pending_challenges[enclave_id]</code>	Per enclave	Attestation freshness	Ephemeral (bounded lifetime)
<code>last_signed[signing_key_id]</code>	Per signing key	Crash recovery	Survives node restart

Atomic durability requirement:

A signing round is valid only if a quorum **durably records** (`signing_key_id`, `batch_nonce`, `checkpoint_hash`, `tx_digest`) **before** releasing signature shares.

Illustrative signing round (non-normative):

SIGNING_ROUND:

1. Receive signing request from leader TEE
2. Verify all preconditions (nonce, checkpoint, attestation, leader status)
3. Durably persist (signing_key_id, batch_nonce, checkpoint_hash, tx_digest) to local st...
4. Wait for quorum of nodes to confirm durable persistence
5. ONLY THEN: release signature share
6. After quorum signature assembly: update last_seen_nonce and last_checkpoint_hash

Constraint: If a node crashes after step 3 but before step 6, it must recover the persisted state and complete the update. If a node crashes before step 3, the signing round fails safely (no state change, no signature).

6.4.10 Validation Gates

The following tests validate the anti-rollback mechanism:

Test	Scenario	Expected Outcome
Nonce replay test	Submit signing request with batch_nonce = last_seen	Rejected with specific error
Nonce regression test	Submit signing request with batch_nonce < last_seen	Rejected with specific error
Nonce skip test	Submit signing request with batch_nonce = last_seen + 2	Rejected (must be strictly sequential)
Checkpoint fork test	Submit two requests with same prev_checkpoint_hash	First succeeds; second rejected
Checkpoint gap test	Submit request skipping checkpoint	Rejected (doesn't chain)
Stale attestation test	Submit request with old challenge nonce	Rejected
Content binding test	Submit request where attestation binds to different tx_digest	Rejected (splice attack detected)
Measurement mismatch test	Submit request with wrong measurement	Rejected
Non-leader request test	Submit signing request from standby (not active leader)	Rejected

Test	Scenario	Expected Outcome
Re-provision attack test	Launch new enclave identity; attempt to sign with nonce 1	Rejected (nonce must continue from <code>last_seen + 1</code> for existing <code>signing_key_id</code>)
Crash recovery test	Simulate crash; restart; submit next batch	Succeeds (chains correctly)
Quarantine test	Trigger rollback detection; verify quarantine	Enclave quarantined; subsequent requests rejected until re-provisioning
MPC node restart test	Restart MPC node; verify state recovery	Node syncs from peers; no security degradation
Atomic durability test	Crash MPC node after durable record, before signature release	On restart, node recognizes nonce as used; will not re-sign

Constraint: All tests must pass before G4 (operational readiness). Crash recovery, quarantine, and atomic durability tests must be executed against realistic deployment topology.

6.4.11 Relationship to Risk Register

This section provides the detailed mitigation for risks identified in §6.3:

Risk ID	Risk	Mitigation in §6.4
TEE-1	Enclave rollback attack	L1 (nonce enforcement per <code>signing_key_id</code>) + L2 (checkpoint chain)
TEE-2	Enclave forking attack	L2 (checkpoint chain) + single-leader model (§6.4.5.1)
TEE-3	Attestation forgery	L3 (content-bound attestation) + measurement binding
KM-4	Signing without valid attestation	L3 (challenge-response) + measurement allowlist

New risk identified during §6.4 specification:

Risk ID	Risk	Mitigation
OP-8	Quarantine-based DoS	Rate limits on challenges/signing attempts; standby promotion; re-provisioning ceremony requires operator quorum

Cross-reference: OP-8 (Quarantine-based DoS) is registered in §6.3.6 with Status: Mitigated.

Security claim (scoped): Assuming:

- An honest-majority MPC threshold (t-of-n with fewer than t nodes compromised)
- Correct attestation verification against the cloud provider's root of trust
- Least-privilege signing keys whose outputs are still gated by on-chain verifiers (ZK proof verification on Midnight; finality + batch membership proof on Ethereum)

Then a malicious host **cannot**:

- Cause the MPC to produce two signatures for the same (signing_key_id, batch_nonce) pair
- Cause the MPC to produce a signature that chains from a stale checkpoint_hash
- Splice a valid attestation onto a different tx_digest (content binding prevents this)
- Reset the nonce counter by re-provisioning a new enclave identity (nonce is per signing_key_id, not per enclave_id)

This claim is testable via the validation gates in §6.4.10.

6.4.12 What This Section Establishes

Claim	Status
Three-layer anti-rollback defense (nonce, checkpoint chain, content-bound attestation)	■ Established
Replay protection scoped to signing_key_id, not enclave_id	■ Established
Strictly sequential nonces (== last + 1, not just > last)	■ Established
Attestation binds to request content (prevents splice attacks)	■ Established
Single active leader model for multi-TEE scaling	■ Established

Claim	Status
Least-privilege signing keys; on-chain verifiers gate fund movement	■ Established
MPC-side state tracking prevents replay regardless of TEE state	■ Established
Recovery protocol distinguishes legitimate restart from attack	■ Established
Hardware protections are defense-in-depth, not primary mechanism	■ Established
Failure modes trigger quarantine; re-provisioning requires operator ceremony	■ Established
Rate limits mitigate quarantine-based DoS	■ Established
Atomic durability: quorum records state before releasing signature	■ Established
Validation gates defined (14 tests)	■ Established
Risk register cross-references (TEE-1, TEE-2, TEE-3, KM-4, OP-6)	■ Established
Testable security claim with explicit assumptions	■ Established

What comes next: §6.5 addresses residual risks—attack vectors that remain after all specified mitigations, including unknown-unknowns and accepted risks.

6.5 Residual Risk Acknowledgment

Question this section answers: What risks remain after all specified mitigations are in place, and why are they accepted rather than mitigated?

A risk register that claims to eliminate all risks is a fantasy document. Every system has a residual risk floor—attack vectors that survive defense-in-depth because eliminating them would require assumptions we cannot make or costs we cannot bear. This section names those risks explicitly. If you deploy NearFall, you accept these residual risks. If any is unacceptable, do not deploy.

6.5.1 The Residual Risk Floor

Definition: The residual risk floor is the set of attack vectors that succeed even when all mitigations in §6.2–§6.4 are functioning correctly.

NearFall's residual risk floor has one fundamental boundary:

> **Residual boundary:** If an attacker simultaneously compromises (i) the NearFall confidential compute boundary (TEE) *and* (ii) the MPC threshold that authorizes signatures, then the attacker can generate policy-conformant signing requests and obtain threshold signatures. At that point, prevention reduces to **on-chain constraints** (caps, rate limits, verifiable membership/finality requirements) and **operator governance** (diversity, monitoring, rapid pause/rotation). The architecture cannot *guarantee* safety under a coordinated compromise of both core trust boundaries.

What on-chain constraints provide: Even with signing authority, an attacker cannot bypass on-chain verifiers. Ethereum custody release still requires valid finality proofs and batch membership proofs (§6.2). Midnight settlement still requires valid ZK proofs. Rate limits and caps bound the blast radius per time window. These are not cryptographic defenses *inside* NearFall—they are constraints on what signatures can *effect* on-chain.

Why this is accepted: Eliminating this risk would require either (a) a third independent authority (adding complexity and another compromise target) or (b) user-held keys for every transaction (destroying the UX that NearFall exists to provide). Neither is acceptable. The risk is mitigated by operational distribution and on-chain constraints, not eliminated by architecture.

Relationship to non-goals (§6.5.2): The residual risk floor describes attacks that defeat NearFall's *specific* defenses—the separation of TEE (confidentiality) and MPC (authorization). Non-goals describe attacks that defeat *foundational assumptions* shared by all systems of this class. MPC supermajority collusion is a non-goal because *any* threshold system fails under this condition; it is not a NearFall-specific vulnerability. TEE+MPC compromise is the floor because NearFall's architecture specifically relies on their separation; defeating both is the NearFall-specific irreducible risk.

Relationship between floor and non-goals: The residual risk floor describes attacks that defeat NearFall's *specific* defenses. Non-goals (§6.5.2) describe attacks that defeat foundational assumptions shared by all systems of this class. MPC supermajority collusion is a non-goal because *any* threshold system fails under this condition—it's not a NearFall-specific vulnerability. TEE+MPC compromise is the floor because NearFall's architecture specifically relies on their separation; defeating both is NearFall-specific.

6.5.2 Explicit Non-Goals

NearFall does not claim to protect against the following scenarios. These are not risks we failed to mitigate; they are risks we explicitly exclude from scope.

Category 1: Foundational Infrastructure Failures

NearFall cannot be more secure than the infrastructure it runs on.

Non-Goal	Rationale
Global TEE platform compromise	If Intel, AMD, or a cloud provider disables or backdoors all TEE hardware, NearFall's confidentiality model fails. This is a civilizational-level event outside our threat model.
Midnight L1 consensus failure	If Midnight's consensus breaks (51% attack, catastrophic bug), NearFall cannot provide settlement finality. NearFall extends Midnight; it cannot be more secure than Midnight.
Ethereum L1 consensus failure	Same reasoning. If Ethereum reorganizes deeply or halts, custody operations are affected.

Category 2: Threat Model Exclusions

These attacks are real but occur outside NearFall's trust boundary.

Non-Goal	Rationale
User key compromise	If a user's private key is stolen, NearFall cannot prevent the attacker from submitting intents as that user. User key security is the user's responsibility.
Client-side malware	If malware on a user's device modifies intents before submission, NearFall sees the modified intent as authentic. Client security is out of scope.

Category 3: By-Definition Threshold Properties

These are not failures of the system; they are the definition of threshold security.

Non-Goal	Rationale
MPC supermajority collusion ($\geq$ threshold)	If t-of-n MPC nodes collude (where t is the threshold), they can sign anything. This is the definition of threshold security, not a failure of it. Mitigation is operator diversity, not protocol change.

Why these are non-goals, not failures: Each represents a compromise of a foundational assumption. NearFall's security claims are conditional on TEE hardware functioning, MPC thresholds holding, and

underlying chains being live and consistent. Removing these conditions would require NearFall to be more secure than the infrastructure it runs on—an impossibility.

6.5.3 Accepted Risks (Mitigated but Not Eliminated)

This section elaborates risks that are accepted rather than fully mitigated. It includes:

- Risks from §6.3 marked "Accepted" (TEE-4)
- Foundational risks not suitable for the component-level register format (ZK soundness, correlated cloud, MPC implementation)

Cross-references: SC-5 (ZK Soundness), OP-6 (Correlated Cloud), and OP-7 (MPC Implementation) are registered in §6.3 with Status: Accepted.

The following risks are accepted because full elimination is impossible or disproportionately costly. Mitigations reduce likelihood or impact but do not eliminate the risk.

TEE-4: Side-Channel Information Leakage

Risk: Attacker extracts intent plaintext via CPU side-channels (Spectre/Foreshadow class).

Why accepted: Side-channel attacks are an active research area. New variants emerge regularly. Mitigations (dedicated hosts, microcode patches, minimized data lifetime) reduce but cannot eliminate the attack surface. A sufficiently sophisticated attacker with physical access or co-tenancy may extract information.

Impact boundary: Information disclosure only—*assuming* custody release is gated by on-chain verifiable conditions (finality/membership proofs per §6.2) and MPC policy checks are correct. Side-channel compromise of a TEE does not grant signing authority; the MPC threshold is a separate system. An attacker who extracts intent data learns trade information (amounts, counterparties, direction) but cannot steal funds without also compromising the MPC threshold.

Residual impact: Privacy breach for affected intents. Potential front-running or market manipulation if attacker acts on leaked information. Not fund theft.

Operational response: If a novel side-channel is published, evaluate applicability to deployed TEE platform. If applicable, rotate to patched infrastructure or pause operations until patch is available.

Novel ZK Soundness Vulnerabilities

Risk: The ZK proof system (Sirius/Halo2 stack) contains an undiscovered soundness bug allowing invalid proofs to verify.

Why accepted: ZK proof systems are mathematically complex. Formal verification of the entire stack is infeasible in the deployment timeframe. Audits reduce but cannot eliminate the possibility of subtle soundness bugs.

Impact boundary: If exploited, invalid state transitions could be accepted by Midnight verification logic. The primary blast radius is Midnight-side accounting and settlement. Ethereum custody is affected only if the custody release path accepts batch commitments derived from broken proofs—but release still requires valid

finality observation and batch membership proofs (§6.2), which provide independent verification.

Acceptance criteria note: This risk does not fully meet the standard "bounded impact" criterion from §6.5.6—a soundness exploit could theoretically affect all Midnight-side state. It is accepted as a *foundational risk* because: (1) elimination is infeasible without abandoning ZK proofs entirely, (2) the alternative (optimistic/fraud-proof systems) introduces different unbounded risks, and (3) the blast radius is bounded to Midnight state, not Ethereum custody.

Residual impact: State manipulation on Midnight. Attacker could mint unbacked wrapped assets, drain Midnight-side liquidity pools, or manipulate settlement outcomes. Unlike TEE-4 (information disclosure), this is a fund-theft risk within Midnight's domain.

Mitigation depth: Multiple audits, property-based testing, known-bad proof corpus. Sirius is a fork of audited code (Halo2); modifications are scoped and audited separately. On-chain verification logic provides defense-in-depth: most invalid proofs should fail verification even with subtle soundness bugs.

Detection mechanism: Security research community publishes vulnerabilities. Proof corpus analysis may detect novel attack patterns. On-chain monitoring for anomalous state transitions (e.g., unexpected mints, balance inconsistencies).

Operational response:

- Immediate settlement pause on CVE disclosure affecting Halo2/Sirius
- Assess whether NearFall's specific proof circuits are affected
- If affected: freeze new locks, hold all pending releases, allow refunds via timeout
- Post-incident: audit all batches processed during vulnerability window
- Deploy patched version; resume only after verification

Correlated Cloud Provider Failure

Risk: Multiple cloud providers experience simultaneous failure or coordinated action (e.g., legal seizure, coordinated vulnerability).

Why accepted: Multi-cloud deployment reduces single-provider risk but cannot eliminate correlated risks. If AWS, GCP, and Azure all experience the same vulnerability (e.g., a CPU microcode bug affecting all x86 platforms), multi-cloud provides no protection.

Impact boundary: Primary expected impact is liveness failure (cannot process settlements) rather than safety failure. Safety can degrade only if correlated failure enables compromise of $\geq t$ MPC shares (e.g., coordinated legal seizure of all operator infrastructure) or circumvents attestation verification (e.g., correlated vulnerability yielding memory disclosure across multiple environments).

Residual impact:

- *Liveness failure:* Settlements halt; users must wait for timeouts and refunds. Funds remain locked but safe.

- *Safety degradation (worst case)*: If correlated failure enables $\geq t$ share compromise, attacker gains signing authority. Blast radius bounded by on-chain constraints.

Detection mechanism: Provider status pages; third-party cloud monitoring services; direct health checks from independent infrastructure; cross-provider incident correlation in ops tooling.

Operational response: (1) Automatic failover to remaining providers if single-provider failure; (2) If multi-provider failure detected, pause settlement and communicate status to users; (3) Funds remain locked (safe) until operations resume; (4) User refund available after LOCK_DURATION regardless of NearFall operational status; (5) If correlated compromise is suspected, halt signing and initiate key rotation ceremony.

Acceptance criteria note: Liveness failure meets all acceptance criteria (bounded, detectable, response defined). The safety degradation scenario ($\geq t$ share compromise) is low-likelihood and would require extraordinary coordination—it is accepted as a tail risk bounded by on-chain constraints.

MPC Protocol Implementation Bug

Risk: The FROST MPC implementation or distributed signing coordination contains an undiscovered bug causing safety or liveness failure.

Why accepted: FROST threshold Schnorr is newer than threshold ECDSA with fewer production deployments. Distributed systems have subtle failure modes (state desync, consensus bugs, race conditions). Audits and testing reduce but cannot eliminate implementation bugs.

Impact boundary:

- *Liveness failure*: Signing halts; settlements cannot complete; users must wait for timeouts and refunds.
- *Safety failure (worst case)*: Bug bypasses policy checks, allowing unauthorized signatures. Blast radius bounded by on-chain constraints (caps, rate limits, finality requirements).

Mitigation depth: Multiple audits of FROST implementation. Integration testing under chaos conditions (§3.5). Canary deployments before full rollout. Formal verification of critical signing paths where feasible.

Detection mechanism: Signing anomaly detection (volume spikes, unexpected digest patterns, policy violations). MPC node health monitoring. Cross-node consistency checks. Audit log analysis for unexpected state transitions.

Operational response:

- If signing anomaly detected, halt signing immediately
- Quarantine affected signing key
- Investigate root cause with MPC node logs
- If safety violation confirmed: rotate shares, notify affected users, assess blast radius
- Rollback deployment to last known good version
- Resume only after root cause addressed and verified

Acceptance criteria note: Liveness failure meets all acceptance criteria. Safety failure is bounded by on-chain constraints and is accepted as a tail risk given the absence of production-grade alternatives for BIP-340

threshold signing.

6.5.4 Residual Risk Tripwires

Residual risks cannot be eliminated, but they can be monitored. The following tripwires convert §6.5 from philosophy into operational policy.

Tripwire	Trigger Condition	Response
TEE security bulletin	Vendor publishes advisory for deployed platform (SGX/SEV-SNP/TDX)	Evaluate applicability → if exploitable: pause intake, rotate to patched hosts, resume after validation
Proof system CVE	Security advisory for Halo2, Sirius, or underlying cryptographic primitives	Assess NearFall exposure → if exploitable: pause settlement, freeze new locks, allow refunds, upgrade
MPC signing anomaly	Unusual signing volume, unexpected digest patterns, policy violations	Quarantine affected signing key → pause release → investigate → rotate shares if compromise suspected
Correlated infrastructure event	Multiple cloud providers report simultaneous issues affecting TEE or attestation	Halt signing → assess scope → if $\geq t$ shares potentially affected: initiate key rotation ceremony
Attestation verification failure spike	Elevated rate of attestation rejections across multiple enclaves	Investigate root cause → if platform-level: pause until resolved; if targeted: quarantine affected enclaves

Monitoring requirements: These tripwires require active monitoring of vendor security channels, internal anomaly detection, and cross-provider incident correlation. Operational runbooks must define escalation paths and authorized responders for each tripwire.

6.5.5 Unknown Unknowns

The preceding sections enumerate *known* residual risks. By definition, we cannot enumerate unknown unknowns. However, we can characterize the attack surface where unknown risks are most likely to emerge.

High-complexity boundaries (most likely to harbor unknown risks):

Boundary	Complexity Source	Unknown Risk Likelihood
TEE ↔ MPC attestation	Multiple cloud providers, attestation formats, verification logic	Medium-High
ZK proof generation	Complex cryptographic stack, curve arithmetic, constraint systems	Medium-High
Cross-chain finality observation	Two chains with different finality models, reorg handling	Medium
FROST MPC implementation	Threshold Schnorr is newer than threshold ECDSA; fewer production deployments	Medium

Lower-complexity boundaries (less likely to harbor unknown risks):

Boundary	Why Lower Risk
Ethereum custody contract	Standard Solidity patterns; heavily audited ecosystem
Intent serialization	Simple data structures; well-understood attack surface
Batch nonce enforcement	Simple integer comparison; hard to get wrong

Response to unknown unknowns: Defense-in-depth is designed to contain unknown risks. If an unknown vulnerability is discovered in one layer, the other layers should prevent immediate fund theft. The separation of TEE (confidentiality) and MPC (authorization) means a vulnerability in one does not automatically compromise the other.

6.5.6 Risk Acceptance Criteria

A risk is accepted (not merely deferred) when all of the following are true:

Criterion	Meaning
Elimination is infeasible	No known mitigation can reduce the risk to zero without unacceptable tradeoffs
Impact is bounded	The risk does not enable unbounded fund theft; impact is limited to specific scenarios
Detection is possible	If the risk materializes, it can be detected (even if not prevented)
Response is defined	An operational response exists for when the risk materializes

Risks that fail these criteria should not be accepted—they should be mitigated or the system should not be deployed.

Note on foundational risks: Some risks (ZK soundness, L1 consensus) affect the system's foundational assumptions. For these, "bounded impact" means bounded by the scope of the assumption and by architectural controls (TVL caps, batch limits, atomicity protocol), not eliminated by them. A ZK soundness bug is bounded by Midnight-side TVL during the vulnerability window; it cannot directly drain Ethereum custody. These risks are accepted because the alternatives (not using ZK proofs, not relying on L1 consensus) eliminate the functionality that justifies the system's existence.

Example: Applying acceptance criteria to TEE-4 (Side-Channel Leakage)

Criterion	TEE-4 (Side-Channel)
Elimination infeasible	■ Novel side-channels emerge regularly; perfect elimination impossible
Impact bounded	■ Information disclosure only; signing requires MPC
Detection possible	■ Security research community publishes new attacks
Response defined	■ Evaluate, patch, rotate infrastructure

TEE-4 meets all criteria and is correctly accepted.

6.5.7 Residual Risk Communication

Users and integrators should understand what NearFall does and does not protect against. The following statements are appropriate for external communication:

What NearFall protects against:

- Single operator seeing intent plaintext (TEE confidentiality)
- Single operator signing unauthorized transactions (MPC threshold)
- Invalid settlements reaching Midnight (ZK verification)
- Cross-chain double-spend under normal conditions (atomicity protocol)
- TEE rollback/forking attacks (anti-rollback mechanism)

What NearFall does NOT protect against:

- Coordinated compromise of TEE + MPC threshold (the residual risk floor)
- Global TEE platform compromise (vendor-level attack)
- MPC supermajority collusion (threshold security assumption)
- Underlying chain consensus failure (L1 dependency)
- Novel zero-day vulnerabilities in any component (unknown unknowns)

Appropriate user-facing language:

> "NearFall provides privacy-preserving cross-chain settlement secured by hardware enclaves and threshold signatures. No single operator can see your trade details or move your funds. However, like all systems, NearFall cannot protect against coordinated attacks by multiple operators or fundamental compromise of the underlying hardware and blockchain infrastructure."

Key message for technical audiences:

> "NearFall is designed so that compromise of any single operator or single layer is insufficient for theft; coordinated compromise of multiple independent operators remains the residual risk."

6.5.8 What This Section Establishes

Claim	Status
Residual risk floor explicitly defined (TEE + MPC threshold compromise with on-chain constraints)	■ Established
Floor vs. non-goal distinction clarified	■ Established
Explicit non-goals categorized (infrastructure failures, threat model exclusions, threshold properties)	■ Established
Accepted risks justified with bounded impact analysis (TEE-4, ZK soundness, correlated cloud, MPC implementation)	■ Established

Claim	Status
Foundational risk criteria clarified	■ Established
Residual risk tripwires defined (5 operational triggers)	■ Established
Unknown unknown boundaries characterized	■ Established
Risk acceptance criteria defined (4 criteria + foundational risk note)	■ Established
User-facing communication guidance provided	■ Established

Cross-references: SC-5 (ZK Soundness), OP-6 (Correlated Cloud), and OP-7 (MPC Implementation) are registered in §6.3 Risk Register with Status: Accepted.

Design invariant (final): NearFall's security claims are conditional on:

- TEE hardware functioning as specified (no global platform compromise)
- MPC threshold holding (fewer than t nodes compromised)
- Underlying chains (Midnight, Ethereum) maintaining consensus
- No unknown soundness vulnerabilities in the ZK proof system

Violations of these conditions are explicitly excluded from NearFall's security guarantees. This is not a limitation of NearFall's design; it is an honest statement of what any system built on these primitives can and cannot provide.

Part VI complete. Sections §6.1–§6.5 cover risk overview, cross-chain atomicity, risk register, anti-rollback mechanism, and residual risk acknowledgment. Together they provide the security analysis required for informed deployment decisions.